TW128: A Tree-Structured Duplex Authenticated Encryption Mode

Coda Hale

Hale Institute for Bicycle Appreciation

Abstract. AEAD tags are often expected to be compact commitments to the full encryption context, an intuition reinforced by common experience with hash-based MACs such as HMAC. The AEAD literature has only recently begun to formalize this expectation through notions such as committing security and encryptment, but standard deployed modes such as AES-GCM and ChaCha20-Poly1305 do not provide it. We present TW128, a nonce-based authenticated encryption mode with associated data that adapts the KangarooTwelve tree-hashing pattern to keyed duplex encryption over Keccak-p[1600, 12]. A serial root duplex binds the key, nonce, and associated data, encrypts the first message chunk, and aggregates tags from independently keyed leaf duplexes for the remaining chunks. The leaf decomposition is canonical, but the number of leaves evaluated at once is an implementation choice: scalar, SIMD, and multicore implementations produce identical ciphertexts and tags without making the degree of parallelism a mode parameter.

The final root tag is the authentication tag and is also analyzed as a digest-like commitment to the full (K, U, A, M) context. In the public permutation model, via an intermediate random oracle and the flat sponge lift, we prove concrete multi-user indistinguishability and derive all-in-one authenticated encryption as the single-user corollary. We also prove integrity under release of unverified plaintext in the sense of [ABL+14]. Separately, we prove that the wrapper returning the tag is a secure hash of the full context—collision-, second-preimage-, and preimage-resistant in the standard hash-function sense. The same tag analysis gives the AEAD-facing committing results, including full-input CMTDs-4 committing security. At the implemented parameters, all stated notions meet a 128-bit security target. Vectorized `amd64` and `arm64` implementations recover the scalable performance profile of the tree design: in bulk they exceed 16 Gbit/s, with the AVX-512 and `arm64` paths reaching 51–76% of hardware-accelerated AES-128-GCM throughput.

Keywords: authenticated encryption · committing security · sponge constructions · duplex constructions · Keccak

1 Introduction

Authenticated encryption with associated data (AEAD) is often treated in practice as if its tag were a compact commitment to the full encryption context: a short string that binds the key, nonce, associated data, and message. This expectation is natural for users who first encounter message authentication through hash-based MACs such as HMAC, but it is not a property of standard deployed AEADs. AES-GCM and ChaCha20-Poly1305, for example, are not key-committing: an adversary can craft a single ciphertext that decrypts to valid plaintexts under two or more keys [DGRW18, ADG+22]. The cause is structural. Their authenticity rests on algebraic MACs whose outputs can often be made to validate under a second context, whereas a digest is expected to resist both collisions and preimages.

The gap is visible in deployed protocols and concrete attacks, not only in definitions. The "invisible salamander" attack defeats Facebook's attachment franking scheme [DGRW18]; related constructions produce ciphertexts that open to different well-formed files under different keys [ADG+22]; partitioning oracles recover passwords from systems built on non-committing AEAD [LGR21]. These attacks differ in surface form, but all expose the same mismatch: the tag is used as if it were a full-context commitment, while the mode only proves ordinary authenticity.

Recent work has begun to model this expectation directly. Compactly committing AEAD was introduced for message franking [GLR17], and the encryptment approach of [DGRW18] builds commitment into a dedicated primitive. The committing hierarchy of [BH22] makes the collision-style requirement explicit: no two distinct contexts should open the same ciphertext and tag. These notions explain when a ciphertext can have another valid opening, but they are still phrased as AEAD games over ciphertexts. The assumption made by protocols is often more tag-centric: the tag itself is treated as a compact commitment string. This paper therefore analyzes the tag with hash notions directly, and then recovers the AEAD commitment notions as game-specific consequences of the same tag analysis.

TW128 (*TreeWrap*, with 128 denoting the 128-bit security target) takes this tag-centric view as a design constraint. If the authentication tag is to behave like a compact digest of the encryption context, then the mode should have a digest-like root rather than a serial stream cipher followed by an algebraic MAC. TW128 adapts the KangarooTwelve [BDP+16] tree-hashing pattern to keyed duplex encryption over Keccak-p[1600, 12]: a serial root duplex binds the key, nonce, associated data, first message chunk, and leaf tags, while independently keyed leaf duplexes encrypt the remaining chunks. As in KangarooTwelve's *kangaroo hopping*, a single-chunk message uses only the root duplex and pays no tree overhead (Section 8.4).

The construction fixes the root/leaf transcript, but not the amount of parallelism used by an implementation. A scalar implementation, an AVX2 implementation, an AVX-512 implementation, a NEON implementation, and a multicore implementation all produce identical ciphertexts and tags. The degree of parallelism is therefore not a mode parameter and is not visible on the wire. This separates TW128 from prior duplex-based AEADs with parallel variants, where the parallelism degree is fixed by the mode instance or otherwise shared by the communicating parties.

We prove that the wrapper $\mathsf{H}_{\mathsf{TW128}}$ returning the root tag is a secure hash of the full (K, U, A, M) input in the sense of [RS04], and use the same root tag analysis to obtain the AEAD-facing committing results. We also prove ordinary AEAD security: in the public permutation model, TW128 has a concrete multi-user indistinguishability bound [BT16], with all-in-one authenticated encryption [RS06] as the single-user corollary. For integrity, the same root-centered structure gives INT-RUP security [ABL+14]: even if an adversary can obtain unverified plaintexts, it cannot make verification accept a fresh ciphertext except within the same tag-guessing and collision terms. At the implemented parameters ($k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$), the bounds meet a 128-bit security target for adversaries running up to $N_{\mathsf{tot}} \le 2^{63}$ Keccak-p[1600, 12] calls—on the order of 2^{71} bytes of processed data. Optimized `amd64` and `arm64` implementations preserve the canonical transcript while evaluating leaf duplexes in batches; in bulk, TW128 exceeds 16 Gbit/s and reaches 51–76% of hardware-accelerated AES-128-GCM throughput (Section 8).

1.1 Our Contributions

- **A scalable tree AEAD mode.** We specify TW128, a nonce-based AEAD with associated data built as a two-level tree of strict [BDPVA11] duplexes over Keccak-p[1600, 12] (Section 3). The mode fixes the root/leaf transcript but delegates the number of leaves evaluated simultaneously to the implementation, so scalar and vectorized implementations interoperate without changing ciphertexts or tags.

- **Digest-like commitment.** We prove that the wrapper $\mathsf{H}_{\mathsf{TW128}}$ returning the root tag is a secure hash of the full (K, U, A, M) input in the sense of [RS04]: collision-, second-preimage-, and preimage-resistant (Theorems 8 to 10 and Corollary 3). Thus the authentication tag behaves like the compact digest practitioners often expect an AEAD tag to be.

- **AEAD commitment consequences.** We instantiate the same root tag analysis in the existing AEAD vocabulary. The all-bodies-equal structure of CMTDs-4 gives the full-input s-way committing bound without the leaf-collision slack of the general hash collision theorem [BH22] (Corollary 5), and the remaining CMT/CMTD notions for $\ell \in \{1, 3, 4\}$ follow from the committing hierarchy and direct source-game arguments (Corollary 6).

- **Multi-user, AE, and RUP integrity.** We give a concrete multi-user indistinguishability bound [BT16] for D users (Theorem 2). The $D = 1$ case yields a concrete all-in-one AE bound [RS06] in the public permutation model (Corollary 1). We also prove INT-RUP integrity [ABL+14], with plaintext-computing oracle work charged through the same transcript resources (Theorem 4). The proof proceeds through a dedicated TW128 random oracle model and is transferred to the public permutation setting by the [BDPVA08] flat sponge lift.

- **Optimized implementations.** We describe and evaluate vectorized `amd64` (AVX-512 and AVX2) and `arm64` (NEON with the SHA-3 extension) implementations whose batched leaf cores realize the mode's scalable parallelism in practice (Section 8).

1.2 Related Work

Committing AEAD. The study of committing authenticated encryption was initiated by [GLR17], who introduced compactly committing AEAD for message franking, and by [DGRW18], who built it from a dedicated primitive (encryptment). [BH22] organized the collision-style notions into the CMT-1/3/4 and CMTD hierarchy and gave the UtC, RtC, and HtE transforms; [CR22] gave the CTX transform, which commits a nonce-based AE scheme to its full context by adding a hash. These transforms are efficient and broadly applicable, but they treat commitment as an external layer. TW128 instead makes the authentication tag itself the hash-like commitment: the root tag is the compact commitment, and the root tag analysis gives both the [BH22] full-input committing consequence. Closest in spirit is the recent work of [DHM+24], which also builds nonce-based AE natively on SHA-3 functions and obtains CMT-4 from collision resistance; it targets session-based communication through a sequential duplex chain, whereas TW128 is a single scalable tree for standalone AEAD (Section 9.3).

Sponge and duplex AEADs. Permutation-based authenticated encryption built on the sponge and duplex constructions of [BDPVA08, BDPVA11] is well established. Keyak [BDP+15] uses the Motorist mode over round-reduced Keccak-p and already admits parallel duplex instances; Ascon [DEMS21] and Xoodyak [DHP+20] are serial duplex AEADs over smaller permutations. TW128 shares the single-permutation, constant-state duplex foundation but differs in where parallelism lives. Keyak fixes the degree of parallelism as a mode parameter, while Ascon and Xoodyak expose no mode-level parallelism. In TW128, the decomposition into indexed leaves is fixed by the transcript, but the number of leaves evaluated at once is chosen by the implementation and is invisible to the wire format (Section 9.2).

Release of unverified plaintext. Andreeva et al. [ABL+14] formalized authenticated encryption in the release-of-unverified-plaintext setting by separating plaintext computation from verification. They define INT-RUP for integrity and plaintext awareness notions for privacy, and show that ordinary ciphertext integrity does not imply INT-RUP. TW128 targets the integrity side of this model: the released stream plaintext is not claimed to be private, but it does not reveal the hidden tag values needed to make a fresh ciphertext verify.

Tree hashing. TW128's two-level shape is inspired by KangarooTwelve [BDP+16], a tree hash over the same permutation. TW128 adopts its choice of permutation and final-node/leaf split and adapts them to authenticated encryption: each leaf chunk is encrypted and tagged independently, leaf tags replace unkeyed chaining values, and the root is keyed by (K, U). The construction also inherits KangarooTwelve's short-message behavior: inputs fitting in one chunk use only the root duplex. Because TW128 has a fixed two-level topology with independently keyed, indexed leaves and phase-separated duplex calls, it does not need KangarooTwelve's Sakura tree encoding [BDPVA14] to make its transcripts unambiguous (Section 3).

1.3 Paper Organization

Section 2 recalls the notation, sponge and duplex constructions, nonce-based AEAD and RUP integrity notions, committing security [BH22], and the hash notions of [RS04]. Section 3 specifies the TW128 construction and parameters. Section 4 fixes the scope of the security claims, the public permutation games, intermediate random oracle model, resource accounting, and loss terms. Section 5 proves multi-user indistinguishability, all-in-one AE security, and INT-RUP integrity. Section 6 proves root tag hash security and committing security, and Section 7 summarizes the concrete bounds. Section 8 reports implementation performance, and Section 9 compares TW128 to prior AEAD and committing constructions and discusses limitations outside the formal scope. The appendices collect structural lemmas, the flat sponge lift, vectorized kernel details, and test vectors.

2 Preliminaries

2.1 Notation

Strings. Write $\{0,1\}$ for the binary alphabet. For an integer $n \geq 0$, $\{0,1\}^n$ denotes the set of n-bit strings, $\{0,1\}^* = \bigcup_{n\geq 0}\{0,1\}^n$ the set of all finite-length bit strings, and ε the empty string. For $x \in \{0,1\}^*$, $|x|$ is the bit length and $x \parallel y$ is the concatenation; the bitwise XOR of two equal-length strings $x, y \in \{0,1\}^n$ is $x \oplus y$. A *byte string* is an element of $(\{0,1\}^8)^*$; its byte length is denoted $\|x\| = |x|/8$. For an integer $0 \leq n < 2^{64}$, $\langle n\rangle_8 \in \{0,1\}^{64}$ is its eight-byte little-endian encoding. For a finite or infinite bit string x and $0 \leq \tau \leq |x|$ when x is finite, $\lfloor x\rfloor_\tau \in \{0,1\}^\tau$ denotes the leading τ bits of x.

Sampling and probability. We write $x \leftarrow\!\!\$\; S$ for sampling x uniformly at random from a finite set S, with implicit coins independent across samples. For an event E in a probability experiment, $\Pr[E]$ denotes its probability.

Sets of maps. For a set S, $\mathsf{Perm}(S)$ denotes the set of permutations on S; for a positive integer b, $\mathsf{Perm}(\{0,1\}^b)$ is the set of b-bit permutations.

Games and advantages. We write $\mathcal{A}^{O_1,\dots,O_k}$ for a (possibly randomized) adversary with oracle access to $O_1,\dots,O_k$, and $\Pr[\mathsf{G}^{\mathcal{A}} \Rightarrow 1]$ for the probability that a game G—an experiment that runs $\mathcal{A}$ against specified oracles and returns a Boolean outcome—returns 1. For a distinguishing experiment between two games $\mathsf{G}_0, \mathsf{G}_1$, the advantage of $\mathcal{A}$ against a scheme Π in notion X is the absolute difference

$$\mathbf{Adv}^{\mathsf{X}}_{\Pi}(\mathcal{A}) = \left|\Pr[\mathsf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathsf{G}_1^{\mathcal{A}} \Rightarrow 1]\right|.$$

For a win-event experiment without a hidden challenge bit it is instead $\Pr[\mathsf{G}^{\mathcal{A}} \text{ wins}]$, the probability the game returns 1; a bit-guessing experiment with a hidden challenge bit uses the rescaled form $\left|2\Pr[\mathsf{G}^{\mathcal{A}} \text{ wins}] - 1\right|$, as in the multi-user notion of Section 2.4.

Concrete security. All bounds in this paper are concrete: each advantage is an explicit function of the resource caps introduced in Section 4.4. We make no use of asymptotic or negligibility conventions.

Reference summary. Table 1 summarizes the resource caps and loss terms used throughout. The caps are defined formally in Section 4.4, the loss terms in Section 4.5.

Table 1: Resource caps and loss terms.

Symbol	Meaning	Defined
N	Construction interface cost (Keccak-p[1600, 12] calls)	§ 4.4
N_{prim}	Direct primitive-interface queries	§ 4.4
N_{tot}	$N + N_{\mathsf{prim}}$ (total query-sequence cost)	§ 4.4
q_{v}, Q_{v}	Valid verification queries / cap	§ 4.4
q_{RO}, Q_{RO}	Direct random oracle queries / cap (RO-model)	§ 4.4
n_{leaf}, N_{leaf}	Distinct construction-generated final leaf transcripts / cap	§ 4.4
$\mathsf{Lift}_c(N_{\mathsf{tot}})$	[BDPVA08] flat sponge loss	§ 4.5
$\mathsf{KeyGuess}_k(Q)$	Ideal system key guess	§ 4.5
$\mathsf{RootColl}_\tau(Q, s)$	s-way root-collision	§ 4.5
$\mathsf{RootPre}_\tau(Q)$	Root-preimage	§ 4.5
$\mathsf{LeafColl}_\tau(Q)$	Known-key leaf-collision	§ 4.5

2.2 Sponges and Duplexes over Keccak-*p*

The Keccak-*p* permutation. The Keccak-p family of permutations, introduced as part of the Keccak design and standardized within FIPS 202 [Nat15], is parameterized by a state width b and a round count n_r; we write $\text{Keccak} - p[b, n_r] \in \mathsf{Perm}(\{0,1\}^b)$ for the corresponding permutation. TW128 uses Keccak-p[1600, 12], the same primitive as KangarooTwelve [BDP+16], fixed throughout. In the public permutation security model of Section 4.1 and the flat sponge lift of Section B, this permutation is modeled as a uniformly random element of $\mathsf{Perm}(\{0,1\}^{1600})$.

The sponge construction. Following [BDPVA08], fix a permutation $\pi \in \mathsf{Perm}(\{0,1\}^b)$, a *sponge-compliant* padding rule pad, and a rate $1 \le r < b$ with capacity $c = b - r$. A padding rule appends $\mathsf{pad}[r](|M|)$, a bit string determined by the input length and the rate, to an input M, extending it to a sequence of r-bit blocks; it is sponge-compliant [BDPVA11, Definition 1] if $M \parallel \mathsf{pad}[r](|M|)$ is never empty and, for all $M \neq M'$ and all $n \ge 0$, $M \parallel \mathsf{pad}[r](|M|) \neq M' \parallel \mathsf{pad}[r](|M'|) \parallel 0^{nr}$. The sponge function $\mathsf{Sponge}[\pi, \mathsf{pad}, r] : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$ maps an input M and an output length ℓ to an ℓ-bit string by absorbing $M \parallel \mathsf{pad}[r](|M|)$ into the all-zero b-bit state in r-bit blocks (each separated by an application of π) and squeezing ℓ bits in r-bit blocks from the resulting state. [BDPVA08, Theorem 2] shows that $\mathsf{Sponge}[\pi, \mathsf{pad10}^*, r]$, where the simple rule $\mathsf{pad10}^*$ appends a single 1 bit followed by the minimum number of 0 bits reaching a multiple of r, is indifferentiable from a random oracle when π is uniformly random. We bound its distinguishing advantage by $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1}$, derived in Lemma 18 from the product upper bounds of [BDPVA08, Lemmas 4 and 5].

The duplex construction. The *duplex* construction of [BDPVA11] extends the sponge to a stateful interface. A duplex object $\mathsf{Duplex}[\pi, \mathsf{pad}, r]$ is initialized to the all-zero state. Each call $\mathsf{duplexing}(\sigma, \ell)$, with $\ell \le r$, absorbs an input string σ of at most $\rho_{\max}(\mathsf{pad}, r)$ bits via π and returns the first ℓ bits of the resulting state. Here $\rho_{\max}(\mathsf{pad}, r) = \min\{x : x+|\mathsf{pad}[r](x)| > r\}-1$ is the largest input length for which the padded input fits in a single r-bit block; for the $\mathsf{pad10^*1}$ rule, $\rho_{\max}(\mathsf{pad10^*1}, r) = r-2$ [BDPVA11]. [BDPVA11, Lemma 3] establishes the duplex-to-sponge equality: for any call sequence $((\sigma_0, \ell_0), \ldots, (\sigma_i, \ell_i))$, the i-th output equals

$$\mathsf{Sponge}[\pi, \mathsf{pad}, r]\big(\sigma_0 \parallel \mathsf{pad}_0 \parallel \cdots \parallel \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad}[r](|\sigma_j|)$ is the padding the duplex appends in its j-th call; the flattened input carries these interior paddings explicitly, and the sponge appends only the final pad_i. The flattening map is injective by [BDPVA11, Lemma 4]: the duplex input-string sequence is recoverable from the flattened sponge input.

TW128 instantiation. TW128 fixes the padding rule $\mathsf{pad10^*1}$ and the rate $r = 1344$, so the capacity is $c = 256$ bits. The largest input length per duplex call is therefore $\rho_{\max}(\mathsf{pad10^*1}, 1344) = 1342$ bits.

Bridge to the H-input domain. The indifferentiability theorem of [BDPVA08] is stated for the simpler $\mathsf{pad10}^*$ padding over its unpadded H-input domain, while TW128 uses $\mathsf{pad10^*1}$ internally. [BDPVA11, Theorem 3] supplies an injective preprocessing bridge $I[r, r_{\max}]$ between the two domains. For TW128, $r = r_{\max} = 1344$, so the bridge reduces to appending $1 \parallel 0^q$ with q determined by the input length modulo $r_{\max}$; on the bridge image, q is recovered as the trailing-zero count. Section 4.3 instantiates this bridge on typed TW128 duplex transcript prefixes as the map $\mathsf{flat}(\cdot)$ used by the random oracle proofs throughout the paper.

2.3 Authenticated Encryption with Associated Data

Syntax. A *nonce-based authenticated encryption scheme with associated data* (AEAD scheme) is a pair $\mathsf{SE} = (\mathsf{Enc}, \mathsf{Dec})$ of deterministic algorithms with associated key space $\mathcal{K} \subseteq (\{0,1\}^8)^*$, nonce space $\mathcal{U} \subseteq (\{0,1\}^8)^*$, associated data space $\mathcal{A} \subseteq (\{0,1\}^8)^*$, message space $\mathcal{M} \subseteq (\{0,1\}^8)^*$, and ciphertext space $\mathcal{C} \subseteq (\{0,1\}^8)^*$. The encryption algorithm $\mathsf{Enc} : \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M} \to \mathcal{C}$ produces a ciphertext $C = \mathsf{Enc}_K(U, A, M)$. The decryption algorithm $\mathsf{Dec} : \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{C} \to \mathcal{M} \cup \{\perp\}$ returns either a plaintext or the rejection symbol $\perp$. The *tag length in bytes* is the constant $\|\mathsf{Enc}_K(U, A, M)\| - \|M\|$, fixed by the scheme. We write nonce values as U to keep N available for the construction-cost resource variable used in the concrete bounds. In this generic syntax, C denotes the full AEAD ciphertext, including any tag material. The TW128 algorithm section (Section 3) writes this same value as $C \parallel T$, with C the ciphertext body and T the root tag.

Correctness. For every $(K, U, A, M) \in \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M}$, $\mathsf{Dec}_K(U, A, \mathsf{Enc}_K(U, A, M)) = M$. An AEAD scheme is *tidy* if every accepting decryption is consistent with encryption: whenever $\mathsf{Dec}_K(U, A, C) = M \neq \perp$, one has $\mathsf{Enc}_K(U, A, M) = C$.

All-in-one AE. We use the all-in-one authenticated encryption notion of [RS06], instantiated for nonce-based AE following [BT16]. The experiment grants either the real pair $(\mathsf{Enc}_K, \mathsf{Ver}_K)$ or the ideal pair $(\mathsf{Rand}, \mathsf{Replay})$, where Rand returns a fresh uniform byte string of the ciphertext length on each encryption query and Replay accepts exactly the tuples Rand produced. The advantage $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{SE}}(\mathcal{A})$ is the distinguishing advantage between these two worlds over nonce-respecting adversaries. This all-in-one notion is instantiated for $\mathsf{TW128}$ in Section 4.1.

Integrity under release of unverified plaintext. For the RUP setting of [ABL+14], a conventional decryption algorithm is separated into a plaintext-computing algorithm and a verification algorithm. The integrity notion $\mathsf{INT\text{-}RUP}$ gives the adversary access to encryption, plaintext computation, and verification, and asks whether verification accepts a fresh ciphertext and tag not returned by encryption. As in the nonce-IV setting of [ABL+14], encryption queries are nonce-respecting, while plaintext computation and verification may be queried on arbitrary nonces. This notion protects only ciphertext integrity; privacy under release of unverified plaintext requires plaintext awareness in addition to ordinary encryption privacy.

2.4 Multi-User Indistinguishability

The multi-user indistinguishability notion of [BT16, Fig. 3] extends the same real-vs-ideal oracle pair to an adversary-selected set of users. The number of users u is adversary-determined: it is the number of New queries the adversary makes, i.e., the final value of the user counter v in the game below. A challenge bit $b \leftarrow\$ \{0,1\}$ selects between the real construction interface (real ciphertexts, real verification) and an ideal interface (uniform strings of matching length, verification that accepts only previously-issued encryption tuples). The adversary also accesses a public primitive interface: the real permutation (π, π^{-1}) in the real world and the simulator $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ in the ideal world. The adversary outputs a guess b' and wins if $b' = b$.

Game and advantage. The game $\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A})$ of [BT16, Fig. 3] samples $b \leftarrow\$ \{0,1\}$ and maintains a user counter v, a set V of (d, U) pairs already used by encryption, and a set W of recorded encryption tuples; its New, Enc, and Vf oracles, together with the public primitive interface—the real permutation when $b = 1$ and the simulator when $b = 0$—are instantiated for $\mathsf{TW128}$ in Section 4.2. We write the user index as d and the associated data input as A, where [BT16] writes i and H, and take the absolute value of their signed advantage so that

$$\begin{aligned}\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) &= \left|2\Pr[\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) \text{ returns } 1] - 1\right| \\ &= \left|\Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=1} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=0} \Rightarrow 1]\right|\end{aligned}$$

the second equality holding because b is uniform. Thus $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}$ is the two-game distinguishing advantage between the real $(\mathsf{Enc}, \mathsf{Ver})$ pair and the ideal $(\mathsf{Rand}, \mathsf{Replay})$ pair instantiated per user. The single-user case $u = 1$ recovers the all-in-one AE notion of Section 2.3, augmented with the public primitive interface (the real permutation in the real world and the simulator in the ideal world).

Per-user nonce uniqueness. Nonce uniqueness is enforced per user via the set V: the same nonce may appear under distinct users but not twice under the same user. This is strictly weaker than the global nonce-uniqueness requirement that would force each nonce value U to appear under at most one user.

2.5 Committing Notions and Root Tag Hash Security

[BH22] formalize a family of committing security notions for nonce-based AEAD schemes, parameterized by how much of the encryption input is committed and by whether commitment is established through decryption (the D-notions) or through encryption agreement (the E-notions). We recall their definitions and the s-way generalization, and then frame the root tag itself as a hash function whose [RS04] security—collision, second-preimage, and preimage resistance—captures the binding and preimage properties expected of a compact commitment. The AEAD committing notions are then proved from the same tag analysis in Section 6.

What-is-committed functions. For $\ell \in \{1, 3, 4\}$, the *what-is-committed* function WiC_ℓ projects an encryption input $(K, U, A, M) \in \mathcal{K} \times \mathcal{U} \times \mathcal{A} \times \mathcal{M}$ to the portion it should commit:

$$\begin{aligned}\mathsf{WiC}_1(K, U, A, M) &= K,\\ \mathsf{WiC}_3(K, U, A, M) &= (K, U, A),\\ \mathsf{WiC}_4(K, U, A, M) &= (K, U, A, M).\end{aligned}$$

CMTDs-ℓ and CMTs-ℓ games. For $s \geq 2$, [BH22, Fig. 5] defines the s-way committing games on tuples $(K_i, U_i, A_i, M_i)_{i=1}^{s}$ with pairwise distinct WiC_ℓ-images, all entries in the scheme's syntactic domain and none equal to $\perp$. The s-way CMTDs-ℓ (decryption) game has the adversary output a ciphertext C alongside the tuples and returns 1 if $\mathsf{Dec}_{K_i}(U_i, A_i, C) = M_i$ for every i; the printed return line of [BH22, Fig. 5] writes C_i, but the game outputs a single ciphertext C. The s-way CMTs-ℓ (encryption) game has the adversary output the tuples alone and returns 1 if all encryptions $\mathsf{Enc}_{K_i}(U_i, A_i, M_i)$ are equal. Advantages $\mathbf{Adv}_{\mathsf{SE}}^{\mathsf{CMTDs}\text{-}\ell}(\mathcal{A})$ and $\mathbf{Adv}_{\mathsf{SE}}^{\mathsf{CMTs}\text{-}\ell}(\mathcal{A})$ are the probabilities that the adversary wins the respective game; the parameter s is suppressed from the notation when clear from context. The two-tuple case $s = 2$ is the original CMTD-ℓ/CMT-ℓ pair of [BH22, Fig. 4]. These definitions are instantiated for $\mathsf{TW128}$ in Section 4.1, with the public permutation lift deferred to that section.

Definition 1 (Root tag wrapper)**.** For every $\mathsf{TW128}$ encryption input $X = (K, U, A, M)$ in the scheme domain, define the root tag wrapper by

$$\mathsf{H}_{\mathsf{TW128}}(K, U, A, M) = T \quad \text{where} \quad \mathsf{Enc}_K(U, A, M) = C \parallel T.$$

We also write $\mathsf{H}_{\mathsf{TW128}}(X)$ for $\mathsf{H}_{\mathsf{TW128}}(K, U, A, M)$. Equivalently, $\mathsf{H}_{\mathsf{TW128}}$ runs $\mathsf{TW128}$ encryption and discards the ciphertext body.

The root tag as a hash. Many uses of commitment store, transmit, or compare a short tag rather than the full ciphertext, and ask that the short string alone be binding—the compactly committing AEAD goal introduced by [GLR17] for message franking. $\mathsf{TW128}$ ciphertexts are written $C \parallel T$ with T the τ_{root}-bit root tag, and we capture this goal directly by treating Definition 1 as a hash function. Keyed by the permutation, this is a hash family, and we ask that it satisfy the hash-function security notions of [RS04], over the $\mathsf{TW128}$ input domain, together with the local s-way multicollision extension used for the committing theorems. For fixed byte lengths a and μ, write

$$\mathcal{X}_{a,\mu} = \{0,1\}^k \times \{0,1\}^\nu \times (\{0,1\}^8)^a \times (\{0,1\}^8)^\mu$$

for the fixed-length slice of valid inputs, with bit length $m_{a,\mu} = k + \nu + 8a + 8\mu$.

Collision resistance (Coll). No adversary finds distinct inputs $X_1, \ldots, X_s$ (any $s \geq 2$) with $\mathsf{H}_{\mathsf{TW128}}(X_1) = \cdots = \mathsf{H}_{\mathsf{TW128}}(X_s)$. The case $s = 2$ is the RS04 Coll notion; larger s is the corresponding multicollision extension.

Preimage resistance (Pre). For fixed a, μ, given $\mathsf{H}_{\mathsf{TW128}}(X^*)$ for a uniform $X^* \leftarrow\!\!\$\ \mathcal{X}_{a,\mu}$, no adversary finds any valid input X with $\mathsf{H}_{\mathsf{TW128}}(X) = \mathsf{H}_{\mathsf{TW128}}(X^*)$.

Second-preimage resistance (Sec/eSec). The standard and everywhere forms of second-preimage resistance of [RS04] follow from collision resistance by [RS04, Proposition 6].

Everywhere preimage resistance (ePre). For a target tag T^* fixed in advance, no adversary finds X with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$.

The advantages $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}$, $\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}$, and $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}$ are the respective success probabilities, instantiated for TW128 in Section 4.1. The Pre guarantee is proved directly in Theorem 9, while the second-preimage guarantees are inherited from Coll in Corollary 3. Coll is the binding property of the tag taken as a compact commitment to (K, U, A, M), Pre is the random-opening preimage property on fixed input slices, and ePre is the fixed-target preimage property for query-independent tags. The [BH22] committing notions are the collision-facing AEAD projection of the same tag analysis: a winning opening gives distinct inputs sharing a tag, while the dedicated CMTDs-4 proof uses the all-bodies-equal game structure to avoid the leaf-collision slack of the general hash collision theorem (Section 6).

Relations. [BH22, Figs. 4 and 5] record the implications among the notions. For the D-notions we use:

$$\mathsf{CMTDs\text{-}4} \;\Rightarrow\; \mathsf{CMTDs\text{-}\ell} \text{ for } \ell \in \{1, 3\}$$

(both immediate: pairwise distinct keys ($\ell = 1$) or pairwise distinct (K, U, A) triples ($\ell = 3$) are *a fortiori* pairwise distinct full tuples, so every CMTDs-ℓ winner is a CMTDs-4 winner). The E-notions are bounded directly in Corollary 6 by the same final-root candidate-collision argument, rather than through an encryption reduction.

3 The TW128 Construction

3.1 Design Overview

TW128 is a nonce-based authenticated encryption scheme with associated data, built as a two-level tree of duplex objects over the Keccak-p[1600, 12] permutation. A single *root* duplex, initialized with the key K and nonce U, absorbs the associated data A when it is nonempty, emits keystream for the first plaintext chunk M_0 of at most β bytes, and absorbs the resulting ciphertext chunk C_0; when A is empty the associated data phase is elided and the initialization call itself emits the initial M_0 keystream; the remaining plaintext is partitioned into *leaf* chunks $M_1, \ldots, M_n$, each handled by an independent leaf duplex initialized with (K, U, j). The leaf duplex emits the keystream for M_j, absorbs the resulting ciphertext chunk C_j, and produces a τ_{leaf}-bit leaf tag T_j. When $n \geq 1$ the leaf tags, together with the eight-byte little-endian leaf count $\langle n \rangle_8$, are absorbed into the root duplex, which emits a single τ_{root}-bit root tag T; when $n = 0$ the aggregation phase is elided and the closing message phase call of the root chunk emits T directly, mirroring the way a leaf duplex emits its leaf tag. The construction is depicted in Figure 1.

Three design properties motivate the tree shape. First, the leaf duplexes operate on disjoint state and consume disjoint inputs once (K, U) is fixed, so leaves $1, \ldots, n$ can be evaluated in parallel. The transcript fixes the leaf decomposition and order, but not the number of leaves an implementation evaluates at once; scalar, SIMD, and multicore implementations therefore produce the same ciphertexts and tags. Section 8 quantifies the resulting throughput on AMD64 and ARM64 hardware. Second, every duplex operates on a bounded 1600-bit state regardless of message length, and the leaf tags are aggregated as a stream rather than buffered in full (Section 3.5), so the working memory is constant in $\|M\|$ at a fixed degree of parallelism, scaling only with the number of leaves evaluated

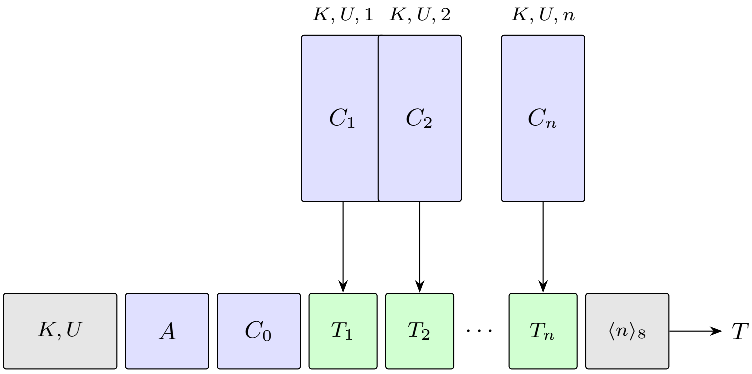

Figure 1: Schematic of TW128 for $n \geq 1$ leaves. The root duplex (bottom row) is initialized with (K, U) and absorbs in turn the associated data A, the root ciphertext chunk C_0, each leaf tag $T_1, \ldots, T_n$, and the eight-byte little-endian leaf count $\langle n \rangle_8$; it emits the root tag T. Each leaf duplex (top row) is initialized with (K, U, j), emits keystream for plaintext chunk M_j, absorbs the resulting ciphertext chunk C_j, and produces T_j. Leaves can be evaluated in parallel. The ciphertext chunks $C_0, C_1, \ldots, C_n$ are obtained by XORing the keystream emitted by the corresponding duplex with the plaintext chunks $M_0, M_1, \ldots, M_n$. When $n = 0$ the diagram reduces to the bottom row without any T_j boxes or aggregation block: the aggregation phase is elided and the closing message phase call on C_0 emits the root tag T directly. When the associated data is empty the A block is absent: the associated data phase is elided and the initialization call emits the initial M_0 keystream directly.

simultaneously. Third, the explicit aggregation of leaf tags through a single deterministic root transcript is the AEAD analog of KangarooTwelve's final-node hashing of per-chunk chaining values. In TW128, the chaining values are the leaf tags and the final digest is the root tag. This makes the root tag the natural hash point for the full (K, U, A, M) context; the formal hash, committing analysis appears in Section 6.

3.2 Parameters and Domain

Table 2 lists the user-visible and derived scheme parameters. The implementation uses the same byte length for the root tag and each leaf tag, but the proofs of Sections 5 and 6 keep τ_{root} and τ_{leaf} as separate symbols.

Table 2: TW128 parameters.

Parameter	Value	Description
k	256	Key length (bits)
ν	256	Nonce length (bits)
τ_{root}	256	Root tag length (bits)
τ_{leaf}	256	Leaf tag length (bits)
β	8183	Plaintext chunk size (bytes)
$M_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum plaintext length
$A_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum associated data length
b	1600	Permutation state width (bits)
n_r	12	Permutation rounds
r	1344	Sponge rate (bits)
c	256	Sponge capacity (bits)
ρ	167	Maximum byte-aligned absorption per duplex call (bytes)

The bound $M_{\max} = 8183 \cdot 2^{64}$ bytes is the largest plaintext whose leaf index fits in $\langle\cdot\rangle_8$: writing $\mathrm{leaves}(M) = \max(0, \lceil\|M\|/\beta\rceil - 1)$, every supported plaintext satisfies $0 \le \mathrm{leaves}(M) \le 2^{64} - 1$, so leaf indices lie in $\{1, \ldots, 2^{64} - 1\}$. The associated data bound $A_{\max}$ is a fixed scheme parameter taken equal to $M_{\max}$ unless stated otherwise. Inputs exceeding these bounds are outside the scheme domain.

The derived per-call absorption bound ρ follows the [BDPVA11] analysis: with $r = 1344$ and $\mathsf{pad10^*1}$ padding, $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits. The byte-aligned data field shares this allotment with the four-bit phase suffix of Section 3.4, so ρ is the largest integer satisfying $8\rho + 4 \le 1342$, giving $\rho = 167$ data bytes (and $8\rho + 4 = 1340 \le 1342$).

3.3 Parameter Choice Rationale

The four free parameters c, k, $\tau_{\mathsf{root}} = \tau_{\mathsf{leaf}}$, and β (the rate $r = b - c$ is then determined by the capacity) are chosen to give 128-bit single-key security against every term of the concrete bounds derived in Section 7.1 while keeping per-byte cost low; the nonce width ν is fixed separately, by the headroom the wide permutation affords. Specifically:

- $c = 256$. The dominant single-key term is the [BDPVA08] flat sponge loss

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1},$$

  which reaches the 2^{-128} threshold at $N_{\mathsf{tot}} \approx 2^{(c-127)/2} \approx 2^{64.5}$ Keccak-p[1600, 12] calls; the work cap $N_{\mathsf{tot}} \le 2^{63}$ used in Section 7.2 keeps this term at most $2^{-131}+2^{-194}$, and hence below 2^{-130}. Setting $c = 256$ thereby delivers 128-bit single-key security on this term. Under the mu-ind bound of Section 5 the per-user security margin degrades by $\log D$ bits in D users; the concrete D-indexed bound appears in Section 7.2.

- $r = 1344$. With state width $b = 1600$ fixed by the Keccak-p[1600, 12] choice, $r = b - c = 1344$ is the largest rate compatible with the chosen capacity. The 12-round Keccak-p[1600, 12] permutation matches KangarooTwelve's choice [BDP+16]; $\mathsf{TW128}$ adopts that established tree-hashing parameter rather than introducing a new reduced-round setting.

- $\tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$. Matching the capacity ensures that the per-tag guessing terms in the authenticity bounds and the tag collision terms in the hash and committing bounds (Section 7.1) are dominated by the capacity birthday rather than the tag length.

- $k = 256$. Matching the capacity ensures that the per-user key guess

$$\mathsf{KeyGuess}_k(N_{\mathsf{prim}}) = N_{\mathsf{prim}}/2^k$$

  remains below the capacity birthday at the target D of Section 7.2.

- $\beta = 8183$. The chunk size is set to $49\rho = 49 \cdot 167$ bytes, the largest multiple of the per-call absorption bound ρ not exceeding 2^{13} bytes. Taking β to be an exact multiple of ρ makes every full leaf split into exactly 49 fully utilized message duplex calls with no short final block, so a full leaf costs one initialization duplex call plus 49 message duplex calls. This amortizes the fixed per-leaf initialization cost over a large chunk while keeping the leaf state working set ($8183 < 2^{13}$ bytes) small enough for L1 cache; Section 8.3 reports the resulting cycles-per-byte figures.

- $\nu = 256$. The nonce width is not constrained by the concrete bounds, which assume a nonce-respecting adversary; it is set generously because the wide permutation makes a wide nonce nearly free. The root initialization absorbs $p_{\mathsf{root}} \parallel K \parallel U$, 560 bits in all, in a single duplex call, comfortably within the 1336-bit ($\rho = 167$-byte) data field

(Section 3.2), so widening the nonce to 256 bits adds neither a permutation call nor any per-message cost. The width also removes any practical concern over random nonce collisions: a uniformly random 256-bit nonce repeats over q messages with probability at most $\binom{q}{2}/2^{256}$, negligible at the work cap of Section 7.2. Deployments that do not require a full-width random nonce lose nothing by the choice: a narrower nonce—a 96-bit packet counter, say—is carried in the fixed ν-bit field by zero-padding, so a single mode serves both random nonce and counter-based settings with no change of parameters.

3.4 Domain Separation

Distinct duplex instances and distinct phases within a duplex are kept separate at the transcript level by two mechanisms. First, the initialization strings carry a 6-byte ASCII prefix and, for leaves, an eight-byte little-endian chunk index:

$$p_{\mathsf{root}} = \texttt{"TW128R"} \in \{0,1\}^{48},$$
$$p_{\mathsf{leaf}} = \texttt{"TW128L"} \in \{0,1\}^{48}.$$

The root duplex's first absorbed string is $p_{\mathsf{root}} \parallel K \parallel U$, and the j-th leaf duplex's first absorbed string is $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$. Together with the fixed k-bit and ν-bit widths of K and U, these prefixes separate root from leaf transcripts and distinguish leaves with different chunk indices.

Second, every byte-aligned absorption carries a four-bit phase suffix. We identify each suffix with the four-bit string given by its Table 3 value written least significant bit first, so that a duplex call absorbing a byte-aligned block σ under a phase suffix σ_{ph} feeds the duplex interface of Section 2.2 the input $\sigma \parallel \sigma_{\mathsf{ph}}$ before $\mathsf{pad10^*1}$ padding. The seven phase suffixes are summarized in Table 3; they correspond to initialization, non-final and final associated data blocks, non-final and final message blocks, and non-final and final aggregation blocks. The seven values are pairwise distinct, so any two transcripts that differ in the phase of any block also differ in the flattened sponge input.

Table 3: Phase suffixes (4 bits each, appended LSB-first).

Name	Hex	Use
σ_{init}	`0xC`	End of initialization (the sole init call)
σ^-_{ad}	`0x9`	Non-final associated data block
σ^+_{ad}	`0xD`	Final associated data block
σ^-_{msg}	`0xA`	Non-final message block
σ^+_{msg}	`0xE`	Final message block
σ^-_{agg}	`0xB`	Non-final aggregation block
σ^+_{agg}	`0xF`	Final aggregation block (emits root tag)

Table 4 summarizes the root and leaf transcript schedule. The algorithms of Sections 3.5 and 3.6 give the normative byte-level specification; the table records which phase is present and what output the final call of that phase supplies.

The combined effect of the initialization prefixes and the phase suffixes is that the flattened sponge input of any TW128 duplex call uniquely determines the duplex instance, the leaf index when applicable, the phase, and the duplex-call position within that phase. Definition 2 (Well-formed TW128 transcript prefixes), Lemma 5 (Transcript Injectivity), and its structural corollaries formalize this parsing fact for the proofs of Sections 5 and 6 and Corollary 6.

Table 4: TW128 transcript schedule. "First M_i keystream" means $8\min(\|M_i\|, \rho)$ output bits for the first duplex block of that message chunk; subsequent message-phase outputs have the length of the next duplex block, as in Algorithm 1.

Instance	Phase	Suffixes	Absorbed string	Output supplied	When
Root	Init	σ_{init}	$p_{\mathsf{root}} \parallel K \parallel U$	0 if $A \neq \varepsilon$; otherwise first M_0 keystream	Always
Root	AD	σ^-_{ad}, σ^+_{ad}	ρ-byte block partition of A	Final call emits first M_0 keystream	$A \neq \varepsilon$
Root	Message	σ^-_{msg}, σ^+_{msg}	ρ-byte block partition of C_0	Non-final calls emit next keystream; final call emits T if $n = 0$, and 0 otherwise	Always
Leaf j	Init	σ_{init}	$p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$	First M_j keystream	$1 \leq j \leq n$
Leaf j	Message	σ^-_{msg}, σ^+_{msg}	ρ-byte block partition of C_j	Non-final calls emit next keystream; final call emits T_j	$1 \leq j \leq n$
Root	Aggregation	σ^-_{agg}, σ^+_{agg}	$T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$	Final call emits T	$n \geq 1$

3.5 Encryption

Algorithm 2 specifies $\mathsf{Enc}_K(U, A, M)$ in terms of two helper procedures, Absorb and StreamEnc, defined in Algorithm 1. These helpers sit below the tree layer: only Algorithms 2 and 3 split messages or ciphertexts into β-byte root and leaf chunks, while the helpers split an already chosen byte string into ρ-byte duplex blocks. Absorb drives a duplex through a sequence of such blocks with non-final σ^- and final σ^+ suffix tags, returning the output of the final call; the empty-input case yields a single empty duplex block carrying the σ^+ tag. The streaming helpers use the same empty-input convention, so a zero-length message string still performs one final message phase duplex call and returns a zero-length body. Algorithm 2 invokes Absorb for the associated data phase only when $A \neq \varepsilon$; when $A = \varepsilon$ the phase is elided and the root σ_{init}call emits the initial M_0 keystream directly. StreamEnc encrypts a plaintext byte string one duplex block at a time, XORing each block with the current keystream, absorbing the resulting ciphertext duplex block back into the duplex with the appropriate phase tag, and using the duplex's output as the next keystream. This is the duplex-level presentation of the overwrite mechanism of [BDPVA11, §6.2 and Algorithm 5]: when the current rate prefix emitted as keystream is Z, XOR-absorbing $m \oplus Z$ has the same update on the message bytes as overwriting that prefix with m, with the phase tag still carried as part of the duplex input. The decryption helper StreamDec is identical to StreamEnc with the roles of the plaintext and ciphertext duplex blocks exchanged: it takes ciphertext duplex blocks c_i as input, recovers each plaintext duplex block as $m_i = c_i \oplus Z$, absorbs the same c_i under the same phase suffix, and returns $(m_1 \parallel \cdots \parallel m_t, Z')$; we omit its pseudocode. In both helpers, each duplex call absorbs a duplex block followed by its phase suffix—σ^- for a non-final block and σ^+ for the final block—and requests the output bits the caller consumes next: ℓ (possibly zero) on the final call, and the next duplex block's keystream length on non-final calls. Each call is the duplex interface of Section 2.2 applied to the suffixed input of Section 3.4.

The encryption procedure (Algorithm 2) instantiates the root duplex and obtains the initial keystream for the root chunk M_0: when $A \neq \varepsilon$ it absorbs the associated data A, and when $A = \varepsilon$ it elides the associated data phase and takes the keystream from the σ_{init}call itself. It then encrypts M_0 while absorbing the resulting ciphertext chunk C_0. When $n = 0$ the closing σ^+_{msg}call of this root chunk requests τ_{root} output bits, which are the root tag T, and the aggregation phase is elided. When $n \geq 1$ it independently encrypts each leaf chunk M_j while absorbing C_j, producing the leaf tag T_j; the leaf tags together with $\langle n \rangle_8$ are then absorbed into the root duplex, and the final σ^+_{agg}call emits the root tag T. The ciphertext is the concatenation $C_0 \parallel C_1 \parallel \cdots \parallel C_n$. Thus this section writes C for the ciphertext body and $C \parallel T$ for the full AEAD ciphertext returned by $\mathsf{Enc}_K(U, A, M)$.

The initial keystream request for a message string X is $8\min(\|X\|, \rho)$, the bit length of

Algorithm 1 TW128 helper procedures.

procedure $\textsc{Absorb}(D,\, s,\, \sigma^-,\, \sigma^+,\, \ell)$
  Partition s into duplex blocks $s_1, \ldots, s_t$ of exactly ρ bytes each, except that the last block may be shorter.
  Require $t \geq 1$; the empty case yields the single empty duplex block $s_1 = \varepsilon$.
  for $i \leftarrow 1$ **to** $t-1$ **do**
    $D.\mathsf{duplexing}(s_i \parallel \sigma^-, 0)$
  end for
  return $D.\mathsf{duplexing}(s_t \parallel \sigma^+, \ell)$
end procedure

procedure $\textsc{StreamEnc}(D,\, m,\, Z,\, \sigma^-,\, \sigma^+,\, \ell)$
  Partition m into duplex blocks $m_1, \ldots, m_t$ of exactly ρ bytes each, except that the last block may be shorter.
  Require $t \geq 1$; if $m = \varepsilon$, set $t = 1$ and $m_1 = \varepsilon$.
  for $i \leftarrow 1$ **to** $t-1$ **do**
    $c_i \leftarrow m_i \oplus Z$
    $Z \leftarrow D.\mathsf{duplexing}(c_i \parallel \sigma^-, 8\|m_{i+1}\|)$
  end for
  $c_t \leftarrow m_t \oplus Z$
  $Z' \leftarrow D.\mathsf{duplexing}(c_t \parallel \sigma^+, \ell)$
  return $(c_1 \parallel \cdots \parallel c_t, Z')$
end procedure

its first duplex block. For the root chunk M_0, that value is emitted by the closing σ^+_{ad}call when $A \neq \varepsilon$, and by the root σ_{init}call itself when $A = \varepsilon$ and the associated data phase is elided. Each leaf's σ_{init}call likewise emits $8\min(\|M_j\|, \rho)$ keystream bits for the first duplex block of M_j, so the empty associated data root σ_{init}call plays exactly the role a leaf σ_{init}call already plays. When the corresponding message string is empty, the requested keystream length is zero and the next message phase duplex call absorbs the empty ciphertext duplex block carrying σ^+_{msg}. The final σ^+_{msg}call in the root chunk requests τ_{root} output bits when $n = 0$, namely the root tag T emitted by the elided-aggregation path, and zero output bits when $n \geq 1$ because the next duplex operation on the root is then the first aggregation block; the corresponding call on a leaf requests τ_{leaf} bits, namely the leaf tag. When $n \geq 1$ the final σ^+_{agg}call requests τ_{root} bits, which is the root tag T.

The leaf loop is annotated **in parallel** to make explicit that the n leaf computations are mutually independent; Section 8.1 discusses the SIMD realizations. The annotation is an implementation freedom, not a mode parameter: evaluating leaves serially, in SIMD batches, or across cores leaves the transcript and wire format unchanged.

Streaming aggregation. The string $G = T_1 \parallel \cdots \parallel T_n$ in Algorithms 2 and 3 names the leaf tags only for specification. The closing $\textsc{Absorb}$ consumes $G \parallel \langle n \rangle_8$ as ρ-byte duplex blocks in leaf order and advances the root state after each, so an implementation need not hold G in full. It can absorb leaf tags in order as they become available, or buffer completed out-of-order leaves until the next in-order tag or SIMD batch can be absorbed, keeping only a bounded number of tags at a time. With the bounded 1600-bit state of every duplex, this keeps the working memory constant in $\|M\|$ for a fixed degree of parallelism, growing only with the number of leaves evaluated at once rather than with the message length; Section 8.1 describes the batched realization.

Algorithm 2 $\mathsf{Enc}_K(U, A, M)$.

Require: $K \in \{0,1\}^k$, $U \in \{0,1\}^\nu$, A with $\|A\| \le A_{\max}$, M with $\|M\| \le M_{\max}$.
Ensure: $C \parallel T$ with $\|C\| = \|M\|$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.

$M_0 \leftarrow$ first $\min(\|M\|, \beta)$ bytes of M (possibly ε).
Partition the remaining suffix into chunks $M_1, \ldots, M_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \text{leaves}(M)$.
$D_{\text{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
  $Z \leftarrow D_{\text{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 8\min(\|M_0\|, \rho))$
else
  $D_{\text{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 0)$
  $Z \leftarrow \textsc{Absorb}(D_{\text{root}}, A, \sigma^-_{\mathsf{ad}}, \sigma^+_{\mathsf{ad}}, 8\min(\|M_0\|, \rho))$
end if
if $n = 0$ **then**
  $(C_0, T) \leftarrow \textsc{StreamEnc}(D_{\text{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{root}})$
  return $C_0 \parallel T$
end if
$(C_0, \_) \leftarrow \textsc{StreamEnc}(D_{\text{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
  $D_j \leftarrow$ new Duplex
  $Z_j \leftarrow D_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8 \parallel \sigma_{\mathsf{init}}, 8\min(\|M_j\|, \rho))$
  $(C_j, T_j) \leftarrow \textsc{StreamEnc}(D_j, M_j, Z_j, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \parallel \cdots \parallel T_n$.
$T \leftarrow \textsc{Absorb}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma^-_{\mathsf{agg}}, \sigma^+_{\mathsf{agg}}, \tau_{\mathsf{root}})$
return $C_0 \parallel C_1 \parallel \cdots \parallel C_n \parallel T$

3.6 Decryption

Algorithm 3 specifies $\mathsf{Dec}_K(U, A, C \parallel T)$. The procedure mirrors Enc at the duplex block level: each ciphertext duplex block c_i is absorbed into the appropriate duplex (root or leaf) under the corresponding σ^-_{msg}or σ^+_{msg}suffix, and the plaintext duplex block is recovered as $m_i = c_i \oplus Z^{(i)}$, writing $Z^{(i)}$ for the value the running keystream variable Z of $\textsc{StreamDec}$ holds when duplex block i is processed ($i < t$ inside the loop, $i = t$ after it), the same value the encryption procedure would have produced. The procedure absorbs ciphertext bytes before the final tag comparison, so the verification transcript is determined by (K, U, A, C) for every in-domain ciphertext, accepting or not. This property is used by the direct mu-ind proof to argue that encryption-first and verification-first transcript classes are well defined regardless of the tag-comparison outcome.

The $n = 0$ and empty-body cases mirror their encryption counterparts (Section 3.5): for $n = 0$ the aggregation phase is elided and the closing σ^+_{msg}call of the root chunk emits the candidate tag T' directly, and an empty body still performs the single empty σ^+_{msg}call of Algorithm 1, requesting τ_{root} output bits.

Correctness. For every $K \in \{0,1\}^k$, $U \in \{0,1\}^\nu$, A with $\|A\| \le A_{\max}$, and M with $\|M\| \le M_{\max}$, $\mathsf{Dec}_K(U, A, \mathsf{Enc}_K(U, A, M)) = M$. This follows by inspection: every duplex call in Dec_K receives the same input as the corresponding call in Enc_K (each ciphertext duplex block is absorbed under the same phase tag in both directions), so the duplex states and the recomputed root tag T' exactly match. The constant-time comparison $T' = T$ succeeds, and the recovered $m_i = c_i \oplus Z^{(i)} = (m_i \oplus Z^{(i)}) \oplus Z^{(i)} = m_i$ at every position.

Algorithm 3 $\mathsf{Dec}_K(U, A, C \parallel T)$.

Require: K, U, A as in Algorithm 2, ciphertext C with $\|C\| \le M_{\max}$, candidate tag $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.
Ensure: M with $\|M\| = \|C\|$, or $\perp$.

$C_0 \leftarrow$ first $\min(\|C\|, \beta)$ bytes of C (possibly ε).
Partition the remaining suffix into chunks $C_1, \ldots, C_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \text{leaves}(C)$.
$D_{\text{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
  $Z \leftarrow D_{\text{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 8\min(\|C_0\|, \rho))$
else
  $D_{\text{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \parallel K \parallel U \parallel \sigma_{\mathsf{init}}, 0)$
  $Z \leftarrow \textsc{Absorb}(D_{\text{root}}, A, \sigma^-_{\mathsf{ad}}, \sigma^+_{\mathsf{ad}}, 8\min(\|C_0\|, \rho))$
end if
if $n = 0$ **then**
  $(M_0, T') \leftarrow \textsc{StreamDec}(D_{\text{root}}, C_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{root}})$
  return M_0 if $T' = T$ (compared in constant time), else $\perp$.
end if
$(M_0, \_) \leftarrow \textsc{StreamDec}(D_{\text{root}}, C_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
  $D_j \leftarrow$ new Duplex
  $Z_j \leftarrow D_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8 \parallel \sigma_{\mathsf{init}}, 8\min(\|C_j\|, \rho))$
  $(M_j, T_j) \leftarrow \textsc{StreamDec}(D_j, C_j, Z_j, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \parallel \cdots \parallel T_n$.
$T' \leftarrow \textsc{Absorb}(D_{\text{root}}, G \parallel \langle n \rangle_8, \sigma^-_{\mathsf{agg}}, \sigma^+_{\mathsf{agg}}, \tau_{\mathsf{root}})$
if $T' = T$ (compared in constant time) **then**
  return $M_0 \parallel M_1 \parallel \cdots \parallel M_n$
else
  return $\perp$
end if

Lemma 1 (TW128 tidiness)**.** *For every valid TW128 decryption input, if* $\mathsf{Dec}_K(U, A, C \parallel T) = M \neq \perp$*, then* $\mathsf{Enc}_K(U, A, M) = C \parallel T$.

Proof. An accepting decryption first passes the syntax and length checks of Algorithm 3, so the recovered message M has $\|M\| = \|C\|$ and the same canonical chunking as the ciphertext body. The decryption transcript absorbs K, U, A, and each ciphertext block under the same duplex instances and phase suffixes used by encryption. Its stream outputs recover each plaintext block as $M_i = C_i \oplus Z_i$, so re-encrypting the recovered message under the same transcript emits $C_i = M_i \oplus Z_i$ and absorbs the same ciphertext block at every message position. The leaf tags and final root candidate tag recomputed by decryption are therefore exactly the values recomputed by this re-encryption. Since decryption accepts only when its final candidate tag equals the submitted tag T, re-encryption returns the submitted body and tag $C \parallel T$. $\square$

4 Security Model

This section sets up the two proof views used throughout the paper. For confidentiality and authenticity, TW128 is treated as a nonce-based AEAD in the lifted public permutation model: real encryption and verification interfaces are compared with ideal random and

Table 5: Proof obligations and the dominant loss shapes used for TW128. The table is only a roadmap; the formal games and resource accounting are defined in Sections 4.1, 4.2 and 4.4.

Notion	Adversary goal	Main result	Loss shape
AE / mu-ind	Distinguish real encryption and verification from random ciphertexts and replay-only verification, with nonce-respecting encryption queries.	Theorem 2 and Corollary 1	Capacity lift, key guess, leaf collision, tag guess
INT-RUP	Produce a fresh accepting ciphertext after encryption and plaintext-computing queries.	Theorem 4	Capacity lift, key guess, leaf collision, tag guess
$\mathsf{H}_{\mathsf{TW128}}$ collision	Find distinct full inputs (K, U, A, M) with the same root tag.	Theorem 8	Capacity lift, root collision, leaf collision
$\mathsf{H}_{\mathsf{TW128}}$ preimage	Find an input hitting a fixed target tag.	Theorem 10	Capacity lift, root preimage
$\mathsf{H}_{\mathsf{TW128}}$ second preimage	Find a second preimage in the standard or everywhere [RS04] sense.	Corollary 3	Inherited from collision resistance
CMT / CMTD	Produce equal encryptions or one valid ciphertext opening under distinct committed inputs.	Corollaries 5 and 6	Capacity lift, root collision

replay interfaces, first in the multi-user setting and then in the single-user all-in-one AE setting. For the digest-like tag claim, the same construction is viewed through the root tag wrapper $\mathsf{H}_{\mathsf{TW128}}$: adversaries supply full inputs or candidate openings, and the games ask whether the root tag collides or can be hit as a preimage. We instantiate these games for TW128 (Sections 4.1 and 4.2), define the intermediate random oracle world through which every bound is first proved (Section 4.3), and fix the resource accounting that all later bounds reference (Section 4.4).

Table 5 summarizes the proof obligations before the formal games. The loss terms named there are defined in Section 4.5, and the implemented-parameter instantiations appear in Section 7.

Scope of the security claims. All results below are public-permutation claims for TW128 with the Keccak-p[1600, 12] permutation modeled as a uniformly random permutation, obtained by first proving random oracle bounds for the flat duplex interface of Section 4.3 and then applying the flat sponge lift. In the AE-style distinguishing games, encryption queries are nonce-respecting and the verification oracle returns only acceptance or rejection. The separate INT-RUP game below gives integrity under a plaintext-computing oracle, but it does not give privacy under release of unverified plaintext or plaintext-awareness security. The claims also do not provide nonce-misuse resistance. The hash and committing games are adversarial-initialization games over the submitted inputs (K, U, A, M) or ciphertext openings and impose no nonce uniqueness condition. Across all games, the stated bounds are birthday-type, single-stage, and leakage-free: beyond-birthday and side-channel security are outside the model. Section 9.4 discusses the consequences of these exclusions and the corresponding open problems.

The proof dependency spine is as follows. Section A first establishes transcript decoding, bridge injectivity, output-point separation, and the local probabilistic accounting lemmas for key tests, leaf tag collisions, and root tag guesses. Section 5 uses those facts directly for the random oracle mu-ind and INT-RUP bounds. Section 6 then uses the same structural facts to build a chronological sequence of final root candidates and apply a single root tag hash bound; the hash and committing theorems differ only in which collision or preimage event is reduced to that candidate sequence. Finally, Section B transfers the random oracle bounds to the public permutation games defined below.

4.1 Public Permutation Games

Primitive convention. In the real public permutation world, the challenger samples $\pi \leftarrow\$ \ \mathsf{Perm}(\{0,1\}^{1600})$ and exposes the direct primitive interface (π, π^{-1}). The real-vs-ideal AE-style games compare this world to a simulator-ideal world, where the construction interface is the ideal interface specified by the game and the public primitive interface is $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$. The simulator $\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}$ together with its inverse $\mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}}$ is fixed in Section B; both directions share an internal lazy table for the random oracle $\mathsf{RO}_{\mathsf{flat}}$ of Section 4.3. The hash and committing games below are one-world success probabilities in the real public permutation world. For them, the simulator appears only in the later lift from the intermediate random oracle model.

Validity convention. All construction interface inputs are checked against TW128's public syntax and domain before any construction computation or construction-internal primitive lookup is performed. An encryption input is valid when $U \in \{0,1\}^{\nu}$, A is a byte string with $\|A\| \leq A_{\max}$, and M is a byte string with $\|M\| \leq M_{\max}$. A verification or decryption input is valid when $U \in \{0,1\}^{\nu}$, A is a byte string with $\|A\| \leq A_{\max}$, and the full ciphertext parses as $C \parallel T$ with $\|C\| \leq M_{\max}$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$. In the hash and committing games, each submitted input (K, U, A, M) must be a valid encryption input including $K \in \{0,1\}^{k}$, and a submitted CMTDs ciphertext must be a valid verification/decryption ciphertext. Invalid encryption queries are rejected and not recorded, invalid plaintext-computing queries return $\perp$, invalid verification queries return 0, and invalid committing-game outputs make the game return 0. Such rejected inputs contribute no construction cost and create no construction transcripts. Throughout, an *accepted* encryption query is one that passes this validity check and any game-specific nonce or user check.

All-in-one AE. The real all-in-one AE game additionally samples $K \leftarrow\$ \ \{0,1\}^{k}$ and exposes the construction oracles $\mathsf{Enc}_K[\pi]$ (the TW128 encryption algorithm of Algorithm 2 with the sampled π) and $\mathsf{Ver}_K[\pi]$, defined by $\mathsf{Ver}_K[\pi](U, A, C, T) = 1$ if $\mathsf{Dec}_K[\pi](U, A, C \parallel T) \neq \perp$ and 0 otherwise. The all-in-one authenticated encryption game [RS06] replaces the construction interface by the ideal pair $(\mathsf{Rand}, \mathsf{Replay})$. On each accepted encryption query (U, A, M), Rand samples a uniformly random byte string of length $\|M\| + \tau_{\mathsf{root}}/8$, parses it as $C \parallel T$ with $T \in \{0,1\}^{\tau_{\mathsf{root}}}$, records (U, A, C, T), and returns $C \parallel T$. The oracle Replay returns 1 exactly on recorded tuples (U, A, C, T) and 0 otherwise. The adversary is nonce-respecting on encryption queries.

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) = \left|\Pr[\mathcal{A}^{\mathsf{Enc}_K[\pi],\, \mathsf{Ver}_K[\pi],\, \pi,\, \pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Rand},\, \mathsf{Replay},\, \mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}},\, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}}} \Rightarrow 1]\right|.$$

This lifted all-in-one AE advantage is the single-user case of the multi-user real-vs-ideal notion of Section 2.4; Corollary 1 therefore derives it from the direct mu-ind bound of Theorem 2.

RUP integrity. Following the separated AE syntax of [ABL+14], define $\mathsf{PDec}_K[\pi]$ as the plaintext-computing oracle that maps (U, A, C, T) to the candidate plaintext computed by $\mathsf{Dec}_K[\pi](U, A, C \parallel T)$ before the final root tag comparison is applied. The computation follows the same decryption transcript as Algorithm 3, but returns the recovered plaintext and never returns the recomputed tag or the verification bit.

The INT-RUP game samples $K \leftarrow\$ \ \{0,1\}^{k}$ and $\pi \leftarrow\$ \ \mathsf{Perm}(\{0,1\}^{1600})$, exposes $\mathsf{Enc}_K[\pi]$, $\mathsf{PDec}_K[\pi]$, $\mathsf{Ver}_K[\pi]$, and (π, π^{-1}), and maintains the replay table W of encryption outputs. Accepted encryption queries are nonce-respecting: an encryption query is rejected if its nonce has appeared in a previous accepted encryption query. Plaintext-computing and verification queries may use arbitrary nonces. On an accepted encryption query (U, A, M) the game returns $\mathsf{Enc}_K[\pi](U, A, M)$, parsed and recorded as (U, A, C, T) in W. On a valid plaintext-computing query (U, A, C, T) it returns $\mathsf{PDec}_K[\pi](U, A, C, T)$, and on an invalid query it returns $\perp$. On a valid verification query (U, A, C, T) it returns $\mathsf{Ver}_K[\pi](U, A, C, T)$; if this bit is 1 and $(U, A, C, T) \notin W$ when the query is asked, the game records a forgery. Invalid verification queries return 0. The game returns 1 iff a forgery is recorded, and

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \Rightarrow 1].$$

Hash games. The hash games for the wrapper $\mathsf{H}_{\mathsf{TW128}}$ (Definition 1) have no challenger-sampled key. In the s-way collision game, all inputs are adversary-supplied: the adversary outputs s inputs, and the challenger returns 1 iff they are pairwise distinct, in the $\mathsf{TW128}$ domain, and share a root tag:

$$\begin{aligned}
&\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600}), \quad (K_i, U_i, A_i, M_i)_{i=1}^{s} \leftarrow\$ \mathcal{A}^{\pi, \pi^{-1}}, \\
&\text{return 0 unless the inputs are pairwise distinct and in the } \mathsf{TW128} \text{ domain}, \\
&\text{return } \big[\mathsf{H}_{\mathsf{TW128}}(K_1, U_1, A_1, M_1) = \cdots = \mathsf{H}_{\mathsf{TW128}}(K_s, U_s, A_s, M_s)\big],
\end{aligned}$$

with advantage $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\text{the game returns 1}]$; no nonce uniqueness is imposed. The [RS04] standard and everywhere second-preimage notions are obtained from this collision game by [RS04, Proposition 6]; no separate second-preimage game is needed. For fixed byte lengths a, μ, the standard preimage game samples $X^* \leftarrow\$ \mathcal{X}_{a,\mu}$ independently of the primitive, computes $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$, gives T^* to $\mathcal{A}^{\pi,\pi^{-1}}$, and returns 1 iff $\mathcal{A}$ outputs a valid input X in the $\mathsf{TW128}$ domain with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$; the advantage is $\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}(\mathcal{A})$. The everywhere preimage game fixes a target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ before the primitive sampling and returns 1 iff $\mathcal{A}(T^*)$ outputs an input X in the domain with $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$, the advantage $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}$ taken as the worst case over query-independent targets.

Committing games. The [BH22] committing games are instantiated as in Section 2.5, with no challenger-sampled key. The s-way CMTDs-ℓ (decryption) game has the adversary output a ciphertext $C \,\|\, T$ and s tuples (K_i, U_i, A_i, M_i) with pairwise distinct WiC_ℓ-images, and returns 1 iff $\mathsf{Dec}_{K_i}[\pi](U_i, A_i, C \,\|\, T) = M_i$ for every i, with the eager length rejection justified by Algorithm 3 ($\mathsf{Dec}_K(U, A, C \,\|\, T)$ returns $\perp$ or a message of length $\|C\|$); its advantage is $\mathbf{Adv}^{\mathsf{CMTDs}\text{-}\ell}_{\mathsf{TW128}}(\mathcal{A})$. The CMTs-$\ell$ (encryption) game has the adversary output the s tuples alone and returns 1 iff the encryptions $\mathsf{Enc}_{K_i}[\pi](U_i, A_i, M_i)$ are all equal; its advantage is $\mathbf{Adv}^{\mathsf{CMTs}\text{-}\ell}_{\mathsf{TW128}}(\mathcal{A})$.

4.2 Multi-User Setting

This subsection instantiates the mu-ind game of [BT16] (Section 2.4) for $\mathsf{TW128}$ in the lifted public permutation model, using the same ideal-side simulator convention as the single-user AE-style games of Section 4.1. The challenger samples a challenge bit $b \leftarrow\$ \{0,1\}$. If $b = 1$, it samples a permutation $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$ and exposes (π, π^{-1}) as the primitive interface; if $b = 0$, it exposes $(\mathcal{S}^{\mathsf{RO_{flat}}}, \mathcal{S}^{-1,\mathsf{RO_{flat}}})$ as the primitive interface. The adversary $\mathcal{A}$ creates users on demand via New, and the game maintains an active-user counter v. We write D for an execution-uniform cap on the number of New calls. Equivalently, the game may sample independent keys $K_1, \ldots, K_D$ at the start of the experiment and let the d-th New call activate K_d; inactive indices are never valid users.

The mu-ind construction oracles inherit the validity convention of Section 4.1. The encryption oracle on input (d, U, A, M) rejects, without recording, if (U, A, M) is outside the $\mathsf{TW128}$ encryption domain, if $d \notin \{1, \ldots, v\}$, or if $(d, U) \in V$; otherwise it sets $C = \mathsf{Enc}_{K_d}[\pi](U, A, M)$ when $b = 1$, and samples a uniformly random byte string C of length $\|M\| + \tau_{\mathsf{root}}/8$ when $b = 0$. The game records (d, U) in V and (d, U, A, C) in W, and returns C. The verification oracle on input (d, U, A, C) rejects if $d \notin \{1, \dots, v\}$ or if (U, A, C) is not a valid $\mathsf{TW128}$ verification/decryption input; if $(d, U, A, C) \in W$ it returns true; if $b = 0$ it returns false; otherwise it returns $[\mathsf{Dec}_{K_d}[\pi](U, A, C) \neq \bot]$. Nonce uniqueness is enforced per user by the set V.

After querying these oracles, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. The mu-ind advantage of $\mathcal{A}$ against $\mathsf{TW128}$ is

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) = |2 \Pr[b' = b] - 1|.$$

A valid verification query is *non-replay* if its tuple is not in W when the query is asked; replay hits are answered by the recorded transcript and are not counted in the verification-query resource below.

Resource caps split per potential user $d \leq D$: $N^{(d)}$, $q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}}^{(d)}$ are the per-user counts under Section 4.4, with inactive users contributing zero. The global totals $N = \sum_{d=1}^{D} N^{(d)}$, $q_{\mathsf{v}} = \sum_{d=1}^{D} q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}} = \sum_{d=1}^{D} n_{\mathsf{leaf}}^{(d)}$ appear in the final bound: N through the lift term $\mathsf{Lift}_c(N_{\mathsf{tot}})$, and n_{leaf} and q_{v} through the leaf-collision and verification-tag terms. N_{prim} remains a single shared count over the primitive interface.

The mu-ind treatment applies only to indistinguishability. The committing security games already give the adversary control of all s keys, so the s-way CMTDs-ℓ statements of Section 4.1 are themselves the multi-key committing statement and need no separate $\mathsf{mu\text{-}cmt}$ notion.

4.3 The TW128 Random Oracle Model

The proofs of Sections 5 and 6 and Corollary 6 are carried out through an intermediate random oracle model whose construction interfaces use the [BDPVA11] bridge of Section 2.2 to address a single random oracle. The flat sponge lift of Section B then converts the resulting bounds to the public permutation form.

The model has a single random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ over the unpadded H-input domain of the [BDPVA08] simple-padded sponge interface, with lazy sampling: each distinct input receives an independent infinite bit string, and repeated queries return the same string. A direct adversary query is finite-output: on input (Y, ℓ) with finite ℓ, the oracle returns the first ℓ bits of the lazy-sampled value for Y.

Construction calls do not query $\mathsf{RO}_{\mathsf{flat}}$ on arbitrary strings. Every root or leaf duplex output is read from $\mathsf{RO}_{\mathsf{flat}}$ at the bridged flattened transcript prefix for that duplex call, and the caller takes only the requested output projection. Direct adversary queries to $\mathsf{RO}_{\mathsf{flat}}$ are the only random oracle queries counted as direct oracle queries; construction lookups are accounted for through the construction cost N of Section 4.4.

For a well-formed $\mathsf{TW128}$ duplex transcript prefix X in the language of Definition 2, at one of the construction transcript lookup points formalized by Lemma 7 and Table 12, let $\mathsf{flat}(X) = I[1344, 1344](\mathsf{raw\_flat}(X))$ be the bridged image of the native flattened input under the [BDPVA11, Theorem 3] preprocessing map of Section 2.2, and define

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)).$$

The construction interfaces $\mathsf{Enc}_K^{\mathsf{RO}}$, $\mathsf{Dec}_K^{\mathsf{RO}}$, $\mathsf{PDec}_K^{\mathsf{RO}}$, and $\mathsf{Ver}_K^{\mathsf{RO}}$ are the $\mathsf{TW128}$ algorithms of Section 3 with every root or leaf duplex output replaced by the corresponding requested projection of $\mathsf{RF}(X)$; the projection length is the duplexing call's output parameter ℓ, which may be zero. The bridge and the typed-restriction convention are formalized in Lemma 7; the direct-query key-hit predicate ignores the adversary's requested output length and is formalized as $\mathsf{KeyedTWOut}$ in Lemma 7.

The random oracle model games are the construction interface variants of Sections 4.1 and 4.2: the primitive interface is replaced by direct adversary access to $\mathsf{RO}_{\mathsf{flat}}$, and each construction oracle uses the corresponding $^{\mathsf{RO}}$ algorithm, the ideal oracles $(\mathsf{Rand}, \mathsf{Replay})$

Table 6: Resource-accounting notation. Uppercase symbols are the caps that appear in public-permutation theorems; lowercase symbols are realized counts in random oracle executions or source games.

Symbol	Counted objects	Primary use
N	Construction-internal KECCAK-p[1600, 12] calls induced by valid construction oracle queries and deterministic challenger checks.	Public-permutation lift and concrete work cap.
N_{prim}	Direct adversary queries to the public primitive interface.	Key-guess, root-collision, root-preimage terms; direct-query part of the lift.
$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$	Total public-primitive transcript length seen by the flat sponge replacement.	Argument of $\mathsf{Lift}_c(N_{\mathsf{tot}})$.
$q_{\mathsf{RO}}(\mathcal{B})$, Q_{RO}	Direct finite-output queries to $\mathsf{RO}_{\mathsf{flat}}$ in the intermediate random oracle model; construction lookups are excluded.	Random oracle bounds; after the lift $q_{\mathsf{RO}} \le N_{\mathsf{prim}}$.
$q_{\mathsf{v}}(\mathcal{A})$, Q_{v}	Valid verification queries that are not replayable hits.	Verification-tag terms in AE, mu-ind, and INT-RUP bounds.
$n_{\mathsf{leaf}}(\mathcal{B})$, N_{leaf}	Distinct construction-generated final leaf transcripts.	Leaf tag collision and tag-guessing terms; always at most N.
D	Execution-uniform cap on users activated in the mu-ind game.	Multi-user key-collision and key-guess terms.

being unchanged. Under this substitution $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{RO}}$ is the all-in-one advantage of Section 4.1 with no simulator on the ideal side; the keyless hash advantages $\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{RO}}(\mathcal{B})$ and $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}(\mathcal{B})$ of $\mathsf{H}_{\mathsf{TW128}}$, the committing $\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B})$, and, for $\ell \in \{1,3,4\}$, $\mathbf{Adv}^{\mathsf{CMTs\text{-}}\ell}_{\mathsf{RO}}(\mathcal{B})$ are the probabilities that the corresponding random oracle hash and committing games return 1, the CMTs challenger computing its deterministic encryption checks with $\mathsf{Enc}^{\mathsf{RO}}$. For mu-ind, $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B})$ denotes the Section 4.2 game with the same New interface and per-user nonce discipline—user d using $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ when $b = 1$ and $(\mathsf{Rand}, \mathsf{Replay})$ when $b = 0$—equivalently, in the two-game notation of Section 2.4,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) = \left|\Pr[\mathcal{B}^{\mathsf{Real}^{\mathsf{mu}}_{\mathsf{RO}},\, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\mathsf{Ideal}^{\mathsf{mu}},\, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1]\right|,$$

where the superscripts name the entire multi-user construction interface in the corresponding world. The random oracle INT-RUP game is the same win-event game as above, with $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{PDec}^{\mathsf{RO}}_K$, and $\mathsf{Ver}^{\mathsf{RO}}_K$ replacing the public-permutation construction oracles and with direct adversary access to $\mathsf{RO}_{\mathsf{flat}}$ replacing (π, π^{-1}); its advantage is $\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}(\mathcal{B})$. Adversary queries to $\mathsf{RO}_{\mathsf{flat}}$ are finite-output and counted in $q_{\mathsf{RO}}(\mathcal{B})$ of Section 4.4. Construction-internal $\mathsf{RF}(\cdot)$ lookups — those made by $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{Dec}^{\mathsf{RO}}_K$, $\mathsf{PDec}^{\mathsf{RO}}_K$, $\mathsf{Ver}^{\mathsf{RO}}_K$, or a committing challenger — are not.

4.4 Resources

Adversary resources are measured by fixed, execution-uniform caps on the number and size of oracle queries. An execution-uniform cap is one that holds on every execution path, including over adversary randomness, oracle randomness, and adaptive query choices. Proofs may also introduce local execution-uniform caps, such as q_e for the number of accepted encryption queries, solely to index a finite hybrid sequence. Such local caps are part of the adversary's fixed query bounds, but they are not listed as headline resource parameters unless they affect the final bound.

Table 6 gives the compact notation summary used in later theorems. Lowercase quantities are realized counts in a random oracle experiment, while uppercase quantities are execution-uniform caps used in the public-permutation bounds. The detailed definitions below fix exactly which computations contribute to each count.

N: the construction interface cost, measured as the number of Keccak-p[1600, 12] calls that the adversary's construction interface queries induce in the real TW128 computation after the validity convention above has been applied. In ideal games, the same accounting applies to the corresponding real TW128 computation on the valid query, even when an ideal oracle answers by table lookup or without running the construction. N counts at per-duplex-call granularity, summing each root initialization, associated data duplex block, root message duplex block, leaf initialization, leaf message duplex block, and aggregation duplex block across the entire query sequence. Valid plaintext-computing computations and valid AE-style non-replay verification computations are counted whether they accept or reject; replay-table verification hits are transcript-consistency checks and are not charged to N or to the verification-tag terms. In the hash and committing games there is no adversary construction oracle: for the $\mathsf{H}_{\mathsf{TW128}}$ collision game, N counts the s deterministic encryptions on the submitted inputs, and for the preimage game it counts both challenger encryptions: the source encryption computing $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$ and the one deterministic encryption on the submitted input; for CMTDs-ℓ, N counts only the deterministic decryption checks performed by the challenger on the common ciphertext; while for the CMTs-ℓ source games of Corollary 6, N counts only the deterministic encryption checks actually used by the challenger to compare the s ciphertexts.

N_{prim}: the number of direct adversary queries to the primitive interface (the pair (π, π^{-1}) in public permutation games; the simulator's $\mathsf{RO}_{\mathsf{flat}}$ table is internal and is not exposed to the application adversary in those games).

$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$: the total query-sequence cost. The flat sponge lift of Section B is applied at N_{tot} because the indifferentiability replacement sees both the construction-internal sponge calls counted by N and the adversary's direct primitive-interface queries counted by N_{prim} in one chronological public primitive transcript.

$q_{\mathsf{v}}(\mathcal{A})$, Q_{v}: the number of verification queries with valid inputs that are not replay-table hits, and a cap on it. Used in the verification terms of the direct mu-ind, AE, and INT-RUP bounds of Section 5. Every counted verification query contributes at least one verification/decryption construction call in the N accounting, so $q_{\mathsf{v}}(\mathcal{A}) \leq N$.

$q_{\mathsf{RO}}(\mathcal{B})$, Q_{RO}: the number of direct finite-output queries to $\mathsf{RO}_{\mathsf{flat}}$ by a random oracle model adversary $\mathcal{B}$, and a cap on it. Repeated direct queries on the same Y are counted again. Construction-internal $\mathsf{RF}(\cdot)$ lookups are not counted. After the derived-adversary construction, q_{RO} counts only simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls from the adversary's direct primitive-interface queries, not the construction calls already counted in N for the flat sponge lift.

$n_{\mathsf{leaf}}(\mathcal{B})$, N_{leaf}: the number of distinct construction-generated final leaf transcripts in the execution of $\mathcal{B}$, and a cap on it. Distinct final leaf transcripts are generated by valid encryption computations, by valid plaintext-computing computations, by valid AE-style verification computations (whether they accept or reject), and by hash and committing challenger construction computations that pass the early validity checks, including the preimage game's source encryption.

For a byte string X,

$$\mathrm{leaves}(X) = \max\bigl(0,\ \lceil \|X\|/\beta \rceil - 1\bigr)$$

counts the leaf chunks induced by a body byte string of length $\|X\|$. Thus n_{leaf} is bounded by the sum of leaves(X) over the valid construction computations that create final leaf transcripts, where X is the plaintext body for encryption computations and the submitted ciphertext body for plaintext-computing, verification, and committing decryption checks. Each distinct final leaf transcript contains a final leaf duplex call counted in N, so $n_{\mathsf{leaf}}(\mathcal{B}) \le N$ unconditionally. The ratio n_{leaf}/N is largest when most processed bytes lie in leaf chunks, where each full leaf contributes $1 + \beta/\rho = 50$ duplex calls to N per final leaf transcript.

All theorems are stated for adversaries whose execution-uniform caps hold at the stated bounds.

4.5 Loss Terms

Five closed-form expressions recur across the security bounds—Lift_c throughout, with $\mathsf{KeyGuess}_k$, $\mathsf{RootColl}_\tau$, $\mathsf{RootPre}_\tau$, and $\mathsf{LeafColl}_\tau$ in some. We define them here, after the resource caps that parameterize them.

Flat sponge loss. The flat sponge differentiating loss is

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1}.$$

For $N_{\mathsf{tot}} < 2^c$ it upper-bounds the [BDPVA08] sponge-differentiating advantage; Lemma 18 carries out the derivation from the [BDPVA08, Lemmas 4 and 5] product bounds through the Weierstrass product inequality. For $N_{\mathsf{tot}} \ge 2^c$, $\mathsf{Lift}_c(N_{\mathsf{tot}}) \ge 1$ and the advantage bound is trivial without any appeal to indifferentiability, and already at $N_{\mathsf{tot}} \ge 2^{c/2}$ the bound exceeds $1/2$ and provides no meaningful security.

Key-guessing term. The ideal system key-guessing term of [BDPVA11] is

$$\mathsf{KeyGuess}_k(Q) = Q/2^k.$$

It accounts for direct $\mathsf{RO}_{\mathsf{flat}}$ queries whose inputs satisfy the decoder $\mathsf{KeyedTWOut}$ of Table 12. Each such query is an adaptive test of the decoded candidate key against the hidden key K, independent of the query's requested output length. Lemma 10 bounds the probability that any of Q direct queries tests the right key by $Q/2^k$.

Root-collision term. The s-way root tag multi-collision term is

$$\mathsf{RootColl}_\tau(Q, s) = \binom{Q+s}{s} / 2^{\tau(s-1)},$$

with the abbreviation $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(Q, s) = \mathsf{RootColl}_\tau(Q, s)$ at $\tau = \tau_{\mathsf{root}}$. The $+s$ is candidate-sequence slack for the s final root transcript inputs generated by the challenger after the adversary outputs its tuples. These lookups are construction-internal and are not added to $q_{\mathsf{RO}}(\mathcal{B})$; after the flat sponge lift of Section B, the public-permutation bounds instantiate Q as N_{prim}.

Root-preimage term. The root tag preimage term is

$$\mathsf{RootPre}_\tau(Q) = (Q + 1)/2^\tau,$$

with the abbreviation $\mathsf{RootPre}_{\tau_{\mathsf{root}}}(Q) = \mathsf{RootPre}_\tau(Q)$ at $\tau = \tau_{\mathsf{root}}$. It is the preimage analog of $\mathsf{RootColl}_\tau$: a single query-independent target tag replaces the s-way internal collision, one challenger lookup replaces the s final-root lookups, and the exponent drops from $\tau(s-1)$ to τ. As in the root-collision term, the flat sponge lift gives $Q = N_{\mathsf{prim}}$.

Leaf-collision term. The known-key leaf tag collision term is

$$\mathsf{LeafColl}_\tau(Q) = \binom{Q}{2}/2^\tau.$$

It is the pairwise birthday bound on τ-bit leaf tag projections used by the collision-resistance theorem (Theorem 8). In this game the keys are adversary-supplied, so direct primitive queries can pre-read candidate leaf tag points; after the flat sponge lift this gives $Q = N_{\mathsf{prim}} + N_{\mathsf{leaf}}$. The hidden-key AE-style bounds instead carry the linear same-slot term $n_{\mathsf{leaf}}/2^{\tau_{\mathsf{leaf}}}$ of Lemma 11, written inline.

5 Authenticated Encryption Security

This section proves the AEAD security of TW128. The first theorem is a random oracle multi-user indistinguishability bound. The public-permutation theorem then follows by the flat sponge lift of Section B, and the all-in-one AE bound is the single-user corollary. The section then proves INT-RUP integrity in the separated-interface model of [ABL+14]. The root tag hash and committing results are proved separately in Section 6; all concrete instantiations are summarized in Section 7.

All random oracle adversaries in this section are flat-RO adversaries (Section 4.3) with the displayed resources, and the flat sponge lift transfers each random oracle bound using the derived-adversary cap $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$.

Theorem 1 (Random Oracle Multi-User Indistinguishability)**.** *For every flat-RO mu-ind adversary $\mathcal{B}$ with execution-uniform user cap D and resource counts of Section 4.2,*

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \frac{D(D-1)}{2^{k+1}} + D\cdot \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Here q_{v} is the non-replay verification-query count of Section 4.4.

Proof. Use the pre-sampled-key formulation of Section 4.2: sample keys $K_1,\dots,K_D$ at the start of the experiment, and let the d-th New call activate K_d, unused keys being ignored. Let coll denote the event that two of the pre-sampled keys $K_1,\dots,K_D$ collide; $\Pr[\mathsf{coll}] \le \binom{D}{2}/2^k = D(D-1)/2^{k+1}$ by union bound. In the random oracle model, use a standard hybrid over the D users: H_d answers users $1,\dots,d$ with $(\mathsf{Rand},\mathsf{Replay})$ and users $d+1,\dots,D$ with the real construction interface, so H_0 is the real mu-ind game and H_D is the ideal one. Let $H_d^\star$ be H_d with a fixed output bit on coll. Since coll has the same probability in all hybrids,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) \le \Pr[\mathsf{coll}] + \left|\Pr[\mathcal{B}^{H_0^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_D^\star} \Rightarrow 1]\right|.$$

By the triangle inequality, it remains to bound each adjacent stopped hybrid. For each d, Lemma 16 gives

$$\left|\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Summing across $d = 1,\dots,D$ gives $D\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$ for the key-test contribution. The leaf tag contribution sums exactly:

$$\sum_d \frac{n^{(d)}_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} = \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}},$$

since $\sum_d n^{(d)}_{\mathsf{leaf}} = n_{\mathsf{leaf}}(\mathcal{B})$. The verification-tag contribution satisfies

$$\sum_d q^{(d)}_{\mathsf{v}}/2^{\tau_{\mathsf{root}}} = q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Combining these estimates with the coll penalty gives the stated RO bound. □

Theorem 2 (Multi-User Indistinguishability)**.** *For every mu-ind adversary $\mathcal{A}$ against $\mathsf{TW128}$ with execution-uniform user cap D and the global resources of Section 4.2,*

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{D(D-1)}{2^{k+1}} + D \cdot \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the mu-ind game, there is a random oracle mu-ind adversary $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$, $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$, and the per-user construction-side caps of Section 4.2 such that

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}).$$

Theorem 1 bounds the random oracle term. Substituting $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$ gives the stated bound. ☐

Corollary 1 (All-in-One AE Security)**.** *For every nonce-respecting AE adversary $\mathcal{A}$ against $\mathsf{TW128}$ with the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Build a mu-ind adversary $\mathcal{A}^{\mu}$ that makes one New query, runs $\mathcal{A}$, and forwards each encryption and verification query of $\mathcal{A}$ to user 1. The primitive interface is forwarded unchanged. The real and ideal views of $\mathcal{A}$ in the all-in-one AE experiment are exactly the real and ideal views of $\mathcal{A}^{\mu}$ in the mu-ind experiment with $D = 1$, and the resource counts are unchanged. Applying Theorem 2 with $D = 1$ eliminates the key-collision term and gives the stated bound. ☐

Theorem 3 (Random Oracle $\mathsf{INT\text{-}RUP}$ Integrity)**.** *For every flat-RO $\mathsf{INT\text{-}RUP}$ adversary $\mathcal{B}$ against $\mathsf{TW128}$ with nonce-respecting encryption queries and the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Plaintext-computing queries contribute to N and n_{leaf}, but not to q_{v}.

Proof. Work in the flat-RO $\mathsf{INT\text{-}RUP}$ game of Section 4.3. Let forge be the event that a valid non-replay verification query accepts. Let $\mathsf{bad}_{\mathsf{key}}$ be the event that a direct $\mathsf{RO}_{\mathsf{flat}}$ query decodes under $\mathsf{KeyedTWOut}$ to the hidden key K and a counted output-point class, and let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that some construction-generated final leaf transcript receives the same τ_{leaf}-bit leaf tag projection as a distinct final leaf transcript that shares its key, nonce, and leaf index and is generated by an encryption computation (the same-slot event of Lemma 11). Final leaf transcripts generated by encryption, plaintext-computing, and verification computations are all included in n_{leaf}.

The key-test term is unchanged by the plaintext-computing oracle. The key-renaming argument of Lemma 13 applies with plaintext oracle replies added to the visible prefix: under the online renaming $K_0 \mapsto K_1$, every construction lookup that determines a returned ciphertext, plaintext block, leaf tag, or root tag is paired with a distinct renamed lookup carrying the same sampled value. A plaintext reply is obtained by XORing the submitted ciphertext block with the same coupled keystream value, and the recomputed tag is not returned. Thus, before $\mathsf{bad}_{\mathsf{key}}$, the hidden key remains uniform outside the candidate keys already tested by direct queries, and Lemma 10 gives

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \leq \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

For the leaf term, before $\mathsf{bad}_{\mathsf{key}}$ no direct query has pre-read a hidden-key final leaf tag point: such a query would decode under $\mathsf{KeyedTWOut}$ to K and the corresponding

$\mathcal{L}_{\mathsf{tag}}(j)$ class. Encryption queries are nonce-respecting, plaintext-computing replies are keystream projections at stream output points, and the visible view agrees with the all-reject continuation of Lemma 15 until the first accepting non-replay submission, the event forge. The same-slot case of Lemma 11, applied to the enlarged set of construction computations, gives

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}} \cup \mathsf{forge}] \leq \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}}.$$

It remains to bound accepting verification before $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$. The INT-RUP root-guessing lemma (Lemma 15) gives

$$\Pr[\mathsf{forge} \text{ before } \mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}] \leq q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Combining the three bounds through the first-occurrence decomposition of $\Pr[\mathsf{forge}]$ proves the theorem. □

Theorem 4 (INT-RUP Integrity)**.** *For every INT-RUP adversary $\mathcal{A}$ against TW128 with nonce-respecting encryption queries and the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the INT-RUP game, there is a random oracle INT-RUP adversary $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$ such that

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{RO}}(\mathcal{B}).$$

Theorem 3 bounds the random oracle term, and substituting the derived-adversary resource caps gives the stated bound. □

6 Root Tag Hash Security and Committing Security

This section proves the digest claim for the root of the TW128 tree. The construction adapts the KangarooTwelve final-node pattern to authenticated encryption: leaf tags play the role of chaining values, and the root emits the single tag returned by $\mathsf{H}_{\mathsf{TW128}}$. The proof therefore tracks the final root inputs that can matter in the hash and committing games. The Root Tag Candidate Sequence lemma (Lemma 2) collects those inputs into a chronological candidate sequence, and the Root Tag Hash and Target-Hit Bound (Lemma 3) turns a collision or target hit among the candidates into the matching adaptive bound of Section A, controlling the chance that distinct inputs land on the same τ_{root}-bit root tag projection.

The hash random oracle theorems—collision (Theorem 5) and preimage (Theorems 6 and 7)—are then short instantiations, differing only in which case of the bound they invoke, whether the target is external or sampled from a fixed input slice, and whether a leaf tag collision term is needed when distinct inputs may use different ciphertext bodies. CMTDs-4 (Theorem 11) and the remaining committing notions reuse the collision case. Section B transfers each instance to the public permutation setting, Section 7 records the concrete bounds, and Corollary 6 forwards the remaining [BH22] committing notions to the same root-collision bound.

The instantiations rest on two structural separation facts, recorded first: distinct contexts (K, U, A) reach distinct final root transcripts regardless of the leaf tags they aggregate (Opening Triple Separation, Proposition 1), and—absent a collision among those leaf tags—so do distinct full inputs (Final-Root Input Separation, Proposition 2).

6.1 Structural Separation

Proposition 1 (Opening Triple Separation)**.** *If two TW128 root computations have distinct (K, U, A) triples, then their bridged final root transcript inputs under $\mathsf{flat}(\cdot)$ are distinct, regardless of whether each computation is an encryption or a decryption check on a common ciphertext.*

Proof. Suppose for contradiction that $\mathsf{flat}(R_i) = \mathsf{flat}(R_j)$. Since the bridge $\mathsf{flat}(\cdot)$ is injective on well-formed transcript prefixes (Lemma 7), the native flattened inputs of the two final-root prefixes are equal. Then Lemma 5 recovers the full duplex-call sequence and reads off the pair $(\sigma, \sigma_{\mathsf{ph}})$ for each call. The seven 4-bit phase suffixes are pairwise distinct, so the σ_{init}, $\sigma^{\pm}_{\mathsf{ad}}$, $\sigma^{\pm}_{\mathsf{msg}}$, and (when present) $\sigma^{\pm}_{\mathsf{agg}}$ calls in the recovered sequence are unambiguously identified. The aggregation phase is present exactly when $n \geq 1$, and $n = \text{leaves}(C)$ is determined by the common ciphertext, so the two recovered sequences either both carry an aggregation phase or both omit it; either way the recovery of (K, U, A) below uses only the $\sigma_{\mathsf{init}}$and AD calls, which precede any message or aggregation block.

Equality of the recovered sequences forces equality of the σ_{init}call, whose σ is $p_{\mathsf{root}} \,\|\, K \,\|\, U$ with fixed-width K and U, hence $K_i = K_j$ and $U_i = U_j$. It also forces equality of the recovered AD subsequences. By the AD bullet of Lemma 5 the associated data phase contributes a σ^{+}_{ad}-tagged block exactly when the associated data is nonempty, so the recovered sequences agree on the presence of the AD phase. If exactly one of A_i, A_j were empty the recovered sequences would differ in the presence of a σ^{+}_{ad} call, contradicting their equality; hence either both are empty, giving $A_i = A_j = \varepsilon$, or both are nonempty, in which case the phase-recovery sub-claim of Lemma 5 inverts the ρ-byte-block partition map on the AD byte string and equal AD σ-block sequences imply $A_i = A_j$. In all cases $(K_i, U_i, A_i) = (K_j, U_j, A_j)$, contradicting the hypothesis of distinct triples. □

Triple Separation handles distinct contexts; when inputs share a context but differ in their messages, distinctness of the final root transcripts needs the absence of a leaf tag collision, recorded next.

Proposition 2 (Final-Root Input Separation)**.** *Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts receive the same τ_{leaf}-bit RF projection. On $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, if two TW128 construction computations reach final root transcripts with $\mathsf{flat}(R) = \mathsf{flat}(R')$, then they share the same context (K, U, A) and the same full ciphertext C. Equivalently, on $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ computations with distinct (K, U, A, C) reach distinct bridged final root transcripts.*

Proof. By bridge injectivity (Lemma 7), $\mathsf{flat}(R) = \mathsf{flat}(R')$ gives equal native flattened final-root inputs. Lemma 5 then recovers the same root initialization $p_{\mathsf{root}} \parallel K \parallel U$—hence the same K and U—the same associated data blocks—hence the same A—the same root ciphertext chunk, and the same leaf count and ordered leaf tag sequence. Under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, Leaf-Transcript Recovery (Lemma 6) identifies the equal leaf tag sequences with the same final leaf transcripts, hence the same leaf ciphertext bodies; with the common root ciphertext chunk this gives the same full C. □

6.2 Root Tag Candidate Bound

Lemma 2 (Root Tag Candidate Sequence)**.** *Consider a random oracle game of Section 4.3 in which the adversary $\mathcal{B}$ makes direct $\mathsf{RO}_{\mathsf{flat}}$ queries and the challenger runs $m \geq 1$ deterministic encryption or decryption checks at fixed points in the game, the i-th reaching a final root transcript R_i whose root tag is read by a single $\mathsf{RO}_{\mathsf{flat}}$ lookup at $\mathsf{flat}(R_i)$. Every execution then admits a chronological sequence of root tag inputs such that*

1. *each distinct direct query, when it is made, whose input Y decodes under* KeyedTWOut *to a root tag output-point class—$\mathcal{R}^{+}_{\mathsf{msg}}$ (the final root message block, which carries the root tag only on the no-leaf path) or $\mathcal{R}_{\mathsf{tag}}$ (the aggregated root tag)—is appended as a candidate before its* $\mathsf{RO}_{\mathsf{flat}}$ *lookup, and contributes at most one candidate; multi-chunk computations also reach $\mathcal{R}^{+}_{\mathsf{msg}}$ with zero requested output before aggregation, so this predicate over-approximates the root tag candidate set, which only loosens the count;*

2. *if the challenger reaches its checks, each* $\mathsf{flat}(R_i)$ *is in the sequence, appended—if not already present—immediately before the i-th root tag answer is sampled;*

3. *no candidate's* $\mathsf{RO}_{\mathsf{flat}}$ *value is read before its append step, so the sequence satisfies the predictability and unrevealed-value premises of Lemma 9; and*

4. *its length is at most $q_{\mathsf{RO}}(\mathcal{B}) + m$.*

Proof. The committing and hash games have no persistent challenger-sampled key: all keys are adversary-supplied except for the hidden source in the standard preimage game. A direct query to a keyed transcript is therefore one of $\mathcal{B}$'s visible $\mathsf{RO}_{\mathsf{flat}}$ queries counted in $q_{\mathsf{RO}}(\mathcal{B})$, not a separate construction query. Each direct-query candidate is appended before that direct query reads its value. The append predicate depends on Y alone through the exact decoder of Lemma 7, which ignores the requested output length, and by Output-Point Separation (Corollary 8) each input decodes to at most one output-point class; hence each distinct direct query contributes at most one candidate, giving (1).

If the challenger reaches its m checks, append $\mathsf{flat}(R_i)$ immediately before the i-th root tag answer is sampled, unless it is already present. The intermediate $\mathsf{RO}_{\mathsf{flat}}$ lookups of check i lie at non-root tag output points, so by Corollary 8 they do not read $\mathsf{flat}(R_i)$. If $\mathsf{flat}(R_i)$ was touched earlier—by a direct query or by an earlier check $j < i$—then Lemma 7 and Corollary 8 make that earlier access a final-root lookup at the same address, which was therefore already appended, so the input is in the sequence and is not appended again. Otherwise its value is still unread when appended. Either way no candidate's value is read before its append step, giving (2) and (3). At most $q_{\mathsf{RO}}(\mathcal{B})$ direct candidates and m challenger candidates are appended, giving (4). □

Lemma 3 (Root Tag Hash and Target-Hit Bound)**.** *Consider a keyless TW128 random oracle game (Section 4.3) in which $\mathcal{B}$ makes direct* $\mathsf{RO}_{\mathsf{flat}}$ *queries and the challenger runs deterministic encryption or decryption checks reaching final root transcripts, with win event* W*. By the Root Tag Candidate Sequence lemma (Lemma 2) the root tag inputs form a chronological candidate sequence meeting the premises of Lemma 9 with $\tau = \tau_{\mathsf{root}}$. Writing $q_{\mathsf{RO}} = q_{\mathsf{RO}}(\mathcal{B})$:*

1. *if* W *forces s checks to reach pairwise distinct bridged final root transcripts carrying a common root tag, then, with $m = s$ checks,* $\Pr[\mathsf{W}] \leq \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}, s)$*;*

2. *if* W *forces a single check ($m = 1$) to carry a root tag equal to a target fixed independently of* $\mathsf{RO}_{\mathsf{flat}}$*, then* $\Pr[\mathsf{W}] \leq \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$*;*

3. *if the game first runs a designated source check, reveals its root tag, and* W *forces one later check to reach a distinct bridged final root transcript carrying that designated tag, then, with $m = 2$ checks,* $\Pr[\mathsf{W}] \leq \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$*.*

Proof. In each case the candidate sequence has the stated length and meets the premises of Lemma 9. *(1)* The s distinct candidates carrying a common τ_{root}-projection are an s-way collision; the collision case with $L = q_{\mathsf{RO}} + s$ gives $\binom{q_{\mathsf{RO}}+s}{s} \cdot 2^{-\tau_{\mathsf{root}}(s-1)} = \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}, s)$. *(2)* The candidate hitting the fixed target is a preimage; the preimage case with $L = q_{\mathsf{RO}} + 1$ gives $(q_{\mathsf{RO}} + 1) \cdot 2^{-\tau_{\mathsf{root}}} = \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$. *(3)* Let j_0 be the position of the designated source check. The later check is required to be a distinct bridged final root transcript, so it is a candidate position $j \neq j_0$ with the same τ_{root}-bit projection. The candidate sequence has length at most $L = q_{\mathsf{RO}} + 2$, and the second-preimage corollary of Lemma 9 gives $(L-1)2^{-\tau_{\mathsf{root}}} = (q_{\mathsf{RO}}+1)2^{-\tau_{\mathsf{root}}} = \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}})$. $\square$

6.3 Hash Random Oracle Bounds

Theorem 5 (Random Oracle Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every $s \geq 2$ and every s-way collision adversary $\mathcal{B}$ in the TW128 random oracle model,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s).$$

Proof. Run the random oracle collision game. Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts receive the same τ_{leaf}-bit RF projection; by the adversary-supplied-key bound of Lemma 11, $\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}] \leq \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B}))$, so it remains to bound the win under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$. After rejecting unless the s output inputs are pairwise distinct and in the $\mathsf{TW128}$ domain, the challenger performs s deterministic encryptions $\mathsf{Enc}^{\mathsf{RO}}_{K_i}(U_i, A_i, M_i)$, reaching $R_1, \ldots, R_s$, all carrying a common root tag T on a win. Pairwise distinct tuples (K_i, U_i, A_i, M_i) have pairwise distinct (K_i, U_i, A_i, C_i): two tuples differing in their context already differ in (K, U, A), while two sharing a context but differing in the message yield distinct ciphertexts, since for a fixed context encryption maps the message to the ciphertext bijectively. So on $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ Final-Root Input Separation (Proposition 2) makes $\mathsf{flat}(R_1), \ldots, \mathsf{flat}(R_s)$ pairwise distinct. The Root Tag Hash and Target-Hit Bound (Lemma 3, part 1) bounds the win under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ by $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$; adding $\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}]$ gives the theorem. $\square$

Theorem 6 (Random Oracle Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Fix byte lengths $0 \leq a \leq A_{\max}$ and $0 \leq \mu \leq M_{\max}$, and let $m_{a,\mu} = k+\nu+8a+8\mu$. For every Pre adversary $\mathcal{B}$ against $\mathsf{H}_{\mathsf{TW128}}$ on the fixed input slice $\mathcal{X}_{a,\mu}$ in the TW128 random oracle model (Section 4.3),*

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; & \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) \\ & + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) + 2^{\tau_{\mathsf{root}} - m_{a,\mu}}.\end{aligned}$$

Proof. Run the random oracle Pre game. The challenger samples $X^* \leftarrow\!\$\; \mathcal{X}_{a,\mu}$, writes $X^* = (K^*, U^*, A^*, M^*)$, computes $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$, gives T^* to $\mathcal{B}$, and later checks the output $X = (K, U, A, M)$. Let same be the event $X = X^*$. For a fixed random oracle table and fixed adversary coins, the adversary's output is a function of the τ_{root}-bit challenge tag T^*, so over the uniform source X^* it can equal X^* with probability at most $2^{\tau_{\mathsf{root}}}/2^{m_{a,\mu}} = 2^{\tau_{\mathsf{root}} - m_{a,\mu}}$. Hence $\Pr[\mathsf{same}] \leq 2^{\tau_{\mathsf{root}} - m_{a,\mu}}$.

It remains to bound winning executions with $X \neq X^*$. Let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts in the source and output checks receive the same τ_{leaf}-bit RF projection. By the leaf-collision bound of Lemma 11,

$$\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}] \leq \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})).$$

On $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, the distinct tuples X^* and X have distinct final root transcripts: distinct contexts are separated by Proposition 1, while equal contexts and distinct messages produce distinct ciphertexts, and then Proposition 2 applies. A win with $X \neq X^*$ therefore forces the later output check to hit the designated source tag T^* at a distinct bridged final root transcript. The sampled-source case of Lemma 3 bounds this probability by $\mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}))$. Adding the source-guess and leaf-collision events gives the theorem. $\square$

Theorem 7 (Random Oracle Everywhere Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every query-independent target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ and every ePre adversary $\mathcal{B}$ in the* TW128 *random oracle model (Section 4.3),*

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) = \frac{q_{\mathsf{RO}}(\mathcal{B})+1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Run the random oracle preimage game for the query-independent target T^*, which is independent of $\mathsf{RO}_{\mathsf{flat}}$. After rejecting unless the output input $X = (K, U, A, M)$ is in the TW128 domain, the challenger performs one encryption $\mathsf{Enc}^{\mathsf{RO}}_K(U, A, M)$, reaching a final root transcript R. A win means $\mathsf{H}_{\mathsf{TW128}}(X) = T^*$, i.e. the root tag $\lfloor \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(R)) \rfloor_{\tau_{\mathsf{root}}}$ at R equals the fixed target T^*. The Root Tag Hash and Target-Hit Bound (Lemma 3, part 2, with target T^*) gives $\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{RO}}(\mathcal{B}) \leq \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}))$. □

6.4 Public Permutation Hash Bounds

The flat sponge lift transfers the preceding random oracle bounds to the public permutation model. For these one-world hash games, the lift contributes $\mathsf{Lift}_c(N_{\mathsf{tot}})$, and the derived random oracle adversary has $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$; challenger construction computations—including the preimage game's source encryption—are construction-internal work counted in N.

Theorem 8 (Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every $s \geq 2$ and every adversary $\mathcal{A}$ against the s-way multi-collision resistance of $\mathsf{H}_{\mathsf{TW128}}$ in the sense of [BH22], with the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s).$$

Proof. By the Lift Application corollary (Corollary 9) for the collision game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$ and $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$. The random oracle bound of Theorem 5 is nondecreasing in both $q_{\mathsf{RO}}(\mathcal{B})$ and $n_{\mathsf{leaf}}(\mathcal{B})$, so the monotone form of Corollary 9 gives the stated bound. The s colliding inputs may differ in their messages, hence in their ciphertext bodies; the $\mathsf{LeafColl}$ term accounts for the only way two distinct inputs sharing a context (K, U, A) can reach the same final root transcript—a collision among the leaf tags it absorbs—while distinct contexts already reach distinct final root transcripts by Opening Triple Separation (Proposition 1). □

Corollary 2 (Two-Way Collision Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Specializing Theorem 8 to $s = 2$ gives the classical two-way collision resistance of [RS04]: for every collision adversary $\mathcal{A}$ against $\mathsf{H}_{\mathsf{TW128}}$ with the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2),$$

with $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2) = \binom{N_{\mathsf{prim}}+2}{2}/2^{\tau_{\mathsf{root}}}$.

Proof. The $s = 2$ case of Theorem 8; [BH22] note that s-way multi-collision resistance at $s = 2$ is the classical collision resistance of [RS04]. □

Corollary 3 (Second-Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every adversary against $\mathsf{H}_{\mathsf{TW128}}$, with the resources of Section 4.4, in either the standard second-preimage (Sec) or everywhere second-preimage (eSec) sense of [RS04], the corresponding advantage is bounded by the two-way collision bound of Corollary 2:*

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, 2).$$

Proof. By [RS04, Proposition 6], collision resistance implies both Sec and eSec. Applying this implication to the two-way collision bound of Corollary 2 gives the stated bound. □

Theorem 9 (Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *Fix byte lengths $0 \le a \le A_{\max}$ and $0 \le \mu \le M_{\max}$, and let $m_{a,\mu} = k + \nu + 8a + 8\mu$. For every adversary against $\mathsf{H}_{\mathsf{TW128}}$, with the resources of Section 4.4, in the standard preimage (Pre) sense of [RS04] on the fixed input slice $\mathcal{X}_{a,\mu}$,*

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathit{TW128}}(\mathcal{A}) \; &\le \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) \\ &\quad + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) + 2^{\tau_{\mathsf{root}} - m_{a,\mu}}.\end{aligned}$$

Proof. By the Lift Application corollary (Corollary 9) for the preimage game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$; per Section 4.4, these caps cover both challenger encryptions, the source encryption's duplex calls and final leaf transcripts included. The random oracle bound of Theorem 6 is nondecreasing in both resources, so the monotone form of Corollary 9 gives the stated bound. □

Theorem 10 (Everywhere Preimage Resistance of $\mathsf{H}_{\mathsf{TW128}}$)**.** *For every target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ fixed in advance and every ePre adversary $\mathcal{A}$ against $\mathsf{H}_{\mathsf{TW128}}$ with the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathit{TW128}}(\mathcal{A}) \; \le \; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) \; = \; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{N_{\mathsf{prim}} + 1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. By the Lift Application corollary (Corollary 9) for the preimage game—whose target T^* is fixed before the primitive sampling—the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$, and the random oracle bound of Theorem 7 is nondecreasing in $q_{\mathsf{RO}}(\mathcal{B})$, so the monotone form of Corollary 9 gives the stated bound. With one fixed target tag, Theorem 7 applies a candidate-preimage bound against that target and drops the exponent from $\tau_{\mathsf{root}}(s-1)$ to τ_{root}. □

Corollary 4 (Root Tag as a Compact Commitment)**.** *The wrapper $\mathsf{H}_{\mathsf{TW128}}$ is a secure hash of the full encryption context (K, U, A, M) in the sense of [RS04]: collision-resistant (Theorem 8), preimage-resistant on each fixed input slice (Theorem 9), second-preimage-resistant (Corollary 3), and everywhere preimage-resistant (Theorem 10). Under the implemented parameters and work cap $N_{\mathsf{tot}} \le 2^{63}$, these advantages are at most 2^{-128} (Corollary 7), so the single τ_{root}-bit root tag is a binding compact commitment to (K, U, A, M) with standard and everywhere preimage resistance, native to the mode.*

Proof. Collision resistance is Theorem 8; standard preimage resistance on fixed input slices is Theorem 9; second-preimage resistance follows from collision resistance by Corollary 3; and everywhere preimage resistance is Theorem 10. These guarantees quantify the adversary's success at the same τ_{root}-bit projection of the final root transcript, and Corollary 7 evaluates them at the implemented parameters. Binding is the compact-commitment property supplied by collision and second-preimage resistance. Standard preimage resistance is the property that a tag produced from a uniformly sampled fixed-length opening does not reveal an efficiently findable opening; everywhere preimage resistance is the corresponding fixed-target property for tags chosen independently of the primitive. □

6.5 Committing Consequences

Theorem 11 (Random Oracle CMTDs-4)**.** *For every $s \ge 2$ and every s-way CMTDs-4 adversary $\mathcal{B}$ in the TW128 random oracle model (Section 4.3),*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \; \le \; \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s) = \frac{\binom{q_{\mathsf{RO}}(\mathcal{B}) + s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. Run the random oracle CMTDs-4 game. After rejecting on any failed syntax, message-length, or pairwise-full-input check, the challenger performs s deterministic decryption checks $\mathsf{Dec}^{\mathsf{RO}}_{K_i}(U_i, A_i, C \| T)$ on the one common body $C \| T$, reaching $R_1, \dots, R_s$. On a winning execution the tuples (K_i, U_i, A_i, M_i) are pairwise distinct, and since all checks decrypt the common body, determinism of Dec makes the triples (K_i, U_i, A_i) pairwise distinct—equal triples would return equal messages, hence equal tuples. By Opening Triple Separation (Proposition 1) the candidates $\mathsf{flat}(R_1), \dots, \mathsf{flat}(R_s)$ are pairwise distinct (leaf tag collisions cannot merge them), and every accepting check carries the common root tag T. The Root Tag Hash and Target-Hit Bound (Lemma 3, part 1) gives $\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$. □

Corollary 5 (CMTDs-4 Committing Security)**.** *For every $s \ge 2$ and every s-way CMTDs-4 adversary $\mathcal{A}$ against* TW128 *with the resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. A CMTDs-4 winner gives one ciphertext $C \,\|\, T$ and s distinct inputs (K_i, U_i, A_i, M_i) such that $\mathsf{Dec}_{K_i}(U_i, A_i, C \,\|\, T) = M_i$ for every i. By TW128 tidiness (Lemma 1), each opened input re-encrypts to the same ciphertext: $\mathsf{Enc}_{K_i}(U_i, A_i, M_i) = C \,\|\, T$. In particular the opened inputs share the root tag T. For the displayed no-$\mathsf{LeafColl}$ bound, use the dedicated root-collision derivation of Theorem 11, lifted by Corollary 9: in this all-bodies-equal case, the single shared body C forces the triples (K_i, U_i, A_i) pairwise distinct; by Opening Triple Separation (Proposition 1), a leaf tag collision cannot merge two openings and no $\mathsf{LeafColl}$ term arises. □

Remaining committing notions. The hierarchy and s-way extension of the [BH22] committing notions are given in [BH22, §3, Figs. 4–5, and App. A]. We use those relations for the D-notions and a direct source-game argument for the CMTs-ℓ notions to preserve the resource split of Section 4.4.

Corollary 6 (Committing-Notion Root-Collision Bound)**.** *For every $s \ge 2$, every*

$$X \in \{\mathsf{CMTDs\text{-}1},\ \mathsf{CMTDs\text{-}3},\ \mathsf{CMTDs\text{-}4},\ \mathsf{CMTs\text{-}1},\ \mathsf{CMTs\text{-}3},\ \mathsf{CMTs\text{-}4}\},$$

and every s-way X-adversary $\mathcal{A}$ against TW128 *with the resources of Section 4.4,*

$$\mathbf{Adv}^{X}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. CMTDs-4 is bounded directly by Corollary 5. For the remaining D-notions, the s-way hierarchy of [BH22, §3, Fig. 5 and App. A] gives, for $\ell \in \{1, 3\}$, $\mathbf{Adv}^{\mathsf{CMTDs\text{-}}\ell}_{\mathsf{TW128}}(\mathcal{A}) \le \mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A})$ for the same output distribution, so the resource counts are unchanged.

For CMTs-ℓ, the challenger performs s deterministic encryption checks, which supply the candidate final-root transcripts (Lemma 2 builds the candidate sequence for encryption and decryption checks alike). On a winning execution all s encryptions are equal, hence of equal length and with one common root tag T. Pairwise WiC_1-distinctness gives pairwise distinct keys, pairwise WiC_3-distinctness gives pairwise distinct (K, U, A) triples, and pairwise WiC_4-distinctness gives pairwise distinct full inputs; in the last case, equal triples and equal ciphertexts would decrypt deterministically to equal messages, contradicting full-input distinctness. Thus each CMTs-ℓ winner reaches pairwise distinct opening triples. Opening Triple Separation (Proposition 1) makes the s final-root candidates distinct while their τ_{root}-bit projections all equal T. The hierarchy argument does not change the output distribution or the resource counts, and the CMTs-ℓ encryption checks are the construction-internal work counted in N by Section 4.4.

By the Lift Application corollary (Corollary 9) for the X game, the derived random oracle adversary $\mathcal{B}$ has $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$. The Root Tag Hash and Target-Hit Bound (Lemma 3, part 1) then bounds its win by $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$ in the random oracle model, which the lift transfers to the public permutation setting at N_{prim}; the encryption checks are construction-internal work counted in N, not in N_{prim}. □

7 Concrete Security Summary

Sections 5 and 6 separate the security analysis into AEAD bounds and root tag hash and committing bounds. This section instantiates those bounds at the implemented $\mathsf{TW128}$ parameters and records the single concrete work cap used throughout the paper.

7.1 Concrete TW128 Bounds

The implemented parameters of Section 3.2 fix

$$k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256.$$

Substituting into Corollaries 1, 3 and 5 and Theorems 2, 4 and 8 to 10 gives the following closed-form bounds.

All-in-one AE.

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}}.$$

Multi-user indistinguishability.

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{DN_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}} + \frac{D(D-1)}{2^{257}}.$$

INT-RUP integrity.

$$\mathbf{Adv}^{\mathsf{INT\text{-}RUP}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}}}{2^{256}}.$$

***s*-way collision resistance of $\mathsf{H}_{\mathsf{TW128}}$.** For every $s \ge 2$,

$$\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+N_{\mathsf{leaf}}}{2}}{2^{256}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

For $s = 2$, this is the ordinary two-input collision bound:

$$\begin{aligned}\mathbf{Adv}^{\mathsf{Coll}}_{\mathsf{TW128}}(\mathcal{A}) \le{}& \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{(N_{\mathsf{prim}}+N_{\mathsf{leaf}})(N_{\mathsf{prim}}+N_{\mathsf{leaf}}-1)}{2^{257}} \\ &+ \frac{(N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.\end{aligned}$$

Second-preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$. By Corollary 3, the standard and everywhere [RS04] second-preimage advantages of $\mathsf{H}_{\mathsf{TW128}}$ are bounded by the same $s = 2$ collision expression above.

Preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$. For fixed byte lengths a, μ, let $m_{a,\mu} = k + \nu + 8a + 8\mu$. By Theorem 9,

$$\mathbf{Adv}^{\mathsf{Pre}[a,\mu]}_{\mathsf{TW128}}(\mathcal{A}) \leq \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{(N_{\mathsf{prim}}+N_{\mathsf{leaf}})(N_{\mathsf{prim}}+N_{\mathsf{leaf}}-1)}{2^{257}} + \frac{N_{\mathsf{prim}}+1}{2^{256}} + 2^{256-m_{a,\mu}}.$$

Under the implemented parameters $\nu = 256$, so the slice term is $2^{-256-8a-8\mu}$.

s-way CMTDs-4 committing security. For every $s \geq 2$, the all-bodies-equal special case drops the leaf tag term:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \ \leq \ \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

Remark 1 (Two-Key CMTDs-4 Specialization). Setting $s = 2$ recovers the ordinary two-key CMTDs-4 statement of [BH22]:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \ \leq \ \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+2}{2}}{2^{256}} \ = \ \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1) + (N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.$$

Everywhere preimage resistance of $\mathsf{H}_{\mathsf{TW128}}$.

$$\mathbf{Adv}^{\mathsf{ePre}}_{\mathsf{TW128}}(\mathcal{A}) \ \leq \ \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}}+1}{2^{256}}.$$

7.2 Concrete Security Claim

Corollary 7 (Concrete $\mathsf{TW128}$ Security)**.** *Under the implemented parameters $k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$, every adversary with work cap $N_{\mathsf{tot}} \leq 2^{63}$—and, in the multi-user case, user cap $D \leq 2^{63}$—has advantage at most 2^{-128} in each of TW128's notions: multi-user indistinguishability (Theorem 2), all-in-one AE (Corollary 1), INT-RUP integrity (Theorem 4), the [RS04] hash security of $\mathsf{H}_{\mathsf{TW128}}$—$s$-way collision resistance for every $s \geq 2$, standard and everywhere second-preimage resistance inherited from collision resistance, standard preimage resistance on every fixed input slice, and everywhere preimage resistance (Theorems 8 to 10 and Corollary 3)—and the [BH22] CMTDs-4 committing bound it yields (Corollary 5).*

Proof. The resource consequences of Section 4.4 give $N_{\mathsf{prim}} \leq N_{\mathsf{tot}}$, $Q_{\mathsf{v}} \leq N_{\mathsf{tot}}$, $N_{\mathsf{leaf}} \leq N_{\mathsf{tot}}$, and $N_{\mathsf{prim}} + N_{\mathsf{leaf}} \leq N_{\mathsf{tot}}$. The user cap D remains independent, since created-but-idle users need not contribute to N.

At $N_{\mathsf{tot}} \leq 2^{63}$, the flat-sponge term and each birthday-scale root or leaf term at $s = 2$ is at most a 2^{-131}-scale term, while each linear root-preimage, key-guessing, verification, or leaf tag term with no factor D is at most a 2^{-193}-scale term. The fixed-slice Pre degradation is at most 2^{-256}, since $m_{a,\mu} \geq k + \nu = 512$. Thus the largest non-multi-user bound is the $s = 2$ collision bound, which is below $4 \cdot 2^{-131} = 2^{-129}$; larger s only decreases the root-collision term. The all-in-one AE and INT-RUP bounds carry $\mathsf{Lift}_c(N_{\mathsf{tot}})$ as their only birthday-scale term. In the multi-user case,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) < 2^{-130} + 2^{-129} + 2^{-131} < 2^{-128},$$

using $D \leq 2^{63}$: the flat-sponge term is below 2^{-130}, the linear terms sum to $(DN_{\mathsf{prim}} + Q_{\mathsf{v}} + N_{\mathsf{leaf}})/2^{256} \leq (2^{126} + 2^{64})/2^{256} < 2^{-129}$, and $D(D-1)/2^{257} < 2^{-131}$. These estimates instantiate every closed form in Section 7.1. $\square$

Table 7: Concrete $\mathsf{TW128}$ security at the work cap $N_{\mathsf{tot}} \le 2^{63}$ (with $D \le 2^{63}$ for mu-ind), under $k = c = \nu = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$ and the resource consequences $N_{\mathsf{prim}}, Q_{\mathsf{v}}, N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$ and $N_{\mathsf{prim}} + N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$ of Section 4.4. The committing rows hold for every $s \ge 2$; larger s only shrinks the root-collision term.

Notion	Dominant scale	At the cap
All-in-one AE	Lift_c	$\le 2^{-128}$
Multi-user ind.	$D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$	$\le 2^{-128}$
INT-RUP	Lift_c	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ collision (Coll)	Lift_c, LeafColl, RootColl	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ second-preimage (Sec/eSec)	inherited from Coll	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ preimage (Pre)	Lift_c, LeafColl, RootPre, slice term	$\le 2^{-128}$
$\mathsf{H}_{\mathsf{TW128}}$ everywhere preimage (ePre)	$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$\le 2^{-128}$
CMTDs-4 [BH22]	Lift_c and RootColl	$\le 2^{-128}$

7.3 Discussion of the Bounds

The bounds of Section 7.1 aggregate the loss terms of Section 4.5, whose sizes at the work cap Table 7 records and whose dominance Section 7.2 proves.

At the concrete cap. At $N_{\mathsf{tot}} \le 2^{63}$, the hash and CMTDs-4 bounds have flat-sponge and root or leaf birthday terms on the same 2^{-131} scale, and the single-user AE and INT-RUP bounds retain the flat sponge loss as their only birthday-scale term. Thus the flat sponge loss is not always the largest summand: under the loose per-term cap, the $s = 2$ root-collision term $\binom{N_{\mathsf{prim}}+2}{2}/2^{256}$ is on the same scale as $\mathsf{Lift}_c(N_{\mathsf{tot}})$. The multi-user bound is the exception: its hybrid-induced term $D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$ can reach 2^{-130} at the extreme cap $D = N_{\mathsf{prim}} = 2^{63}$. Expressed in bytes, $N_{\mathsf{tot}} \le 2^{63}$ corresponds to at most $\rho \cdot 2^{63} = 167 \cdot 2^{63} < 2^{71}$ bytes of plaintext, associated data, ciphertext, and aggregation through the duplex (Section 3.2).

Tightness.

- $\mathsf{KeyGuess}_k(Q) = Q/2^k$. Tight at constants: a direct random-guess attack on a k-bit key matches the per-query success probability $1/2^k$. The multi-user bound's dominant term $D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}}) = DN_{\mathsf{prim}}/2^k$ is likewise tight at the leading constant: against the D independent user keys each direct guess matches some key with probability $D/2^k$, so N_{prim} guesses recover a user key—and distinguish—with probability $\approx DN_{\mathsf{prim}}/2^k$, matching the bound.

- $\mathsf{RootColl}_\tau(Q, s) = \binom{Q+s}{s}/2^{\tau(s-1)}$. Tight up to lower-order terms: it is the standard s-way multi-collision bound on a τ-bit projection of $\mathsf{RO}_{\mathsf{flat}}$, and a generic birthday-style adversary against the random oracle matches the leading-order term. It is the root term of the collision bound ($\mathsf{H}_{\mathsf{TW128}}$ Coll, Theorem 8). Its N_{prim} counts every primitive query, though by Output-Point Separation (Corollary 8) only those whose induced $\mathsf{RO}_{\mathsf{flat}}$ call lands at a root tag output point ($\mathcal{R}_{\mathsf{tag}}$, or $\mathcal{R}^{+}_{\mathsf{msg}}$ for single-chunk ciphertexts) can enter the candidate sequence; a class-filtered count would be no larger, and potentially smaller, but the displayed bound keeps the N_{prim} form for uniformity with Corollary 1 and Theorem 2, the birthday threshold at $\tau_{\mathsf{root}} = 256$ being unaffected.

- $\mathsf{RootPre}_\tau(Q) = (Q+1)/2^\tau$. Tight at constants: a generic preimage search against a fixed τ-bit target matches the per-query $2^{-\tau}$ rate. It is the root term of the standard and everywhere preimage bounds, where a single target tag yields a linear term.

- *Leaf tag term* $N_{\mathsf{leaf}}/2^{\tau_{\mathsf{leaf}}}$. Tight at constants by a nonce-respecting attack: encrypt a single two-chunk message under nonce U, obtaining ciphertext with one leaf chunk and root tag T, then submit non-replay verification queries that each replace the leaf chunk by fresh bytes of the same length, keeping T. Each query recomputes the leaf tag at the hidden slot $(K, U, 1)$ and accepts exactly when that fresh tag equals the encryption's hidden leaf tag T_1, since the aggregation string—and hence the recomputed root tag—is then unchanged. Each verification-side final leaf transcript therefore forges with probability $2^{-\tau_{\mathsf{leaf}}}$, matching the bound's per-transcript rate.

- *Known-key leaf collision* $\mathsf{LeafColl}_\tau(Q)$. Genuinely birthday-tight: when keys are adversary-supplied, direct queries can target leaf tag points, and a birthday search over chosen leaf bodies yields two distinct $\mathsf{H}_{\mathsf{TW128}}$ inputs sharing a leaf tag and hence colliding openings; the collision proof uses $Q = N_{\mathsf{prim}} + N_{\mathsf{leaf}}$.

- *Flat sponge lift.* The term $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1}$ dominates the random-permutation differentiating bound $f_P(N_{\mathsf{tot}})$ of [BDPVA08, Lemma 5] via Lemma 18. The remaining slack is the unimproved tightness of the [BDPVA08] simulator itself, which is an open problem on the sponge side and not specific to TW128.

Cross-scheme bound-level comparisons against prior sponge AEADs and the committing AEAD line of [BH22] are deferred to Sections 9.1 to 9.3.

8 Performance Evaluation

8.1 Optimized Implementations

We analyze a Go implementation of TW128[1] with optimized assembly paths for the performance-critical permutation and absorb/squeeze loops. We compare three vectorized assembly paths: `amd64` AVX-512, `amd64` AVX2, and `arm64` NEON using the Armv8.2 SHA-3 extension. The framing, domain separation, padding, tree bookkeeping, and dispatch remain in shared Go code. A pure Go path uses a scalar permutation and a scalar eight-instance loop, and serves both as a correctness oracle for the assembly cores and as a constant-time fallback on other architectures.

The tree structure of TW128 exposes two distinct permutation workloads. The root duplex—which absorbs the associated data, processes chunk 0, and absorbs the leaf tag aggregation transcript—is inherently sequential, since each call consumes the previous call's output. It is driven by a *single-duplex* ($\times 1$) core, except for its chunk-0 message phase. The leaf duplexes operate on disjoint state and disjoint inputs (Section 3), so complete leaf chunks are batched and run through an *eight-way* ($\times 8$) core. Leftover runs of two to seven complete chunks use narrower remainder kernels, while a single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail fall back to the single-duplex core.

Chunk 0 itself runs on the batched kernels rather than serially, a technique we refer to as *chunk* 0 *fusion*. The root duplex's chunk-0 message phase has the same kernel-visible schedule as a leaf chunk and differs only in its initial state. The implementation therefore seeds lane 0 of the first batched call with the root state, runs chunk 0 alongside the first leaf chunks, then writes lane 0's output state back to the root duplex before the root absorbs the leaf tags from that same call. This avoids a serial pass over chunk 0 and is the reason the cycles-per-byte curve drops as soon as complete leaves are present (Section 8.3). Section C gives the lane-major layout, remainder-kernel strategies, lane 0 writeback details, and instruction-level kernel structure.

[1]The implementation is available at https://github.com/codahale/treewrap.

Architecture-specific cores. On `amd64` the optimized implementation selects an AVX-512 or AVX2 core once at startup from the `AVX512VL` CPUID feature bit. On `arm64` the optimized implementation uses NEON with the Armv8.2 SHA-3 extension (`FEAT_SHA3`, available on Apple Silicon and Neoverse V1/V2 and N2). Both architecture-specific paths provide a single-duplex core and batched leaf kernels; the assembly stays below the mode boundary, so framing, domain separation, padding, and tree bookkeeping remain shared with the reference path.

Constant time. The scalar reference and all three vectorized implementations are constant time with respect to all inputs. The permutation and the XOR-based absorb/squeeze contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number; batched loads, stores, gathers, scatters, and inactive-lane handling are all controlled by public chunk counts and fixed chunk strides. Section C records the architecture-specific addressing details.

8.2 Benchmark Methodology

We measure on two hosts, one per architecture. The `amd64` host is a Google Compute Engine `c4-standard-4` instance with an Intel Xeon Platinum 8581C (Emerald Rapids, 2.30 GHz base, AVX-512 including `AVX512VL`, `AVX512_VBMI2`, `GFNI`, and `VAES`), two physical cores with simultaneous multithreading enabled, 48 KiB L1d and 32 KiB L1i per core, 2 MiB L2 per core, and a 260 MiB shared L3, running Debian 12 (kernel 6.1) under KVM. The guest does not expose a `cpufreq` interface, so the frequency governor and turbo state are not under our control; we mitigate the resulting variance statistically (below). The `arm64` host is an Apple M4 Pro (10 performance cores and 4 efficiency cores; per performance core, 128 KiB L1i, 64 KiB L1d, and a 16 MiB shared L2) running macOS 26.5.1 (Darwin 25.5), with `FEAT_SHA3` present. macOS exposes no userspace frequency control.

Measurements are collected by a standalone harness (`cmd/cpb`) that locks its goroutine to an OS thread and, for each algorithm, operation, and message length, first calibrates an iteration count that fills at least 100 ms, then records 100 samples, from which it reports the median cycles per byte and median wall-clock throughput together with the interquartile range as a dispersion measure (Table 10). Both metrics are read from the same timed loop, so the cycles-per-byte and throughput figures for a given measurement come from one interleaved run rather than two separately scheduled ones. Each sample reuses the same source and destination buffers, so the reported figures are warm-cache, steady-state rates. The single-threaded measurement leaves the SMT sibling of the `amd64` host idle. We report a sweep of seven message lengths from 1 B to 16 MiB.

The cycle counters differ by architecture, and this affects how the cycles-per-byte figures may be read. On `amd64` the harness reads `RDTSC`; on this part the time-stamp counter is invariant and ticks at the 2.30 GHz reference frequency, so the reported figures are *reference* cycles per byte and are unaffected by turbo (and convert directly to wall-clock time). On `arm64` the harness reads `CNTVCT_EL0` and scales it by the ratio of the peak core frequency to `CNTFRQ_EL0`; since Apple exposes no frequency API, the peak is auto-detected by timing a dependent integer-add chain and taking the top observed P-state, yielding *core* cycles per byte. Consequently the cycles-per-byte figures are comparable within an architecture and against the same-host AES-128-GCM baseline, but not directly across architectures; absolute throughput (Table 9) is the cross-platform metric.

The baseline is Go's AES-128-GCM implementation (`crypto/aes` with `crypto/cipher`). It uses AES-NI and `PCLMULQDQ` on `amd64`, and the AES and `PMULL` instructions on `arm64`. The AES block cipher, GCM wrapper, and TW128 AEAD object are constructed once before timing, so the measurements exclude key schedule and GHASH key-power precomputation and compare steady-state `Seal`/`Open` calls under a fixed key. Each timed

Table 8: Median cycles per byte (100 samples). The `amd64` figures are `RDTSC` reference cycles at the 2.30 GHz invariant time-stamp counter; the `arm64` figures are core cycles at the auto-detected peak performance-core frequency. Cycle counts are not directly comparable across architectures (Section 8.2).

	Message length						
	1 B	64 B	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids), AVX-512							
TW128 encrypt	571	9.04	1.81	0.73	0.39	0.38	0.38
TW128 decrypt	605	9.55	1.82	0.73	0.39	0.38	0.38
AES-128-GCM seal	119	2.74	0.30	0.29	0.29	0.29	0.29
AES-128-GCM open	99	2.17	0.29	0.28	0.28	0.28	0.29
Intel Xeon 8581C (Emerald Rapids), AVX2							
TW128 encrypt	647	10.18	2.21	1.15	1.10	1.09	1.10
TW128 decrypt	682	10.70	2.21	1.06	1.10	1.08	1.06
Apple M4 Pro, NEON + `FEAT_SHA3`							
TW128 encrypt	620	9.88	1.97	0.92	0.89	0.89	0.91
TW128 decrypt	671	10.65	2.04	0.90	0.87	0.87	0.89
AES-128-GCM seal	113	2.84	0.44	0.43	0.43	0.44	0.46
AES-128-GCM open	123	2.56	0.42	0.41	0.41	0.42	0.43

TW128 call still performs the root duplex initialization inside the call. Both directions are measured for each scheme; the cycles-per-byte table reports both, and the throughput table reports the encryption direction.

8.3 Throughput

Table 8 reports cycles per byte across the message-length sweep and Table 9 the corresponding wall-clock throughput in the bulk regime. The dominant feature of the TW128 rows is the descent produced by the tree-parallel leaf core: the per-byte cost falls as a message lengthens and its chunks occupy more SIMD lanes. A message of at most one chunk has no complete leaf chunk for chunk 0 to share a kernel with, so it runs entirely on the single-duplex core, and the 8 KiB column sits at that core's rate. Past one chunk, chunk 0 fusion (Section 8.1) keeps every complete chunk on the batched kernels, and the descent tracks lane occupancy: at 32 KiB, chunk 0 and three complete leaves run four-wide in the `amd64` remainder kernels and as two full pairs on `arm64`, and 64 KiB is the first length whose eight complete chunks ($8\beta \approx 64$ KiB) fill the eight-way ($\times 8$) core. On the `amd64` AVX-512 path TW128 encryption falls $1.81 \to 0.73 \to 0.39$ across the 8, 32, and 64 KiB columns and is already at its bulk rate at 64 KiB, holding at 0.38–0.39 through 16 MiB. The AVX2 path follows the same shape at lower throughput ($2.21 \to 1.15 \to 1.10$ over the same span, thereafter roughly 1.1 cycles per byte); it runs the eight instances as two groups of four rather than in a single 8-wide register, leaving the AVX-512 core about $2.9\times$ faster in bulk. The `arm64` curve flattens earliest ($1.97 \to 0.92 \to 0.89 \to 0.89 \to 0.91$ cycles per byte from 8 KiB to 16 MiB): the eight-way core itself processes its batch as four two-instance pairs, so the fused 2-wide kernel runs at essentially the same per-chunk rate, and a 32 KiB message—chunk 0 and three leaves as two full pairs—is already close to the long-message rate. In wall-clock terms TW128 reaches 6.00 GB/s on the `amd64` AVX-512 host and 4.93 GB/s on the M4 Pro for 16 MiB messages.

Table 9: Measured wall-clock throughput (GB/s, GB = 10^9 bytes) from the same `cmd/cpb` run as Table 8, encryption direction. The `amd64` AES-128-GCM row is the AVX-512 host and is unaffected by the TW128 core selection.

	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids)					
TW128 (AVX-512)	1.27	3.17	5.93	6.10	6.00
TW128 (AVX2)	1.04	2.00	2.10	2.10	2.08
AES-128-GCM	7.75	8.00	8.05	8.00	7.87
Apple M4 Pro					
TW128 (NEON+SHA3)	2.27	4.84	5.04	5.01	4.93
AES-128-GCM	10.04	10.27	10.33	10.07	9.71

Table 10: Cycles-per-byte measurement dispersion for TW128 across the seven-length sweep (seal and open, 100 samples per point), as the interquartile range relative to the median. The final column is the widest single-point min–max range; it is excluded from the IQR as a tail effect.

	Relative IQR		Max
Path	median	max	range
Xeon 8581C, AVX-512	0.3%	0.7%	4.9%
Xeon 8581C, AVX2	0.4%	0.9%	12.0%
Apple M4 Pro	1.2%	1.9%	16.7%

Dispersion. Each cell of Tables 8 and 9 is the median of 100 samples; Table 10 summarizes their spread as the interquartile range (IQR) relative to the median. The medians are stable: across the sweep the relative IQR of the TW128 cycles-per-byte measurements is at most 0.7% on the `amd64` AVX-512 path, 0.9% on AVX2, and 1.9% on the M4 Pro, with per-path medians of 0.3%, 0.4%, and 1.2%. The full min–max range is wider at isolated points—up to 4.9%, 12.0%, and 16.7% on the three paths—because turbo, governor, and scheduler states are not under our control (Section 8.2); these are tail excursions that the IQR excludes. We therefore report medians throughout and treat the IQR as the dispersion measure.

8.4 Latency

For short messages the cost is set by the serial root duplex and is independent of the parallel leaf machinery. A message of at most $\rho = 167$ bytes occupies a single rate block and never enters leaf mode, so it costs exactly two permutations—one to initialize the root duplex and one to close the message block and extract the tag. The 1 B and 64 B columns of Table 8 bear this out: per-message latency is nearly flat across them at roughly 571 reference cycles on `amd64` (571 at 1 B versus $9.04 \times 64 \approx 578$ at 64 B) and 620 core cycles on `arm64` (620 versus $9.88 \times 64 \approx 633$), i.e. about 285 and 310 cycles per Keccak-p[1600, 12] respectively. Because such messages bypass the leaf core entirely, leaf-level parallelism imposes no latency penalty on them.

The latency/throughput trade-off appears instead in the intermediate regime, which chunk 0 fusion has narrowed to roughly the one-to-eight-chunk band. The worst per-byte cost in the sweep is the single-chunk regime at 8 KiB (1.8–2.2 cycles per byte across the three paths), where the message is the chunk-0 phase alone on the single-duplex core. Once the message has complete leaf chunks for chunk 0 to share a kernel with, the cost falls quickly: by 32 KiB TW128 is within a factor of two of its AVX-512 floor and close to its `arm64` floor, and the floor itself is reached once the eight-way core fills at 64 KiB. The root duplex also bounds the tail of a long message: the final tag cannot be produced until chunk 0 has been processed and every leaf tag has been absorbed into the aggregation transcript. That absorption is cheap—each rate block of the root absorbs five 32-byte leaf tags, so a 16 MiB message ($\approx$ 2000 leaves) adds on the order of 400 root permutations against roughly 10^5 permutations of leaf work, under 0.5% overhead—so the serial root does not materially cap bulk throughput.

8.5 Comparison with Other AEADs

We compare against the one baseline measured under the same harness on the same hosts, the Go standard library's hardware-accelerated AES-128-GCM (Tables 8 and 9). The harness times TW128 and AES-128-GCM back-to-back at each length and reads both metrics from the same loop, so the cycles-per-byte and throughput comparisons hold under identical conditions and agree. In the bulk regime AES-128-GCM is faster on both architectures, as expected for a primitive with dedicated AES and carryless-multiply instructions: at 16 MiB it runs at 0.29 cycles per byte on `amd64` and 0.46 on `arm64`, against 0.38 and 0.91 for TW128—about 1.3× and 2.0× faster. The wall-clock throughputs show the same margins: TW128 reaches 6.00 and 4.93 GB/s against AES-128-GCM's 7.87 and 9.71 GB/s. The wider `arm64` gap reflects the strength of Apple's AES and `PMULL` units. TW128 attains these rates with a permutation that has no dedicated hardware support, relying only on the SHA-3 extension on `arm64` and on general SIMD on `amd64`.

AES-128-GCM is also faster for short messages in this steady-state measurement: at 1 B it costs 119 and 113 cycles on the two hosts against TW128's 571 and 620. This is the expected result when AES-GCM key setup is amortized outside the timed call. In the sub-bulk regime AES-128-GCM is several times faster at one chunk (0.30 versus 1.81 cycles per byte at 8 KiB on `amd64`), but the gap narrows once the fused kernels engage: about 2.5× at 32 KiB and the bulk ratio from 64 KiB on. TW128's intended value relative to AES-128-GCM lies in the tree structure and committing security properties analyzed in Sections 5 and 6 rather than in raw speed.

9 Discussion

9.1 Comparison Table

Table 11 situates TW128 against representative authenticated encryption schemes along the axes that distinguish it: the underlying primitive, whether the mode is parallelizable, the key, nonce, and tag sizes, associated data support, nonce-misuse resistance, and committing security (CMT). The rows are representative rather than exhaustive: sponge/duplex relatives, AES baselines, a misuse-resistant AES baseline, and a generic committing transform. The comparison is structural, not a total security ranking; TW128's own concrete bounds and the regimes in which each term dominates are treated in Section 7.3, and its measured throughput against hardware-accelerated AES-128-GCM in Section 8.5. The CMT column should be read within the named committing notions only, and a negative entry there does not assert a confidentiality or integrity break of the underlying AEAD. It records the published committing result or attack most relevant to the comparison: TW128's 128-bit CMTDs-4 bound, Ascon's roughly 64-bit committing bound [KSW23], published low-cost attacks where known [KSW23, CR22, ADG+22], and the hash-collision level inherited by CTX. The table does not separately track compact (tag-only) commitment, since that property has not been systematically analyzed for the comparison schemes.

Table 11: Structural comparison of TW128 with representative AEADs. Sizes are in bytes; AES-GCM-SIV also admits a 32-byte key. “Misuse” is nonce-misuse resistance in the sense of [RS06]; CMT records a published committing result or attack in the relevant committing notion, with “broken” denoting a published low-cost committing attack and “hash CR” denoting the collision-resistance level of the hash used by the transform. A dash means the entry depends on the base scheme. These entries are notion-specific summaries, not broad AE security claims. TW128’s concrete bounds appear in Section 7.3.

Scheme	Primitive	Par.	K/U/T	AD	Misuse	CMT
TW128	Keccak-p[1600, 12]	yes	32/32/32	yes	no	128 b
Ascon-128a [DEMS21]	Ascon-p (320 b)	no	16/16/16	yes	no	≈ 64 b
Xoodyak [DHP+20]	Xoodoo (384 b)	no	16/16/16	yes	no	2^{17} atk.
AES-GCM [Dwo07]	AES	yes	16/12/16	yes	no	broken
AES-GCM-SIV [GLL17]	AES	yes	16/12/16	yes	yes	broken
nAE + CTX [CR22]	any nAE	—	—	yes	—	hash CR

9.2 Comparison with Prior Sponge AEADs

TW128 belongs to the family of single-permutation duplex AEADs initiated by [BDPVA11]. The dedicated lightweight members of that family, Ascon [DEMS21] and Xoodyak [DHP+20], target 128-bit security on small permutations (320 and 384 bits) with a strictly serial duplex: each call depends on the previous one, so there is no mode-level parallelism to exploit. TW128 instead pays for a larger permutation, Keccak-p[1600, 12], and a 256-bit capacity—the same round count as KangarooTwelve [BDP+16]—in exchange for a tree that processes leaf chunks in parallel and for the headroom that yields 128-bit security with a comfortable margin. The cost of this choice is visible in Section 8: on short, single-leaf inputs TW128 is slower per byte than a compact serial duplex would be, and its advantage appears only once a message spans enough leaves to fill the vector units.

TW128 reuses KangarooTwelve’s final-node/leaf split but not its Sakura coding [BDPVA14]. In KangarooTwelve, Sakura makes the final node parseable as a message hop followed by a chaining hop: the first chunk, the sequence of leaf chaining values, the coded number of chaining values, and the final-node frame bits all have explicit roles [BDP+16]. In TW128, the analogous chaining values are the leaf tags, but they are not anonymous strings waiting to be interpreted by the root. Each leaf duplex is initialized with a chunk-specific state containing the leaf prefix and leaf index, while the root absorbs the leaf tags in index order and then absorbs the leaf count $\langle n \rangle_8$ (Sections 3 and 3.4). Sakura’s structural job is therefore handled by mode initialization and domain separation rather than by a tree coding. The transcript separation lemmas make this explicit (Lemma 5 and Corollary 8). The committing bounds do not depend on tree-coding soundness: distinct (K, U, A) triples already diverge before leaf aggregation (Proposition 1), and the leaf encoding enters the hash collision bound only through the explicit leaf tag collision term of Theorem 8.

Keyak [BDP+15], whose Motorist mode is already parallelizable, is the closest prior duplex AEAD in spirit; Section 1.2 contrasts its wire-visible, encryptor-and-decryptor-shared degree of parallelism with TW128’s run-time leaf choice. Strobe [Ham17] is a serial sponge framework aimed at whole protocols rather than at a standalone AEAD, and like Ascon and Xoodyak it does not parallelize. Committing security is not unique to TW128 in this family: a dedicated analysis of the sponge-based NIST lightweight finalists proves Ascon committing while breaking others [KSW23], and some constructions, such as Strobe, have not been examined in that setting at all. What distinguishes TW128 is that it makes the root tag a secure compact commitment to the full input—collision-, second-preimage-, and preimage-resistant as a hash, which in particular yields the 128-bit full-input CMTDs-4 bound—as a proven, designed property of the mode without leaving the single-permutation duplex setting.

9.3 Comparison with Committing AEAD Constructions

The dominant route to committing AEAD is a generic transform over a non-committing base scheme. [BH22] give the UtC, RtC, and HtE transforms, which add key- or context-commitment to an arbitrary AEAD, and [CR22] give CTX, which makes any nonce-based AE scheme commit to its full context at the cost of a single hash over a short string and with no ciphertext expansion. These transforms are inexpensive and broadly applicable; CTX in particular is nearly free. It is nevertheless a committing-AE transform, not a compact-commitment transform: it binds the transformed ciphertext, but does not make the tag a standalone digest-like commitment to the full input. TW128 differs not by undercutting their cost but by integration: it is committing as the mode itself, and its root tag is the compact committing object. The commitment uses the same permutation already used for confidentiality and integrity and the same tag already used for authentication, so no separate commitment primitive, key-derivation step, or auxiliary hash is invoked, and the multi-input full-commitment bound follows from a single root tag collision bound (Section 6). More strongly, Theorem 8 shows that the root tag alone is a compact commitment string even when the ciphertext bodies used as openings differ, up to the additional leaf tag birthday term. This places TW128 alongside the dedicated committing primitives—the compactly committing AEAD of [GLR17] and the encryptment of [DGRW18]—which likewise build commitment in rather than bolt it on, but which were designed for message franking rather than as general nonce-based AEADs. The contrast with transformed conventional schemes is sharpened by the attacks of Section 1.2: base schemes such as GCM and OCB are not committing on their own [CR22], and a transform is the only way to make them so.

Closest to TW128 here is the recent work of [DHM+24], which builds nonce-based AE directly on the SHA-3 functions SHAKE and TurboSHAKE—the latter using the same round-reduced Keccak-p permutation as TW128—and likewise achieves CMT-4 natively, from the collision resistance of the underlying function rather than from a transform. The two designs make different structural choices: [DHM+24] targets session-based communication, chaining a stream of messages through intermediate tags and generalizing the keyed duplex into a *duplex cipher*, whereas TW128 is a single two-level tree whose independent leaves expose run-time-selectable parallelism at constant memory (Sections 1.2 and 8.1). Because a session chain processes each message serially, TW128's principal advantage is throughput on large messages (Section 8): its parallel leaves fill vector units a serial duplex leaves idle, at the cost of the session support [DHM+24] provides and TW128 does not. Both demonstrate that committing AE needs no add-on transform once it is built on a well-scrutinized Keccak-p permutation.

9.4 Limitations and Open Problems

Section 4 states the formal scope of the security claims; this subsection discusses the consequences of the excluded settings and the corresponding open problems.

Nonce misuse. As scoped in Section 4, TW128's AE and mu-ind bounds require nonce-respecting encryption queries, and the mode is not misuse-resistant: a repeated nonce reuses leaf keystream and breaks confidentiality, as for any online duplex stream cipher. Schemes such as AES-GCM-SIV [GLL17], in the deterministic/misuse-resistant lineage of [RS06], tolerate nonce repetition at the price of a second pass. A misuse-resistant variant of TW128 is left open.

Unverified plaintext release. The INT-RUP theorem of Theorem 4 covers integrity when an adversary can obtain plaintext-computing oracle outputs before verification, in the release-of-unverified-plaintext setting of [ABL+14]. This is an integrity-only statement. It does not give plaintext awareness or privacy under release of unverified plaintext: TW128 is online and single-pass, and exposing unverified stream plaintext can reveal keystream information. Application behavior on unverified plaintext is likewise outside the provable model.

Conservative lift. Per-term tightness is itemized in Section 7.3; one term is conservative because our proof follows the unkeyed sponge/hash line. The flat sponge differentiating loss $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1}$ bounds the indifferentiability loss of [BDPVA08], used here to lift the random oracle analysis to the public permutation model. This proof route matches the role of the root tag: TW128's hash and committing claims are consequences of hash security, and sponge hash security is supported by a mature literature.

A direct keyed-duplex treatment would have to close two gaps. First, existing keyed-duplex bounds are hidden-key AE bounds: the game samples the key, and the keyed initialization material is not part of the adversary's output. The root tag claims here are adversarial-initialization statements. In the hash and committing games, the adversary may choose the key, nonce, associated data, message, or competing opening. Thus the proof must handle adversarially chosen initialization transcripts, not only hidden-key encryption and verification queries.

Second, the available keyed-duplex AE interface is not a direct substitute for a byte-string AEAD with arbitrary final-block length. The modern bounds synthesized by [Men23] use a duplex call phased as permutation, squeeze, then absorb or overwrite ([Men23, Sec. 3.4, Table 1]). In the authenticated-encryption application, this interface is instantiated as MonkeySpongeWrap [Men23, Alg. 8]. As written, however, the overwrite interface does not reconstruct the missing padding bits of a truncated final ciphertext block on decryption: encryption absorbs the padded final plaintext block after squeezing and then truncates the resulting ciphertext to the message length, while decryption receives only this truncated ciphertext, pads it independently, and uses the overwrite branch to set the rate. The decryption transcript can therefore diverge before tag recomputation.

TW128 instead stays with the BDPV11 duplex phasing: a call absorbs the actual input string, padded by the duplex rule, and then returns the requested output. At the stream layer this supports arbitrary length messages by splitting them into bounded duplex blocks, with the final short ciphertext block absorbed as it is rather than reconstructed through an overwrite branch (Section 3). A direct keyed-duplex theorem for TW128 would therefore need both this BDPV11 phasing and adversarial-initialization hash and committing games. We retain the indifferentiability lift because it covers these claims uniformly and already meets the 128-bit target (Section 7.3).

Beyond-birthday security and leakage. All bounds are birthday-type on the 256-bit capacity, delivering the 128-bit target but no more; a beyond-birthday analysis is open. The model is also single-stage and leakage-free: side-channel resistance is addressed only at the implementation level, through the constant-time properties of Section 8.1, and is not part of the provable-security claims.

10 Conclusion

TW128 starts from the premise that an AEAD tag should behave like the compact digest applications often take it to be, and builds the mode from a digest-shaped construction rather than adding commitment as an external layer. It adapts the KangarooTwelve tree-hashing pattern into a two-level tree of keyed duplexes over a single KECCAK-p[1600, 12] permutation. The chunk decomposition is canonical, while the number of leaf duplexes evaluated in parallel is an implementation choice rather than a mode parameter. This gives scalar, SIMD, and multicore implementations the same ciphertexts and tags, with constant working memory and scalable parallelism.

The central security claim is that the resulting root tag satisfies the digest intuition: it is a compact commitment to the full (K, U, A, M) context. In the public permutation model, via an intermediate random oracle and the flat sponge lift, we prove concrete multi-user indistinguishability and derive all-in-one AE as its single-user corollary, and prove INT-RUP integrity under release of unverified plaintext. We also prove collision, second-preimage, and preimage security for the root tag wrapper. Those hash-security bounds yield the AEAD-facing committing results, including full-input CMTDs-4 security, at a 128-bit target for the implemented parameters.

The implementation results show why the tree-hashing origin matters. Vectorized `amd64` and `arm64` implementations exploit the scalable leaf structure and exceed 16 Gbit/s in bulk, with the AVX-512 and `arm64` paths reaching 51–76% of hardware-accelerated AES-128-GCM throughput.

Several questions remain open (Section 9.4): a nonce-misuse-resistant variant, privacy under release of unverified plaintext, and sharper bounds, whether through a keyed duplex treatment of the BDPV11-style phasing used here or through a beyond-birthday argument. More broadly, TW128 adds to the evidence that a single, well-scrutinized KECCAK-p permutation can support authenticated encryption whose tag is digest-like without giving up scalable hashing-style performance.

Acknowledgments

A hearty thank-you to my wife, who remains reasonably mystified at my choice of hobbies.

References

[ABL+14] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Nicky Mouha, and Kan Yasuda. How to securely release unverified plaintext in authenticated encryption. In *Advances in Cryptology – ASIACRYPT 2014*, volume 8873 of *Lecture Notes in Computer Science*, pages 105–125. Springer, 2014.

[ADG+22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308. USENIX Association, 2022.

[BDP+15] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. CAESAR submission: Keyak v2. CAESAR Competition, Round 3 Candidate, 2015.

[BDP+16] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. KangarooTwelve: Fast hashing based on Keccak-p. Cryptology ePrint Archive, Paper 2016/770, 2016.

[BDPVA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.

[BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In *Selected Areas in Cryptography – SAC 2011*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.

[BDPVA14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sakura: A flexible coding for tree hashing. In *Applied Cryptography and Network Security – ACNS 2014*, volume 8479 of *Lecture Notes in Computer Science*, pages 217–234. Springer, 2014.

[BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13276 of *Lecture Notes in Computer Science*, pages 845–875. Springer, 2022.

[BT16] Mihir Bellare and Björn Tackmann. The multi-user security of authenticated encryption: AES-GCM in TLS 1.3. In *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *Lecture Notes in Computer Science*, pages 247–276. Springer, 2016.

[CR22] John Chan and Phillip Rogaway. On committing authenticated encryption. In *Computer Security – ESORICS 2022*, Lecture Notes in Computer Science. Springer, 2022.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.

[DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryptment. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 155–186. Springer, 2018.

[DHM+24] Joan Daemen, Seth Hoffert, Silvia Mella, Gilles Van Assche, and Ronny Van Keer. Shaking up authenticated encryption. Cryptology ePrint Archive, Paper 2024/1618, 2024.

[DHP+20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology*, 2020(S1):60–87, 2020.

[Dwo07] Morris J. Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Technical Report Special Publication 800-38D, National Institute of Standards and Technology, 2007.

[GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Specification and analysis. Cryptology ePrint Archive, Paper 2017/168, 2017.

[GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 66–97. Springer, 2017.

[Ham17] Mike Hamburg. The STROBE protocol framework. Cryptology ePrint Archive, Paper 2017/003, 2017.

[KSW23] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges: Security analysis of the NIST LWC finalists. Cryptology ePrint Archive, Paper 2023/1525, 2023.

[LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 195–212. USENIX Association, 2021.

[Men23] Bart Mennink. Understanding the duplex and its security. *IACR Transactions on Symmetric Cryptology*, 2023(2):1–46, 2023.

[Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication FIPS PUB 202, U.S. Department of Commerce, August 2015.

[RS04] Phillip Rogaway and Thomas Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In *Fast Software Encryption – FSE 2004*, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004.

[RS06] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390. Springer, 2006.

A Supporting Lemmas

The proof core separates structural decoding from probabilistic accounting. The well-formed transcript language of Definition 2 fixes which flattened queries are valid TW128 construction prefixes. Lemmas 4, 5 and 7 then show how such a query is flattened, parsed as a typed transcript, and assigned an output point; the exact keyed decoder of Definition 4 then decodes direct queries to candidate keys, when any. Lemma 10 turns those decoded keys into an adaptive key-test bound. The remaining proof interfaces expose the events their consumers must control: encryption-side freshness, verification hidden-key exposure and root tag guessing, leaf tag collisions, and the committing candidate-sequence collision event. Thus the mu-ind and committing proofs only have to derive the relevant local event premise before applying the corresponding accounting argument.

How to read this appendix. Figure 2 records the proof dependency map. It groups local interfaces when the exact proof citations would obscure the spine. The Bridge & Decode node is Lemma 7 together with the exact keyed decoder of Definition 4. The Root/Output Separation node groups Output-Point Separation (Corollary 8), Opening Triple Separation (Proposition 1), and Final-Root Input Separation (Proposition 2); the use of Leaf-Transcript Recovery (Lemma 6) is folded into the final-root separation fact. The Key-Indexed Renaming node (Lemma 8) supplies the key-field disjointness and renaming used by hidden-key encryption and verification accounting. The Adaptive Candidate node (Lemma 9) is the generic probabilistic engine: it bounds the Leaf Tag Collision lemma (Lemma 11) and, through the Root Tag Candidate Sequence and Root Tag Hash and Target-Hit Bound (Lemmas 2 and 3) of Section 6, the root tag hash and committing proofs. The mu-ind proof reuses the same structural lemmas, adding freshness, key-test, leaf-collision, and verification-first root guessing accounting.

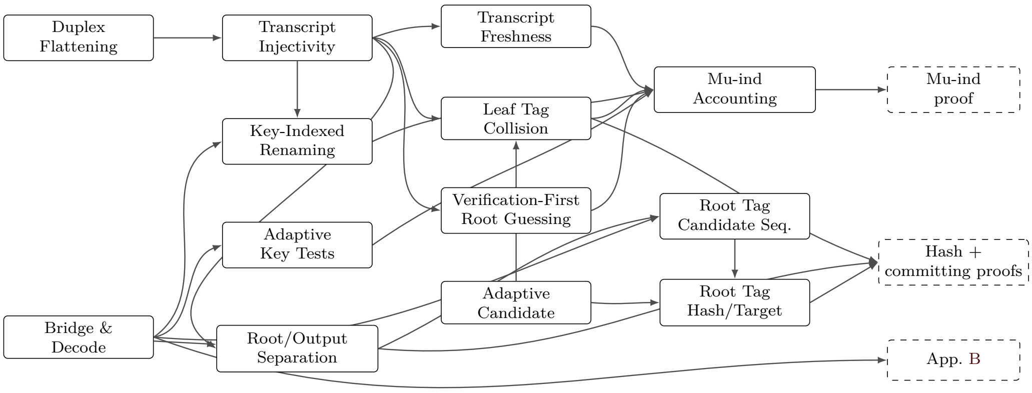

Figure 2: Proof dependency map for the supporting components in this appendix and their consumers. Solid boxes are lemmas or grouped local proof interfaces; dashed boxes are proof components that consume them.

A.1 Duplex Flattening

Lemma 4 (Duplex Flattening)**.** *Every* TW128 *root or leaf duplex output is equal to a [BDPVA11] sponge output on the corresponding flattened duplex-call transcript, and the ciphertext-absorbing message phase admits a flattened duplex transcript that is well defined for both encryption and decryption.*

Proof. Let D be a duplex object using permutation π, rate r, and padding $\mathsf{pad10^*1}$. For a sequence of duplex calls $Z_i = \mathsf{D.duplexing}(\sigma_i, \ell_i)$ with each σ_i short enough to fit in one duplex block after padding, [BDPVA11, Lemma 3] gives

$$Z_i = \mathsf{Sponge}[\pi, \mathsf{pad10^*1}, r]\big(\sigma_0 \parallel \mathsf{pad}_0 \parallel \sigma_1 \parallel \mathsf{pad}_1 \parallel \cdots \parallel \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad10^*1}[r](|\sigma_j|)$, and Lemma 4 says that, for fixed $\mathsf{pad10^*1}$ and r, the map from the duplex input-string sequence to the flattened sponge input is injective.

In TW128, each duplex call absorbs a byte-aligned block σ followed by a 4-bit phase suffix $\sigma_{\mathsf{ph}} \in \{\sigma_{\mathsf{init}}, \sigma^-_{\mathsf{ad}}, \ldots, \sigma^+_{\mathsf{agg}}\}$ (Section 3.4), so the duplex-call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ followed by $\mathsf{pad10^*1}$. With $r = 1344$ and $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits, the maximum byte-aligned input is $\rho = 167$ bytes (Section 3.2), giving a worst-case duplex-call input of $8{\cdot}167{+}4 = 1340 \leq 1342$ bits. The two initialization calls are shorter still: $|p_{\mathsf{root}} \| K \| U| + 4 = 8 \cdot (6 + 32 + 32) + 4 = 564$ bits and $|p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8| + 4 = 8 \cdot (6 + 32 + 32 + 8) + 4 = 628$ bits. [BDPVA11, Lemmas 3 and 4] therefore apply independently to every TW128 duplex object.

The root duplex is initialized by absorbing $p_{\mathsf{root}} \parallel K \parallel U$ under σ_{init}(Algorithm 2), then absorbs the associated data blocks under $\sigma^-_{\mathsf{ad}}/\ \sigma^+_{\mathsf{ad}}$, the root ciphertext blocks in the message phase under $\sigma^-_{\mathsf{msg}}/\ \sigma^+_{\mathsf{msg}}$, and the leaf tag aggregation blocks under $\sigma^-_{\mathsf{agg}}/\ \sigma^+_{\mathsf{agg}}$. By the duplex-to-sponge equality, every root duplex output is the sponge function evaluated on the root transcript prefix accumulated up to the relevant output point. The same holds for each leaf duplex, initialized by $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$ and absorbing only its own ciphertext blocks in the message phase. In decryption and verification, Algorithm 3 absorbs the submitted ciphertext blocks before the root tag comparison, so the same flattened message phase transcript is defined whether the comparison accepts or rejects. □

A.2 Well-formed Transcript Prefixes

For a duplex-call prefix $((\sigma_0, \sigma_{\mathsf{ph},0}), \ldots, (\sigma_i, \sigma_{\mathsf{ph},i}))$, write

$$\mathsf{Flat}\big((\sigma_0, \sigma_{\mathsf{ph},0}), \ldots, (\sigma_i, \sigma_{\mathsf{ph},i})\big) = \big\Vert_{j=0}^{i-1} \big(\sigma_j \parallel \sigma_{\mathsf{ph},j} \parallel \mathsf{pad}_j\big) \parallel \sigma_i \parallel \sigma_{\mathsf{ph},i}.$$

The current call's padding pad_i is not part of Flat; it is applied internally by the sponge function. Equivalently, after the i-th call the duplex state is the sponge state after absorbing $\mathsf{Flat}(\cdot) \parallel \mathsf{pad}_i$.

Definition 2 (Well-Formed TW128 Transcript Prefixes)**.** A typed TW128 transcript prefix is *well formed* if it is a prefix, ending at a construction transcript lookup point, of one of the following two call languages.

- A root transcript starts with $(p_{\mathsf{root}} \parallel K \parallel U, \sigma_{\mathsf{init}})$, with fixed-width fields $K \in \{0,1\}^k$ and $U \in \{0,1\}^\nu$. It then absorbs the associated data byte string (only when the associated data is nonempty; the phase is otherwise elided, Section 3.5), the root ciphertext byte string, and the aggregation byte string (only when $n \geq 1$; when $n = 0$ the aggregation phase is elided, Section 3.5, and the root tag is emitted by the final σ^+_{msg} message call), in that order. Each present phase byte string is partitioned by the block rule of Algorithm 1: the empty string gives one empty final block, and a nonempty string gives zero or more ρ-byte non-final blocks followed by one final block. The phases use respectively $\sigma^-_{\mathsf{ad}}/\sigma^+_{\mathsf{ad}}$, $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, and $\sigma^-_{\mathsf{agg}}/\sigma^+_{\mathsf{agg}}$.
- A leaf-j transcript starts with $(p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8, \sigma_{\mathsf{init}})$, where $1 \leq j < 2^{64}$ and the fields have fixed width. It then absorbs the leaf ciphertext byte string under $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, using the same block rule.

The message byte string in this language is always the ciphertext byte string absorbed by the construction: encryption obtains it by XORing the plaintext with the current keystream, while decryption and verification use the submitted ciphertext. Thus a verification transcript is well formed for every in-domain ciphertext before the tag comparison is made; acceptance is not part of the transcript language.

The root aggregation byte string has the form $T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$, where each T_j has fixed byte length $\tau_{\mathsf{leaf}}/8$. The trailing fixed-width field therefore gives a unique parse of the leaf tag sequence. The output-point class of a well-formed counted prefix is the class in Table 12 determined by its duplex instance and final-call suffix.

Name the tag-emitting transcript families:

$$\begin{aligned} \mathcal{R}_{\mathsf{tag}} &= \text{root transcript prefix whose sponge output gives the final root tag,} \\ \mathcal{L}_{\mathsf{tag}}(j) &= \text{leaf-}j\text{ transcript prefix whose sponge output gives the leaf tag.} \end{aligned}$$

The final root tag is emitted by the σ^+_{agg} aggregation call (the $\mathcal{R}_{\mathsf{tag}}$ family) when $n \geq 1$, and by the closing σ^+_{msg} message call (an $\mathcal{R}^+_{\mathsf{msg}}$ prefix, in the notation of Corollary 8) when $n = 0$ and the aggregation phase is elided. Since $n = \text{leaves}(C)$ is fixed by $\|C\|$, every computation on a given ciphertext emits its tag at the output point selected by $\|C\|$. Intermediate construction transcript lookup points (init, non-final AD, non-final message, and non-final aggregation) are also typed prefixes covered by Lemma 7; the exact keyed direct-query decoder names them in Table 12.

Definition 3 (Root Tag Output Point and Final Root Transcript Class)**.** For any in-domain encryption or verification computation on ciphertext C, the *root tag output point* is the construction lookup that emits the root tag: $\mathcal{R}_{\mathsf{tag}}$ when $\text{leaves}(C) \geq 1$, and the closing root message point $\mathcal{R}^{+}_{\mathsf{msg}}$ when leaves$(C) = 0$ and aggregation is elided. The *final root transcript class* of such a computation is the set of construction computations that realize the same bridged final root input at this root tag output point. A class is *encryption-first* if its first construction generation is by an encryption query, and *verification-first* if its first construction generation is by a non-replay verification query.

A.3 Transcript Injectivity

Lemma 5 (Transcript Injectivity)**.** *Well-formed* $\mathsf{TW128}$ *flattened duplex-call transcript prefixes are injectively encoded and separated by duplex instance (root vs leaf), leaf index* j*, phase suffix, and duplex-call order. For final-root prefixes, equality of flattened inputs recovers the same root initialization, AD blocks, root ciphertext blocks, aggregation input, and leaf tag sequence.*

Proof. By Lemma 4, every well-formed output point can be written as a [BDPVA11] flattened sponge input for the corresponding duplex-call prefix, namely Flat as defined above. The deterministic current-call $\mathsf{pad10}^*\mathsf{1}$ is applied internally by the sponge function and is not part of the input to which [BDPVA11, Lemma 4] is applied. That lemma recovers the duplex-call sequence from the flattened sponge input; in $\mathsf{TW128}$, each call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ with byte-aligned σ and fixed 4-bit suffix σ_{ph}, so recovering the call input recovers the pair $(\sigma, \sigma_{\mathsf{ph}})$.

Definition 2 fixes the root and leaf initialization fields, the admissible phase order, and the final-call suffixes. The distinct prefixes p_{root} and p_{leaf} separate root from leaf transcripts; the fixed-width leaf field $\langle j \rangle_8$ separates leaves; and the seven suffix values of Table 3 separate phases and non-final/final calls. Thus equality of flattened inputs forces equality of the recovered instance type, key, nonce, leaf index when present, phase suffixes, and duplex-call order.

Phase-recovery sub-claim. For each present phase $P \in \{\mathsf{ad}, \mathsf{msg}, \mathsf{agg}\}$ (the ad phase is present exactly when the associated data is nonempty; see Section 3.5), $\mathsf{TW128}$ absorbs the phase's input byte string s_P by partitioning into ρ-byte blocks (with the empty case yielding a single empty block, per Section 3.5) and feeding each block to the duplex with suffix σ^{-}_{P} (non-final) or σ^{+}_{P} (final). Lemma 4 and [BDPVA11, Lemma 4] recover the corresponding $(\sigma, \sigma_{\mathsf{ph}})$ sequence from the flattened input. Because the partition map is deterministic and uniquely invertible on its image (the empty case yielding the single empty block carrying σ^{+}_{P}), concatenating the recovered σ blocks reconstructs s_P exactly. The map from s_P to the phase's $(\sigma, \sigma_{\mathsf{ph}})$ sequence is therefore injective.

Applied phase by phase to a well-formed final-root transcript:

- AD. The associated data phase contributes a σ^{+}_{ad}-tagged block exactly when the associated data is nonempty; an empty associated data elides the phase and contributes no block. When the phase is present the sub-claim recovers the AD byte string. Thus distinct associated data strings $A_i \neq A_j$ are separated either by the presence versus absence of the AD phase (when exactly one is empty) or, when both are nonempty, by their distinct AD σ-block sequences.

- MSG. Distinct absorbed root or leaf ciphertext byte strings produce distinct MSG σ-block sequences; the message language of Definition 2 is ciphertext-absorbing for encryption and decryption alike.

- AGG. The aggregation phase is present exactly when $n \geq 1$ (Section 3.5). When it is present, the sub-claim applied to the aggregation byte string $T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$ recovers that byte string. The final eight bytes give $\langle n \rangle_8$, and each leaf tag has fixed byte length $\tau_{\mathsf{leaf}}/8$, so the byte string has a unique parse as a leaf tag sequence. Two root aggregation encodings are therefore equal exactly when their leaf tag sequences are equal. When $n = 0$ the aggregation phase is absent and the final-root transcript ends at the recovered $\sigma^{+}_{\mathsf{msg}}$ root message call; the MSG bullet recovers the root ciphertext byte string, and the absence of any $\sigma^{-}_{\mathsf{agg}}/\sigma^{+}_{\mathsf{agg}}$ call distinguishes such a prefix from every $n \geq 1$ final-root prefix.

Therefore the recovered final-root transcript determines the root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the aggregation byte string and leaf tag sequence. □

Lemma 6 (Leaf-Transcript Recovery)**.** *Let two construction-generated final-root prefixes have equal flattened inputs, and suppose that, for each position j of their common leaf tag sequence, the two computations' position-j final leaf transcripts are either equal or receive distinct leaf tags. Then equality of flattened inputs recovers the same leaf transcript sequence, and therefore the same full absorbed ciphertext. Both computations' position-j final leaf transcripts are leaf-j transcripts under the common recovered (K, U), so the hypothesis holds whenever no two distinct construction-generated final leaf transcripts with the same key, nonce, and leaf index receive the same leaf tag; when one of the two prefixes is generated by an encryption computation, it suffices that no construction-generated final leaf transcript receives the same leaf tag as a distinct encryption-generated final leaf transcript with the same key, nonce, and leaf index.*

Proof. By Transcript Injectivity (Lemma 5), the two equal flattened final-root inputs recover the same root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the same aggregation byte string and leaf tag sequence; the trailing $\langle n \rangle_8$ field fixes the common leaf count n. When $n = 0$ no leaf is present and the recovered root ciphertext blocks already constitute the full absorbed ciphertext. When $n \geq 1$, the aggregation byte string parses uniquely as the leaf tag sequence $T_1 \parallel \cdots \parallel T_n$; the two leaf tag sequences are equal, so for each j the two computations' position-j final leaf transcripts—leaf-j transcripts carrying the common recovered (K, U) (Definition 2, Section 3.5)—receive the same leaf tag, and by hypothesis are equal, hence so are their absorbed leaf ciphertext bodies. Together with the common root ciphertext blocks this recovers the same full absorbed ciphertext. □

A.4 Bridge and Typed Decoding

Lemma 7 (Bridge Injectivity)**.** *The random oracle games used in Sections 5 and 6 and Corollary 6 are ordinary random oracle games on a single random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ over the unpadded [BDPVA08] H-input domain (Section 4.3), with* TW128 *root and leaf duplex outputs answered by the typed restriction*

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)), \qquad \mathsf{flat}(X) = I[r, r_{\max}]\big(\mathsf{raw\_flat}(X)\big),$$

for $r = r_{\max} = 1344$. Here $\mathsf{raw\_flat}(X)$ is the native flattened input $\mathsf{Flat}(\cdot)$ of Section A.2 applied to the duplex-call sequence of X, and $I[r, r_{\max}]$ is the [BDPVA11] Theorem 3 preprocessing map. The induced map $X \mapsto \mathsf{flat}(X)$ is injective on well-formed typed TW128 *duplex transcript prefixes.*

Proof. The bridge $I[r, r_{\max}]$ is described by the proof of [BDPVA11, Theorem 3] in three steps: (1) pad the input with $\mathsf{pad10^*1}[r]$; (2) for each r-bit block, append $r_{\max} - r$ zero bits; (3) strip the trailing $\mathsf{pad10^*}$ from the result. The $\mathsf{pad10^*1}$ padding in step (1) appends a 1, then $q = (-|\mathsf{raw\_flat}(X)| - 2) \bmod r$ zero bits, then a closing 1; the $\mathsf{pad10^*}$-unpadding in step (3) strips the $r_{\max} - r$ trailing zero bits introduced by step (2) together with the closing 1 of step (1). For TW128 with $r = r_{\max}$, step (2) appends an empty string and step (3) strips zero trailing zeros and the closing 1, so the net effect of steps (1) and (3) is to map a native flattened input $W = \mathsf{raw\_flat}(X)$ to $W \parallel 1 \parallel 0^q$. Any string in this image has a unique inverse: q is the number of trailing zero bits, the preceding bit is the inserted 1, and deleting this suffix recovers W.

Injectivity of flat on well-formed typed $\mathsf{TW128}$ prefixes then follows from Lemma 5: distinct well-formed typed prefixes have distinct $\mathsf{raw\_flat}(\cdot)$ images, and the bridge is injective on that image. Direct adversary queries on inputs outside the bridged $\mathsf{TW128}$ well-formed transcript language still count in $q_{\mathsf{RO}}(\mathcal{B})$ (Section 4.4) but cannot trigger $\mathsf{bad}_{\mathsf{key}}$. □

The bridge injectivity established here underlies Opening Triple Separation (Proposition 1, Section 6): distinct (K, U, A) triples yield distinct bridged final root transcripts.

Corollary 8 (Output-Point Separation)**.** *Every well-formed $\mathsf{TW128}$ typed transcript prefix at a counted construction transcript lookup point belongs to exactly one of the following output-point classes, identified by the recovered duplex instance and the 4-bit suffix of its final call:*

- $\mathcal{R}_{\mathsf{init}}$ *(root duplex, suffix σ_{init}),*
- $\mathcal{L}_{\mathsf{init}}(j)$ *(leaf-j duplex, suffix σ_{init}),*
- $\mathcal{R}^{-}_{\mathsf{ad}}$ / $\mathcal{R}^{+}_{\mathsf{ad}}$ *(root duplex, suffix σ^{-}_{ad} or σ^{+}_{ad}),*
- $\mathcal{R}^{-}_{\mathsf{msg}}$ / $\mathcal{R}^{+}_{\mathsf{msg}}$ *(root duplex, suffix $\sigma^{-}_{\mathsf{msg}}$ or $\sigma^{+}_{\mathsf{msg}}$),*
- $\mathcal{L}^{-}_{\mathsf{msg}}(j)$ *(leaf-j duplex, suffix $\sigma^{-}_{\mathsf{msg}}$),*
- $\mathcal{L}_{\mathsf{tag}}(j)$ *(leaf-j duplex, suffix $\sigma^{+}_{\mathsf{msg}}$),*
- $\mathcal{R}_{\mathsf{agg}}$ *(suffix $\sigma^{-}_{\mathsf{agg}}$),*
- $\mathcal{R}_{\mathsf{tag}}$ *(root duplex, suffix $\sigma^{+}_{\mathsf{agg}}$).*

For any two such prefixes whose bridged flattened inputs are equal, the separation of Lemma 5 recovers the same duplex instance, leaf index j when present, (K, U) pair, output-point class, and duplex-call sequence. Equivalently, any two distinct counted transcript prefixes already differ in one of these recovered fields, and hence in their flattened inputs.

Proof. Lemma 7 recovers equality of native flattened inputs from equality of bridged inputs, after which Lemma 5 recovers the duplex-call sequence and its per-call $(\sigma, \sigma_{\mathsf{ph}})$ pairs. The first call's σ is $p_{\mathsf{root}} \parallel K \parallel U$ for root instances and $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$ for leaf instances, with fixed-width fields, so it fixes the instance type (by p_{root} vs p_{leaf}), the (K, U) pair, and the leaf index j; within each instance type the phase suffixes are pairwise distinct, so the recovered instance type together with the final-call suffix fixes the output-point class, and each prefix therefore lies in exactly one class. Conversely, two distinct well-formed prefixes in the same class on the same instance differ in some recovered call's σ or in the sequence length, hence in their flattened inputs. □

Definition 4 (Exact Keyed Decoder)**.** For a direct query to $\mathsf{RO}_{\mathsf{flat}}$ with input Y and finite length ℓ, define $\mathsf{KeyedTWOut}(Y)$ as a partial map from $\{0,1\}^*$ to $\{0,1\}^k \times \mathcal{C}_{\mathsf{out}}$. The map ignores ℓ. It returns $(K, \mathcal{C})$ iff there exists a typed $\mathsf{TW128}$ duplex-call prefix X such that $Y = \mathsf{flat}(X)$, the raw flattened prefix of X parses as a well-formed $\mathsf{TW128}$ transcript prefix in the class $\mathcal{C}$ listed in Table 12, and the first call of X contains the candidate key K in the fixed-width root field $p_{\mathsf{root}} \parallel K \parallel U$ or leaf field $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$. If no such prefix exists, $\mathsf{KeyedTWOut}(Y) = \perp$. In particular, inputs with the right final suffix but a malformed phase order, noncanonical block partition, malformed aggregation string, or out-of-range leaf index are outside the well-formed language and decode to $\perp$. A non-$\perp$ decoding always fixes the candidate key K and nonce U in the fixed-width initialization field. It fixes the associated data A only for root prefixes whose recovered phase order determines the complete AD string: a final AD prefix, or any later root message or aggregation prefix, with absence of AD calls determining $A = \varepsilon$. We use the key projection $(K, \mathcal{C})$ throughout. When a game hides a single context component $Z \in \{K, U, A\}$, write $\mathsf{KeyedTWOut}_Z(Y)$ for the decoded value of that component when it is determined, and $\perp$ otherwise.

By Lemma 7 and Corollary 8, $\mathsf{KeyedTWOut}$ is well defined: a bridged input cannot parse as two distinct counted well-formed prefixes, and an accepted input fixes a unique candidate key and output-point class. Inputs outside this well-formed transcript language decode to $\perp$.

Table 12: Counted keyed transcript classes for $\mathsf{KeyedTWOut}$.

Class $\mathcal{C}$	Instance	Final call / role
$\mathcal{R}_{\mathsf{init}}$	root	σ_{init}, root initialization
$\mathcal{R}^{-}_{\mathsf{ad}}$	root	σ^{-}_{ad}, non-final AD block
$\mathcal{R}^{+}_{\mathsf{ad}}$	root	σ^{+}_{ad}, final AD block
$\mathcal{R}^{-}_{\mathsf{msg}}$	root	$\sigma^{-}_{\mathsf{msg}}$, non-final root message block
$\mathcal{R}^{+}_{\mathsf{msg}}$	root	$\sigma^{+}_{\mathsf{msg}}$, final root message block
$\mathcal{R}_{\mathsf{agg}}$	root	$\sigma^{-}_{\mathsf{agg}}$, non-final aggregation block
$\mathcal{R}_{\mathsf{tag}}$	root	$\sigma^{+}_{\mathsf{agg}}$, final aggregation / root tag
$\mathcal{L}_{\mathsf{init}}(j)$	leaf j	σ_{init}, leaf initialization
$\mathcal{L}^{-}_{\mathsf{msg}}(j)$	leaf j	$\sigma^{-}_{\mathsf{msg}}$, non-final leaf message block
$\mathcal{L}_{\mathsf{tag}}(j)$	leaf j	$\sigma^{+}_{\mathsf{msg}}$, final leaf message block / leaf tag

The table is a transcript-lookup table, not an observation-length table. A class is counted even when the construction's requested output length at that call is zero, including root initialization, non-final AD and aggregation calls, a final AD call whose following root chunk requests no keystream bytes, and the final root message call. A direct query triggers the same decoder regardless of ℓ: short finite-output queries and zero-length queries $(Y, 0)$ still count as direct queries to the address Y. Leaves have no AD or aggregation classes; a leaf transcript enters the table only through leaf initialization, non-final leaf message blocks, or the final leaf tag point. When the associated data is empty the root transcript likewise has no $\mathcal{R}^{-}_{\mathsf{ad}}$ or $\mathcal{R}^{+}_{\mathsf{ad}}$ class (Section 3.5); the $\mathcal{R}_{\mathsf{init}}$class then supplies the first root keystream block, exactly as $\mathcal{L}_{\mathsf{init}}(j)$ supplies the first leaf keystream block, and the decoder still ignores ℓ. Symmetrically, when $n = 0$ the root transcript has no $\mathcal{R}_{\mathsf{agg}}$or $\mathcal{R}_{\mathsf{tag}}$ class (Section 3.5); the final root message call $\mathcal{R}^{+}_{\mathsf{msg}}$ then supplies the root tag, exactly as $\mathcal{L}_{\mathsf{tag}}(j)$ supplies a leaf tag, and the decoder again ignores ℓ. An $n \geq 1$ computation also reaches an $\mathcal{R}^{+}_{\mathsf{msg}}$ point when closing chunk 0, but there the construction requests zero output and continues into the aggregation phase, so the $\mathcal{R}^{+}_{\mathsf{msg}}$ value is exposed as a tag only on the $n = 0$ path; since $n = \text{leaves}(C)$ is fixed by $\|C\|$, the two uses never coincide on a single ciphertext.

The hidden-key encryption, verification, and mu-ind accounting all reindex transcript addresses by the key field. We record that reindexing once.

Lemma 8 (Key-Indexed Transcript Renaming)**.** *For a key $K \in \{0,1\}^k$, let $\mathcal{Y}_K$ be the set of bridged inputs $\mathsf{flat}(X)$ of well-formed TW128 transcript prefixes X whose first-call key field is K—the counted hidden-key transcript addresses of Table 12 for key K. Then:*

1. Disjointness. *For distinct keys $K_0 \neq K_1$, the sets $\mathcal{Y}_{K_0}$ and $\mathcal{Y}_{K_1}$ are disjoint.*

2. Renaming. *The map $\Phi_{K_0 \to K_1}$ that replaces the first-call key field K_0 by K_1 and leaves every other recovered field unchanged—nonce, leaf index, phase sequence, AD blocks, ciphertext blocks, aggregated leaf tag bytes, and root/leaf instance—induces*

through flat *a bijection* $\mathsf{flat} \circ \Phi_{K_0 \to K_1} \circ \mathsf{flat}^{-1} : \mathcal{Y}_{K_0} \to \mathcal{Y}_{K_1}$ *preserving output-point classes and equality relations among addresses.*

3. Direct-query avoidance. *A direct query input $Y \in \mathcal{Y}_K$ has* $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ *for a counted class $\mathcal{C}$; hence if no prior direct query decodes to K then $\mathcal{Y}_K$ is disjoint from all prior direct-query inputs.*

Proof. By bridge injectivity (Lemma 7) and Transcript Injectivity (Lemma 5), a bridged input $\mathsf{flat}(X)$ determines the native flattened prefix and hence the first duplex call $p_{\mathsf{root}} \| K \| U$ or $p_{\mathsf{leaf}} \| K \| U \| \langle j \rangle_8$, in particular its fixed-width key field K. *(1)* If $\mathsf{flat}(X) = \mathsf{flat}(X')$ with key fields K_0 and K_1 then $K_0 = K_1$, so distinct keys give disjoint address sets. *(2)* $\Phi_{K_0 \to K_1}$ rewrites only the key field, leaving the recovered duplex-call sequence otherwise intact, so it is a bijection on well-formed prefixes preserving the output-point class (Corollary 8) and the equality pattern among prefixes; flat transports it to the stated bijection on addresses. *(3)* By Lemma 7 a $Y = \mathsf{flat}(X) \in \mathcal{Y}_K$ decodes under $\mathsf{KeyedTWOut}$ to $(K, \mathcal{C})$; the contrapositive gives the avoidance claim. ☐

A.5 Adaptive Candidate Bounds

Lemma 9 (Adaptive Candidate Collision and Preimage)**.** *Consider any interaction with a lazy random oracle* $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ *that builds a sequence of distinct* candidate *inputs $Y_1, \ldots, Y_m$. The process may make arbitrary non-candidate oracle queries and may choose each candidate adaptively from the full prior transcript and private state sampled before the candidate's unread oracle value. Whenever a new candidate Y_j is appended, two premises hold:* predictability*—both the decision to append and the string Y_j are determined by the state preceding the append step—and* unrevealed value*—no oracle lookup before the append step has requested a positive number of bits of* $\mathsf{RO}_{\mathsf{flat}}(Y_j)$*; zero-length lookups at Y_j are permitted. Fix a projection length τ, and suppose the sequence has at most L candidates ($m \le L$).*

1. Collision. *For every $s \ge 2$, the probability* $\Pr[\mathsf{coll}_{s,\tau}]$ *that some s distinct candidates have equal τ-bit projections,*

$$\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_1}) \right\rfloor_\tau = \cdots = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_s}) \right\rfloor_\tau \quad \textit{for some } 1 \le i_1 < \cdots < i_s \le m,$$

satisfies $\Pr[\mathsf{coll}_{s,\tau}] \le \binom{L}{s} \cdot 2^{-\tau(s-1)}$.

2. Preimage. *For every target $t \in \{0,1\}^\tau$ chosen independently of* $\mathsf{RO}_{\mathsf{flat}}$*, the probability* $\Pr[\mathsf{pre}_{\tau,t}]$ *that some candidate's τ-bit projection equals t, i.e.* $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t$ *for some $1 \le j \le m$, satisfies* $\Pr[\mathsf{pre}_{\tau,t}] \le L \cdot 2^{-\tau}$.

The preimage case has a second-preimage *corollary: for a designated candidate position j_0, the probability* $\Pr[\mathsf{sec}_{\tau,j_0}]$ *that some candidate $j \ne j_0$ has* $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{j_0}) \right\rfloor_\tau$*, i.e. collides with the designated candidate on its τ-bit projection, satisfies* $\Pr[\mathsf{sec}_{\tau,j_0}] \le (L-1) \cdot 2^{-\tau}$.

Proof. Both parts expose the interaction in chronological order and sample each candidate's $\mathsf{RO}_{\mathsf{flat}}$ value lazily at its append step, which the premises license: when Y_j is appended its string is fixed by the prior state (predictability) and no bit of $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ has yet been revealed (unrevealed value), so $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ may be drawn then, uniform and independent of everything so far.

Collision. For a fixed position set $I = \{i_1 < \cdots < i_s\} \subseteq \{1, \ldots, L\}$, let E_I be the event that these candidate positions exist and their τ-bit projections are all equal. Condition on any positive-probability prefix immediately before position i_r is appended, for $r \ge 2$, and on the values sampled at the earlier positions in I. By the above, $\mathsf{RO}_{\mathsf{flat}}(Y_{i_r})$ is then uniform and independent of the conditioning, so its τ-bit projection equals the already fixed projection of Y_{i_1} with probability $2^{-\tau}$. Multiplying over $r = 2, \ldots, s$ gives $\Pr[E_I] \le 2^{-\tau(s-1)}$; if some position in I is never appended then E_I is false and the bound still holds. A union bound over the $\binom{L}{s}$ position sets gives the claim.

Preimage. For a fixed position $j \in \{1, \ldots, L\}$, let E_j be the event that position j exists and $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t$. Condition on any positive-probability prefix immediately before Y_j is appended; the target t is a constant. By the above, $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ is uniform and independent of the conditioning, so its τ-bit projection equals t with probability $2^{-\tau}$. If position j is never appended, E_j is false. A union bound over the L positions gives the claim.

Second-preimage corollary. If the designated position j_0 is not appended, the event is false. Otherwise split the other candidates according to whether they occur before or after j_0. For earlier candidates, condition on any positive-probability prefix immediately before Y_{j_0} is appended, including the already sampled projections of candidates $1, \ldots, j_0 - 1$. The value $\mathsf{RO}_{\mathsf{flat}}(Y_{j_0})$ is then fresh and uniform, so its τ-bit projection hits one of those at most $j_0 - 1$ earlier projections with probability at most $(j_0 - 1)2^{-\tau}$. For a later candidate Y_j, $j > j_0$, condition on the full prefix immediately before it is appended, including the fixed projection $t_0 = \lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{j_0}) \rfloor_\tau$. The value $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ is fresh and uniform at its append step, so it hits t_0 with probability $2^{-\tau}$. A union bound over the at most $L - j_0$ later candidates gives an additional $(L - j_0)2^{-\tau}$. Adding the two sides gives $\Pr[\mathsf{sec}_{\tau, j_0}] \le (L-1) \cdot 2^{-\tau}$; the designated candidate is excluded, so its trivially equal projection does not enter the count. $\square$

A.6 Adaptive Key Tests

Lemma 10 (Adaptive Key-Test Bound)**.** *Consider a TW128 random oracle model game with a single hidden context component Z sampled uniformly from $\{0,1\}^w$ for some $w \ge k$—the standard case being the hidden key, $Z = K$ and $w = k$, used in the AE-style proofs—direct adversary access to $\mathsf{RO}_{\mathsf{flat}}$, and an immediate abort event $\mathsf{bad}_{\mathsf{key}}$ that fires on a direct query (Y, ℓ) exactly when Y decodes to some counted output-point class $\mathcal{C}$ of Table 12 with $\mathsf{KeyedTWOut}_Z(Y) = Z$. Suppose that, before the abort, the following invariant holds for every direct query position i: after conditioning on the full visible execution prefix and on the set S_i of distinct decoded candidate Z-values from earlier direct queries, the hidden Z is uniform on $\{0,1\}^w \setminus S_i$. Then*

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \le q_{\mathsf{RO}}/2^w \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}) = q_{\mathsf{RO}}/2^k,$$

where q_{RO} is the execution-uniform count of direct $\mathsf{RO}_{\mathsf{flat}}$ queries.

Proof. If $q_{\mathsf{RO}} \ge 2^w$ the right-hand side $q_{\mathsf{RO}}/2^w$ is at least one, so the claim is trivial. Assume $q_{\mathsf{RO}} < 2^w$.

Order the direct $\mathsf{RO}_{\mathsf{flat}}$ queries. By the definition of $\mathsf{KeyedTWOut}_Z$ and Lemma 7, each query input either decodes to $\perp$ or decodes to a unique candidate Z-value and counted output-point class. Queries that decode to $\perp$, and repeated queries whose decoded Z-value already lies in the current set S_i, do not change the set of possible first successful tests. Removing them can only shorten the sequence, so it suffices to analyze the subsequence of distinct fresh decoded candidate Z-values $Z'_1, \ldots, Z'_m$, where $m \le q_{\mathsf{RO}}$.

Let F_i be the event that the first i fresh decoded candidate values all miss the hidden Z. The invariant says that, conditioned on the visible prefix before the $(i+1)$-st fresh test and on F_i, the hidden Z is uniform over the $2^w - i$ values not yet tested. The next fresh candidate value Z'_{i+1} is one of those remaining values and is determined by the adversary's next direct query, so

$$\Pr[Z = Z'_{i+1} \mid F_i] = \frac{1}{2^w - i}, \qquad \Pr[F_{i+1} \mid F_i] = 1 - \frac{1}{2^w - i}.$$

Multiplying the miss probabilities gives

$$\Pr[F_m] = \prod_{i=0}^{m-1}\left(1 - \frac{1}{2^w - i}\right) = \prod_{i=0}^{m-1}\frac{2^w - i - 1}{2^w - i} = \frac{2^w - m}{2^w}.$$

Thus the probability that some fresh decoded candidate value equals the hidden Z is $m/2^w \le q_{\mathsf{RO}}/2^w \le q_{\mathsf{RO}}/2^k$, the last step by $w \ge k$. This event is exactly the first $\mathsf{bad}_{\mathsf{key}}$ abort, because $\mathsf{KeyedTWOut}_Z$ returns either $\perp$ or a unique candidate Z-value. □

A.7 Leaf Tag Collision Bound

Lemma 11 (Leaf Tag Collision Bound)**.** *Consider any TW128 random oracle model game with an adversary $\mathcal{B}$ that has direct access to $\mathsf{RO}_{\mathsf{flat}}$ and construction computations that may generate final leaf transcripts—by encryption, plaintext-computing, or verification computations, whether they accept or reject. Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcript prefixes $X \ne X'$ in leaf tag output-point classes receive the same τ_{leaf}-bit projection from $\mathsf{RF}(\cdot)$, and let $n_{\mathsf{leaf}}(\mathcal{B})$ be the execution-uniform count of distinct construction-generated final leaf transcripts (Section 4.4). The first bound applies the collision case ($s = 2$, $\tau = \tau_{\mathsf{leaf}}$) of Lemma 9 to a chronological leaf tag candidate sequence; the second is a same-slot conditioning argument in the all-reject continuation.*

1. Adversary-supplied keys. *When the game's keys are adversary-supplied—so direct queries may themselves land on leaf tag output points—*

$$\Pr[\mathsf{bad}_{\mathsf{leaf}}] \;\le\; \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}\big(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})\big) = \binom{q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})}{2} \Big/ 2^{\tau_{\mathsf{leaf}}}.$$

2. Hidden key, same slot. *Suppose the game has a hidden key, nonce-respecting encryption queries, and a visible interface that exposes no τ_{leaf}-bit leaf tag projection of a construction-generated final leaf transcript: work in the all-reject continuation of the stopped interaction, in which every non-replay verification submission performs its construction computation but returns reject (as in Lemma 14). Say two final leaf transcripts* share a slot *if they recover the same key, nonce, and leaf index (K, U, j) (Corollary 8). Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that some construction-generated final leaf transcript receives the same τ_{leaf}-bit projection as a distinct final leaf transcript that shares its slot and is generated by an encryption computation, and let $\mathsf{hit}_{\mathsf{leaf}}$ be the event that a direct adversary query hits the bridged input of a construction-generated hidden-key final leaf transcript before that transcript is first created. Then, in the continuation,*

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \textit{ before } \mathsf{hit}_{\mathsf{leaf}}] \;\le\; \frac{n_{\mathsf{leaf}}(\mathcal{B})}{2^{\tau_{\mathsf{leaf}}}}.$$

Proof. Build a chronological leaf tag candidate sequence: a candidate is the bridged input of a leaf tag output point ($\mathcal{L}_{\mathsf{tag}}(j)$), appended the first time it is reached—by a direct query in $\mathcal{B}$'s oracle phase or by a construction computation generating a final leaf transcript—and not appended again. By Lemma 7, Corollary 8, and Lemma 5, two reaches of the same bridged input come from the same final leaf transcript, so repeated evaluations add no candidate—the lazy $\mathsf{RO}_{\mathsf{flat}}$ value is reused—and distinct final leaf transcripts give distinct candidates. No candidate's $\mathsf{RO}_{\mathsf{flat}}$ value is read before its append step, so the sequence meets the predictability and unrevealed-value premises of Lemma 9, whose collision case with $s = 2$ and $\tau = \tau_{\mathsf{leaf}}$ bounds the probability that two candidates share a τ_{leaf}-bit projection by $\binom{L}{2} \cdot 2^{-\tau_{\mathsf{leaf}}}$ for a length-L sequence. Every $\mathsf{bad}_{\mathsf{leaf}}$ pair is a pair of construction-generated candidates.

(1) Adversary-supplied keys. Direct queries may decode to a leaf tag class $\mathcal{L}_{\mathsf{tag}}(j)$ and are appended as candidates; together with the $n_{\mathsf{leaf}}(\mathcal{B})$ construction-generated transcripts the sequence has length at most $q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})$, so the case with $L = q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})$ gives $\Pr[\mathsf{bad}_{\mathsf{leaf}}] \le \binom{q_{\mathsf{RO}}(\mathcal{B})+n_{\mathsf{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\mathsf{leaf}}} = \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B}))$.

(2) Hidden key, same slot. Every $\mathsf{bad}_{\mathsf{leaf}}$ pair is a pair of construction-generated hidden-key transcripts. Before $\mathsf{hit}_{\mathsf{leaf}}$ no direct query has read the bridged $\mathcal{L}_{\mathsf{tag}}(j)$ input of any such transcript—by Lemma 7 such a query would decode under $\mathsf{KeyedTWOut}$ to the hidden key and its $\mathcal{L}_{\mathsf{tag}}(j)$ class, which is exactly $\mathsf{hit}_{\mathsf{leaf}}$—so each projection is first sampled, uniformly, when a construction computation first reads it. Nonce-respecting encryption uses each nonce in at most one encryption query, and within one encryption computation the leaf index separates final leaf transcripts, so each slot contains at most one encryption-generated final leaf transcript; each construction-generated final leaf transcript therefore belongs to at most one $\mathsf{bad}_{\mathsf{leaf}}$ pair, and at most $n_{\mathsf{leaf}}(\mathcal{B})$ pairs can occur.

In the all-reject continuation no leaf tag projection is exposed: non-replay verification replies are the constant reject, replay decisions read the visible replay table, plaintext-computing replies are keystream projections at stream output points (Corollary 8), and an encryption reply exposes ciphertext blocks and its final root tag, an $\mathsf{RO}_{\mathsf{flat}}$ value at a final-root address distinct from every $\mathcal{L}_{\mathsf{tag}}$ address. Condition on the visible continuation prefix and on every lazy-table coordinate other than the $\mathcal{L}_{\mathsf{tag}}$ projections of construction-generated hidden-key final leaf transcripts, deferring the sampling of those projections and of the final-root values that absorb them until needed. Under this conditioning the deferred projections are independent and uniform, while the identity of each slot's encryption-generated transcript—which may be a transcript first created by an earlier verification or plaintext-computing computation, when the encryption's absorbed leaf body reproduces an earlier absorbed body—is fixed. Each of the at most $n_{\mathsf{leaf}}(\mathcal{B})$ pairs therefore collides on its τ_{leaf}-bit projections with probability $2^{-\tau_{\mathsf{leaf}}}$, and a union bound gives the claim. By Leaf-Transcript Recovery (Lemma 6), on $\neg\mathsf{bad}_{\mathsf{leaf}}$ equal flattened final-root inputs of an encryption computation and any construction computation identify the same leaf transcript sequence. □

A.8 Encryption Transcript Marginal Uniformity

Lemma 12 (Encryption Transcript Marginal Uniformity)**.** *For any fixed nonce-respecting encryption query in the TW128 random oracle experiment, let* $\mathsf{hit}_{\mathsf{fresh}}$ *be the event that, before this query reaches some counted transcript point, a prior direct adversary query to* $\mathsf{RO}_{\mathsf{flat}}$ *has already hit the bridged input of that hidden-key point. On* $\neg\mathsf{hit}_{\mathsf{fresh}}$*, every transcript output of the query that contributes to a returned ciphertext block, to an internal leaf tag, or to the final root tag is either fresh when this query first generates its bridged input, or reuses the lazy-table value sampled uniformly when that input was first generated by an earlier construction computation. The counted output points first generated by the fixed query are pairwise distinct.*

Proof. Work on an execution outside $\mathsf{hit}_{\mathsf{fresh}}$ for the fixed query. This rules out prior adversary reads of the hidden-key construction inputs considered below. A previous construction computation may nevertheless have generated one of these inputs, for example through a non-replay verification query using the same nonce. Such a lookup samples the lazy-table value uniformly when it first occurs; if the fixed encryption query later reaches the same bridged input, it reuses that same value. Thus it remains only to show that the counted output points newly generated by the fixed encryption query are distinct from one another and from previous nonce-respecting encryption queries. Conditional information leaked by earlier verification comparisons of a final root tag is accounted for separately by Lemma 14, not by this lemma.

First root keystream block. The first root keystream block, when present, is produced by the root duplex after the initialization call $p_{\mathsf{root}} \parallel K \parallel U$ and any AD absorption (elided when $A = \varepsilon$, in which case the σ_{init}call itself emits the first root keystream block). Since the nonce-respecting adversary never repeats a nonce, no previous encryption query can have used the same root initialization transcript. Within the current query, Lemma 5 separates initialization, AD, message, and aggregation calls by their suffixes and by the duplex-call encoding, so the first root keystream transcript is distinct from the other counted output points generated by this query. If the requested keystream length is zero, no ciphertext block is returned at that point and there is nothing to prove for that output.

First leaf keystream block. For each leaf chunk $j \geq 1$, the first leaf keystream block is produced after the initialization call $p_{\mathsf{leaf}} \parallel K \parallel U \parallel \langle j \rangle_8$. The prefix separates leaves from the root, the nonce separates this encryption query from all previous encryption queries, and $\langle j \rangle_8$ separates leaves within this query. Hence every first leaf keystream transcript newly generated by the query is distinct.

Later message outputs. Argue inductively within each root or leaf message stream. Suppose the current message-output transcript is either first generated by this query or reuses a value generated by an earlier construction computation. If its output is used as a keystream block, write its $\mathsf{RO}_{\mathsf{flat}}$ value as Z_i. Since the adaptive plaintext block M_i is a deterministic function of the prior view,

$$C_i = M_i \oplus Z_i$$

is now fixed. TW128 then absorbs C_i with the appropriate message phase suffix (σ^-_{msg} or σ^+_{msg}) before requesting the next keystream block. Once C_i is fixed, the next transcript is fixed; Lemma 5's injectivity and domain separation imply it has not appeared earlier unless the same duplex-call prefix has appeared earlier. Nonce uniqueness excludes this across encryption queries. By Corollary 8, the next message-keystream output point can coincide with a within-query earlier output point only if both belong to the same output-point class on the same duplex instance (root or leaf-j with matching j) and have the same duplex-call sequence; the current message-stream prefix is strictly longer than every earlier prefix in that stream, so this is excluded. Across distinct instances within the same query (root vs leaf, or two leaves with distinct j), the prefixes already differ in the first call's σ by $p_{\mathsf{root}}/p_{\mathsf{leaf}}$ or $\langle j \rangle_8$ respectively. Each later message-output transcript that emits keystream is therefore distinct from the other points generated by this query, unless it is a reuse of a point generated by an earlier construction computation. The terminal message-output transcript of a leaf emits the leaf tag rather than a following keystream block; the same separation argument applies to that transcript, so each internal leaf tag newly generated by the query is sampled at a distinct point. Empty inputs still induce a duplex call, but with zero keystream output they contribute no ciphertext block.

Final root tag transcript. The final root tag is sampled at the root tag output point selected by $\|C\|$: the $\mathcal{R}_{\mathsf{tag}}$ point after the root duplex absorbs the aggregation transcript built from the leaf tags and $\langle n \rangle_8$ when $n \geq 1$, and the $\mathcal{R}^+_{\mathsf{msg}}$ point closing the root message phase when $n = 0$ (Section 3.5). When $n \geq 1$, the leaf tags are themselves $\mathsf{RO}_{\mathsf{flat}}$ outputs on final leaf transcripts; condition on their sampled values, on the sampled ciphertext, and on the adversary-visible prior view and $\mathsf{RO}_{\mathsf{flat}}$ table state, so that the root aggregation transcript is fixed. When $n = 0$ there is no aggregation transcript; condition on the sampled ciphertext and the adversary-visible prior view alone. By Corollary 8, the selected root tag output-point class is separated from every other output-point class in the same query — including root initialization (suffix σ_{init}), AD-phase outputs, the remaining message phase outputs, leaf-side output points (separated by p_{leaf}), and, in the $n \geq 1$

case, earlier $\mathcal{R}_{\mathsf{agg}}$-prefix output points (separated by the $\sigma^{-}_{\mathsf{agg}}$vs $\sigma^{+}_{\mathsf{agg}}$suffix). Even if two encryption queries realize the same aggregation transcript (or, for $n = 0$, the same root ciphertext), the full root transcript also contains the root initialization $p_{\mathsf{root}} \parallel K \parallel U$, and nonce uniqueness separates that initialization from all previous encryption queries. The final root tag transcript is therefore either newly generated at a distinct point of the fixed query, or is a reuse of a value first sampled by an earlier construction computation. In the newly generated case, its τ_{root}-bit $\mathsf{RO}_{\mathsf{flat}}$ output is a fresh lazy-table sample; in the reuse case, it is the same uniform sample drawn when the bridged input was first generated. ☐

A.9 Mu-Ind Verification Coupling

The direct mu-ind adjacent-hybrid proof uses the generic key-test and leaf-collision accounting lemmas together with the verification-specific facts below. We use the root tag output point and final root transcript classes of Definition 3; in particular, computations on a common ciphertext share the same root tag output point because leaves(C) is fixed by $\|C\|$.

Lemma 13 (Verification Visible-Prefix Key Uniformity)**.** *In the stopped single-key* TW128 *random oracle encryption/verification interaction with hidden key K, direct access to* $\mathsf{RO}_{\mathsf{flat}}$*, and immediate abort on* $\mathsf{bad}_{\mathsf{key}}$*, order the direct* $\mathsf{RO}_{\mathsf{flat}}$ *queries. For a direct query position i, let S_i be the set of distinct candidate keys decoded by* KeyedTWOut *from earlier direct-query inputs. Condition on any positive-probability visible execution prefix immediately before query i, on no earlier* $\mathsf{bad}_{\mathsf{key}}$*, and on S_i. Then the hidden key K is uniform on $\{0,1\}^k \setminus S_i$. Consequently the uniformity invariant required by Lemma 10 holds in the stopped interaction.*

Proof. The visible prefix contains the adversary's oracle queries and replies: direct $\mathsf{RO}_{\mathsf{flat}}$ outputs, encryption outputs and the resulting replay table, verification bits, and the adversary state determined by these values. Fix such a prefix and two keys $K_0, K_1 \notin S_i$.

Let $\Phi_{0\to1} = \Phi_{K_0\to K_1}$ be the key-renaming map of Lemma 8 and $\Phi_{1\to0}$ its inverse. By that lemma, $\mathsf{flat} \circ \Phi_{0\to1} \circ \mathsf{flat}^{-1}$ is a bijection $\mathcal{Y}_{K_0} \to \mathcal{Y}_{K_1}$ between the bridged hidden-key construction addresses for K_0 and for K_1, preserving output-point classes and equality relations among addresses.

This bijection is applied online, not to a pre-existing fixed address set. Whenever a construction computation under K_0 would first query a hidden-key address Y, the coupled computation under K_1 queries $\Phi_{0\to1}(Y)$ and receives the same lazy-table value; repeated queries are coupled consistently through the same map. If the queried address is a root aggregation address containing leaf tag bytes, those bytes are already coupled equal because the corresponding leaf tag lookups were either repeated or first sampled earlier under the same renaming rule. Thus adaptively generated message transcripts and root aggregation transcripts are renamed step by step with the same visible ciphertext, leaf tag, and root tag bytes.

Prior direct-query inputs are not renamed: they are adversary-chosen visible strings. Since $K_0, K_1 \notin S_i$, neither $\mathcal{Y}_{K_0}$ nor $\mathcal{Y}_{K_1}$ contains a prior direct-query input (Lemma 8, direct-query avoidance). Direct-query lazy-table values are therefore coupled identically and remain disjoint from the renamed hidden construction addresses.

Use the same adversary coins, the same direct-query lazy-table values, and the online renamed construction-internal lazy-table values. Encryption outputs have the same distribution under the coupling, because every construction lookup that determines ciphertext, leaf tags, or the final root tag is paired with a distinct renamed lookup carrying the same sampled value. Replay verification decisions depend only on the visible replay table. Non-replay verification computations are renamed in the same online way as encryption computations, and the final root tag comparison therefore gives the same accept/reject bit. Verification does not reveal plaintext on rejection or acceptance; it reveals only that bit.

Thus every visible prefix with hidden key $K_0 \notin S_i$ has the same conditional probability as the corresponding execution with hidden key $K_1 \notin S_i$. Since K is initially uniform and the condition "no earlier $\mathsf{bad}_{\mathsf{key}}$" removes exactly the keys in S_i, all keys in $\{0,1\}^k \setminus S_i$ remain equally likely. $\square$

Foreign-key key-test claim. In the pre-sampled-key mu-ind environment of Lemma 16, fix user d and work in the original experiment, where the key-collision branch outputs a fixed bit rather than conditioning on pairwise distinct keys. Let $\mathsf{bad}_{\mathsf{key}}^{(d)}$ be the event that a direct $\mathsf{RO}_{\mathsf{flat}}$ query decodes under $\mathsf{KeyedTWOut}$ to K_d and a counted output-point class. Then

$$\Pr[\mathsf{bad}_{\mathsf{key}}^{(d)}] \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

Fix the foreign keys and leave their transcript addresses unchanged. Before the first user-d key hit, the visible prefix is invariant under renaming K_d among keys that have not already been tested and are not equal to a fixed foreign key, by the same online coupling as Lemma 13 and the key-field disjointness of Lemma 8. For each fresh candidate key tested by one of $\mathcal{B}$'s direct $\mathsf{RO}_{\mathsf{flat}}$ queries, the contributing event is contained in $\{K_d$ equals that candidate$\}$ and costs at most 2^{-k} in the original pre-sampled-key experiment; a candidate equal to a fixed foreign key contributes only on the fixed-output key-collision branch and therefore contributes nothing to $\mathsf{bad}_{\mathsf{key}}^{(d)}$. A union bound over at most $q_{\mathsf{RO}}(\mathcal{B})$ direct queries gives the claim.

Lemma 14 (Verification-First Root Guessing)**.** *In the stopped single-key* TW128 *random oracle encryption/verification interaction before* $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$*, let* $\mathsf{win}_{\mathsf{vf}}$ *be the event that some accepting non-replay verification submission belongs to a verification-first final root transcript class. Then*

$$\Pr[\mathsf{win}_{\mathsf{vf}}] \le q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Proof. Throughout, a *non-replay verification submission* means a valid non-replay verification query; by the validity convention, an invalid verification query returns 0 and performs no construction computation, so it realizes no bridged final root input, belongs to no final root transcript class, and contributes to no $G_{\mathcal{C}}$.

Work in the game stopped at the first occurrence of $\mathsf{bad}_{\mathsf{key}}$ or $\mathsf{bad}_{\mathsf{leaf}}$. For accounting, define an all-reject continuation: encryption queries are answered normally, replay verification queries accept normally, and every non-replay verification submission performs the same construction computation but returns reject to the adversary. Lazy sampling is deferred for each verification-first class's final root tag value: the continuation records the class address and the submitted tag, but while the class remains verification-first it does not expose that root tag value to the adversary. The execution-uniform count $q_{\mathsf{v}}(\mathcal{B})$ applies to this continuation.

Fix a verification-first final root transcript class $\mathcal{C}$ and follow the all-reject path until the first later encryption query generating the same class, if any. Let $G_{\mathcal{C}}$ be the set of distinct candidate tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff. Repeated tags only shrink this set. The class's correct root tag $U_{\mathcal{C}}$ is the τ_{root}-bit projection of $\mathsf{RO}_{\mathsf{flat}}$ at the class's bridged root tag input. Before the stop, no direct query has hit this hidden-key input: such a query would be decoded by $\mathsf{KeyedTWOut}$ to the hidden key and would trigger $\mathsf{bad}_{\mathsf{key}}$. Before the cutoff, no encryption query has revealed $U_{\mathcal{C}}$ as an encryption tag. (When the class's root tag point is $\mathcal{R}^{+}_{\mathsf{msg}}$, i.e. $n = 0$, an encryption query on a longer message that shares chunk 0 may touch this same address while closing its chunk-0 message phase, but it requests zero output there and continues into its aggregation phase, so it does not reveal $U_{\mathcal{C}}$; its own tag is emitted at a later $\mathcal{R}_{\mathsf{tag}}$ point.) The all-reject path and the set $G_{\mathcal{C}}$ are therefore functions only of the adversary's coins, visible oracle replies, and lazy-table values at inputs other than this class's root tag address. By lazy sampling, conditioned on those values and on the fixed path, $U_{\mathcal{C}}$ may be sampled after $G_{\mathcal{C}}$ is fixed and is uniform independently of $G_{\mathcal{C}}$.

After the cutoff, if it exists, class $\mathcal{C}$ contributes no accepting non-replay submission. Let the cutoff encryption query return (U_e, A_e, C_e, T_e). Any later verification submission in the same final root transcript class has, by the final-root determinacy of Lemma 5 and Leaf-Transcript Recovery (Lemma 6) under the absence of $\mathsf{bad}_{\mathsf{leaf}}$—which supplies the recovery hypothesis because the cutoff computation is an encryption computation—the same (U, A, C) as that encryption computation. If the submitted tag is T_e, the tuple is a recorded encryption tuple and the verification is a replay; if the submitted tag is not T_e, the final tag comparison fails.

Consider the event that the first accepting non-replay verification submission in the stopped game belongs to class $\mathcal{C}$. Up to that global first accept, the adversary's visible answers match the all-reject continuation: every earlier non-replay submission rejects, replay queries accept in both executions, and encryption answers are unchanged. The construction computations also agree at every address except that the all-reject continuation defers the final root tag value for class $\mathcal{C}$ until the comparison is needed. Therefore the first accepting non-replay submission can belong to class $\mathcal{C}$ only before the cutoff, and only if $U_{\mathcal{C}} \in G_{\mathcal{C}}$, which has probability at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

Each non-replay verification submission deterministically reconstructs one final root transcript at the root tag output point. If the class is verification-first and the submission occurs before that class's encryption cutoff, the submitted tag contributes to at most one set $G_{\mathcal{C}}$; otherwise it contributes to none. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \le q_{\mathsf{v}}(\mathcal{B})$, and a union bound over the verification-first classes gives the claim. □

Lemma 15 (INT-RUP Root Guessing)**.** *In the stopped single-key TW128 random oracle encryption/plaintext-computing/verification interaction before* $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$*, let* forge *be the event that some valid non-replay verification query accepts. Then*

$$\Pr[\mathsf{forge} \textit{ before } \mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}] \le q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Proof. Throughout, a *non-replay verification submission* means a valid non-replay verification query; invalid verification queries return 0, perform no construction computation, realize no bridged final root input, and contribute to no set of guessed tags.

Work in the game stopped at the first occurrence of $\mathsf{bad}_{\mathsf{key}}$ or $\mathsf{bad}_{\mathsf{leaf}}$. A final root transcript class is *tag-revealed* once an encryption query has returned the root tag for that class. If a non-replay verification submission belongs to a tag-revealed class, compare it with an encryption computation that revealed the class. The two computations are well-formed typed transcript prefixes with the same bridged final root input; by Lemma 7, they have the same native flattened input. Then final-root determinacy (Lemma 5) and Leaf-Transcript Recovery (Lemma 6) under $\neg\mathsf{bad}_{\mathsf{leaf}}$—which supplies the recovery hypothesis because the comparand is an encryption computation—recover the same (U, A, C). If the submitted tag is the revealed tag, the tuple is a replay-table hit; if it is any other tag, the final comparison rejects. Thus any accepting non-replay verification submission must lie in a class whose root tag has not yet been revealed by encryption.

For tag-unrevealed classes, define an all-reject continuation: encryption and plaintext-computing queries are answered normally, replay verification queries accept normally, and each non-replay verification submission performs the same construction computation but returns reject to the adversary. Plaintext-computing queries may generate the same stream and leaf tag transcripts as decryption, but they do not reveal the final root tag value. Equivalently, the final root tag sample for a tag-unrevealed class can be deferred until it is first needed for an encryption answer or a verification comparison; this does not change any plaintext reply. By Output-Point Separation (Corollary 8), the keystream values used to compute returned plaintext blocks are projections at stream output points, not at the final root tag output point.

Fix a tag-unrevealed final root transcript class $\mathcal{C}$ and follow the all-reject path until the first later encryption query that reveals the class, if any. Let $G_{\mathcal{C}}$ be the set of distinct tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff. Before the cutoff, no encryption query has returned the class tag and no direct query has read the hidden-key root tag point without triggering $\mathsf{bad}_{\mathsf{key}}$. Conditioned on the all-reject path and on all lazy-table values except this final root tag projection, the correct tag for $\mathcal{C}$ is uniform in $\{0,1\}^{\tau_{\mathsf{root}}}$ and independent of $G_{\mathcal{C}}$. The probability that any pre-cutoff verification submission in the class accepts is therefore at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

Each valid non-replay verification query contributes its submitted tag to at most one such set $G_{\mathcal{C}}$, and replay-table hits do not contribute. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \leq q_{\mathsf{v}}(\mathcal{B})$, and a union bound over tag-unrevealed classes gives the claim. □

A.10 Mu-Ind Adjacent-Hybrid Lemma

The multi-user indistinguishability theorem (Theorem 2) telescopes a hybrid over the D users, replacing one real user by its ideal $(\mathsf{Rand}, \mathsf{Replay})$ interface at each step. The following lemma bounds one such step directly inside the shared random oracle environment. It couples the real and ideal user-d interfaces until one of three local events occurs: a direct hidden-key transcript hit, a hidden leaf tag collision, or an accepting non-replay verification query.

Lemma 16 (Mu-Ind Adjacent-Hybrid Bound)**.** *Let $\mathcal{B}$ be a flat-RO mu-ind adversary in the pre-sampled-key formulation of Section 4.2, with keys $K_1, \ldots, K_D$ sampled at the start of the experiment. Fix $d \in \{1, \ldots, D\}$. Let $H^\star_{d-1}$ and $H^\star_d$ be the adjacent stopped hybrids that output a fixed bit if any two sampled keys collide, and otherwise run $\mathcal{B}$ with one shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table as follows: users $e < d$ are answered by $(\mathsf{Rand}, \mathsf{Replay})$, users $e > d$ are answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e})$, and user d is answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ in $H^\star_{d-1}$ and by $(\mathsf{Rand}, \mathsf{Replay})$ in $H^\star_d$. If $n^{(d)}_{\mathsf{leaf}}$ and $q^{(d)}_{\mathsf{v}}$ are the user-d resource caps, then*

$$\left|\Pr[\mathcal{B}^{H^\star_{d-1}} \Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_d} \Rightarrow 1]\right| \leq \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. The fixed-output branch on a key collision is identical in $H^\star_{d-1}$ and $H^\star_d$, so only the branch with pairwise distinct keys contributes. On that branch, the two hybrids differ only in user d: users $e < d$ are ideal in both hybrids, users $e > d$ are real in both hybrids, and user d is answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ in $H^\star_{d-1}$ and by $(\mathsf{Rand}, \mathsf{Replay})$ in $H^\star_d$.

Let $\mathcal{E}_d$ denote the common surrounding environment. It runs $\mathcal{B}$, answers users $e < d$ by their $(\mathsf{Rand}, \mathsf{Replay})$ interfaces, answers users $e > d$ by $\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e}$, forwards $\mathcal{B}$'s direct $\mathsf{RO}_{\mathsf{flat}}$ queries unchanged, and shares one lazy $\mathsf{RO}_{\mathsf{flat}}$ table with user d. The coupling treats user d's encryption and verification interfaces together. Direct $\mathsf{RO}_{\mathsf{flat}}$ queries that decode to K_d can affect both interfaces, so the local key-test event is charged once in the adjacent-hybrid bound.

Coupling until divergence. Run the two hybrids on shared adversary coins and one shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table. Introduce the user-d local events $\mathsf{bad}^{(d)}_{\mathsf{key}}$, that a direct $\mathsf{RO}_{\mathsf{flat}}$ query decodes under $\mathsf{KeyedTWOut}$ to K_d and a counted output-point class; $\mathsf{bad}^{(d)}_{\mathsf{leaf}}$, that some user-$d$ construction-generated final leaf transcript receives the same τ_{leaf}-bit projection as a distinct user-d final leaf transcript that shares its key, nonce, and leaf index and is generated by a user-d encryption computation (the same-slot event of Lemma 11); and $\mathsf{forge}^{(d)}$, that a non-replay user-d verification query is accepted by $\mathsf{Ver}^{\mathsf{RO}}_{K_d}$.

Couple the two hybrids on shared adversary coins and one shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table, with user d's Rand answers in $H^\star_d$ set equal to user d's real encryption answers in $H^\star_{d-1}$. We show the two hybrids' visible transcripts agree until $\mathsf{bad}^{(d)}_{\mathsf{key}} \cup \mathsf{forge}^{(d)}$.

Claim 1: the coupled ideal coordinate has the true law. Each real user-d encryption output is a projection of $\mathsf{RO}_{\mathsf{flat}}$ at transcript points that are distinct within its computation and, by encryption nonce-uniqueness and Transcript Injectivity (Lemma 5), across distinct encryptions. By Lemma 12, a point first generated by an encryption query receives a fresh lazy-table sample, while a point first generated by an earlier construction computation is later reused with the same sampled value. The present environment adds a real user-d verification oracle, absent from a pure encryption-replacement game. Because only encryption queries are nonce-unique, a non-replay verification may reuse an encryption nonce and lazily read a keystream or root tag point the same-nonce encryption later reaches; but lazy sampling draws each point uniformly when first read, so its value is unchanged and stays marginally uniform. In $H_d^\star$ the user-d verification oracle is Replay, which answers from the recorded ciphertexts without reading $\mathsf{RO}_{\mathsf{flat}}$. Hence the coupled $H_d^\star$ transcript has the law of the true $H_d^\star$ until $\mathsf{bad}_{\mathsf{key}}^{(d)}$: its encryption answers are the marginally uniform strings of Rand and its verification answers are the replay decisions, with no further $\mathsf{RO}_{\mathsf{flat}}$ dependence. (Conditioning on a particular $H_{d-1}^\star$ verification outcome can constrain a coupled tag, but that is a property of the joint coupling; the coupled $H_d^\star$ marginal is what must match the true $H_d^\star$, and it does.)

Claim 2: agreement until divergence. Encryption answers are equal by construction. A verification query whose tuple is recorded returns accept in both hybrids through the shared replay-table membership check, which precedes the real-versus-ideal decryption branch (Section 4.2), so real decryption is never run on a recorded tuple. A non-replay query returns reject in $H_d^\star$ and the bit $[\,\mathsf{Ver}_{K_d}^{\mathsf{RO}}$ accepts$\,]$ in $H_{d-1}^\star$; the two agree unless that bit is accept, the event $\mathsf{forge}^{(d)}$. A real verification reads hidden-key points but exposes only this bit, which is independent of the keystream values it reads—decryption absorbs the submitted ciphertext, not the recovered plaintext (Lemma 4)—and depends on an encryption output only through the root tag it recomputes. In the reused-nonce case above this correlates the $H_{d-1}^\star$ bit with that encryption's tag, but the $H_d^\star$ verification is Replay and the coupled encryption answers coincide, so the two coordinates still agree on such a query unless it accepts. Direct-query answers are read from the shared table and agree until a direct query reads a user-d hidden-key point ($\mathsf{bad}_{\mathsf{key}}^{(d)}$). Therefore the visible transcripts agree until $\mathsf{bad}_{\mathsf{key}}^{(d)} \cup \mathsf{forge}^{(d)}$, and a union bound ordered by first occurrence—splitting $\Pr[\mathsf{forge}^{(d)}$ before $\mathsf{bad}_{\mathsf{key}}^{(d)}]$ at the first occurrence of the leaf abort into $\Pr[\mathsf{bad}_{\mathsf{leaf}}^{(d)}$ before $\mathsf{bad}_{\mathsf{key}}^{(d)} \cup \mathsf{forge}^{(d)}] + \Pr[\mathsf{forge}^{(d)}$ before $\mathsf{bad}_{\mathsf{key}}^{(d)} \cup \mathsf{bad}_{\mathsf{leaf}}^{(d)}]$—gives

$$\begin{aligned}&\left|\Pr[\mathcal{B}^{H_{d-1}^\star} \Rightarrow 1] - \Pr[\mathcal{B}^{H_d^\star} \Rightarrow 1]\right|\\ &\le \Pr[\mathsf{bad}_{\mathsf{key}}^{(d)}] + \Pr[\mathsf{bad}_{\mathsf{leaf}}^{(d)} \text{ before } \mathsf{bad}_{\mathsf{key}}^{(d)} \cup \mathsf{forge}^{(d)}]\\ &\qquad + \Pr[\mathsf{forge}^{(d)} \text{ before } \mathsf{bad}_{\mathsf{key}}^{(d)} \cup \mathsf{bad}_{\mathsf{leaf}}^{(d)}].\end{aligned}$$

Claim 3: the environment does not enlarge the user-d events. The real users $e > d$ perform construction-internal $\mathsf{RO}_{\mathsf{flat}}$ lookups while simulating $\mathsf{Enc}_{K_e}^{\mathsf{RO}}$ and $\mathsf{Ver}_{K_e}^{\mathsf{RO}}$. Whenever such a lookup lies in a counted keyed transcript class, Lemma 7 decodes its candidate key as K_e, which on the non-collision branch differs from K_d; a lookup outside the counted keyed language is irrelevant to $\mathsf{bad}_{\mathsf{key}}^{(d)}$. Foreign-user lookups therefore neither trigger user d's key-test event nor pre-read a user-d hidden transcript point, and they create no user-d leaf tag collision or user-d non-replay verification submission, which are counted by $n_{\mathsf{leaf}}^{(d)}$ and $q_{\mathsf{v}}^{(d)}$. The three accounting events are thus local to user d's keyed transcript language.

Claim 4: bounding the three terms. For $\Pr[\mathsf{bad}^{(d)}_{\mathsf{key}}]$, the foreign-key key-test claim above gives $\Pr[\mathsf{bad}^{(d)}_{\mathsf{key}}] \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$. For the leaf term, before $\mathsf{bad}^{(d)}_{\mathsf{key}}$ no direct query has pre-read the bridged input of a user-d hidden-key final leaf transcript—such a query would decode under $\mathsf{KeyedTWOut}$ to K_d and its $\mathcal{L}_{\mathsf{tag}}(j)$ class and would itself be $\mathsf{bad}^{(d)}_{\mathsf{key}}$. User-$d$ encryption is nonce-respecting on the distinct-keys branch, and the visible view agrees with the all-reject continuation until the first accepting non-replay submission ($\mathsf{forge}^{(d)}$), so the same-slot case of Lemma 11 gives $\Pr[\mathsf{bad}^{(d)}_{\mathsf{leaf}}$ before $\mathsf{bad}^{(d)}_{\mathsf{key}} \cup \mathsf{forge}^{(d)}] \le n^{(d)}_{\mathsf{leaf}}/2^{\tau_{\mathsf{leaf}}}$. For the forgery term, before $\mathsf{bad}^{(d)}_{\mathsf{key}} \cup \mathsf{bad}^{(d)}_{\mathsf{leaf}}$ every encryption-first final root transcript class admits no accepting non-replay submission—by the final-root determinacy of Lemma 5 and Leaf-Transcript Recovery (Lemma 6), whose hypothesis $\neg\mathsf{bad}^{(d)}_{\mathsf{leaf}}$ supplies because the comparand is the class's encryption computation, a non-replay submission in such a class reaches the already-returned root tag and fails the tag comparison—so every accepting non-replay submission is verification-first, and Lemma 14 gives $\Pr[\mathsf{forge}^{(d)}$ before $\mathsf{bad}^{(d)}_{\mathsf{key}} \cup \mathsf{bad}^{(d)}_{\mathsf{leaf}}] \le q^{(d)}_{\mathsf{v}}/2^{\tau_{\mathsf{root}}}$.

Summing the three bounds gives

$$\left|\Pr[\mathcal{B}^{H^\star_{d-1}} \Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_d} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}}{2^{\tau_{\mathsf{leaf}}}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}},$$

with the user-d key-test event $\mathsf{bad}^{(d)}_{\mathsf{key}}$ charged a single time for the adjacent hybrid. □

B The Flat Sponge Lift

This appendix supplies the flat sponge lift machinery (Lemmas 18 and 19 and Corollary 9) invoked by the AEAD security bounds of Section 5, the root tag bounds of Section 6, and the committing-notion bound of Corollary 6. The machinery applies uniformly to the games named once here.

The lift has two steps. First, the [BDPVA08] indifferentiability theorem replaces the construction interface by a flat random oracle and answers only direct public-permutation queries through a simulator over that oracle, at cost $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ counts construction-internal primitive calls induced by construction queries together with direct primitive queries. Second, the simulator-world adversary is converted into a flat-RO adversary whose direct random oracle query count is at most N_{prim}. Thus the random oracle bounds are instantiated at $q_{\mathsf{RO}} \le N_{\mathsf{prim}}$, while the public permutation lift pays $\mathsf{Lift}_c(N + N_{\mathsf{prim}})$.

Definition 5 (Single-Stage $\mathsf{TW128}$ Games)**.** The *single-stage $\mathsf{TW128}$ games* are the games of Sections 4.1 and 4.2 whose entire public-permutation transcript is handled by one [BDPVA08] replacement, including deterministic challenger checks after the adversary's final output. They are all-in-one AE, mu-ind, $\mathsf{INT\text{-}RUP}$, CMTDs-4, the $\mathsf{H}_{\mathsf{TW128}}$ collision and preimage games, and CMTs-ℓ for $\ell \in \{1, 3, 4\}$.

The derived adversary forwards the following game data to the corresponding flat-RO experiment.

Game family	Forwarded data
AE / mu-ind / $\mathsf{INT\text{-}RUP}$	construction queries, including New where present and replay state
$\mathsf{H}_{\mathsf{TW128}}$ hash games	final structured hash game output
CMTDs / CMTs	final openings and deterministic challenger checks

B.1 Simulator Convention

Throughout this appendix, $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ denotes the public primitive simulator supplied by [BDPVA08, Theorem 2] — the indifferentiability theorem for the simple padded sponge over a random permutation — after applying the [BDPVA11, Theorem 3] bridge of Lemma 7 to TW128's native pad10*1 transcript inputs. The superscript records that both simulator directions share the same internal lazy $\mathsf{RO}_{\mathsf{flat}}$ table.

This is the simulator referenced by the Sections 4.1 and 4.2 definitions of $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}$ and $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}$: the ideal side of each AE-style experiment exposes the construction interface specified by the game $((\mathsf{Rand}, \mathsf{Replay})$, instantiated per user in mu-ind) together with $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ as the public primitive interface. The $\mathsf{RO}_{\mathsf{flat}}$ table in this notation is internal to the ideal system and is not an extra interface exposed to the application adversary; the committing security advantages remain the real public permutation success probabilities of Section 4.1, with the simulator appearing only inside the reduction to the corresponding random oracle model adversary.

Lemma 17 (At Most One Simulator RO Call)**.** *For the simulator* $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$*, every primitive-interface query induces at most one simulator-side* $\mathsf{RO}_{\mathsf{flat}}$ *call. More precisely, every inverse primitive-interface query and every forward primitive-interface query violating one of the following conditions induces no simulator* $\mathsf{RO}_{\mathsf{flat}}$ *call. The symbols* s*,* p*,* $\mathcal{R}$*,* $\mathcal{O}$*, and* $\mathcal{C}$ *are the local internal notation of the BDPV simple-padded sponge simulator, not TW128 resource notation:*

- **(C1) New edge.** *The simulator's internal graph has no outgoing edge from the input node* s*.*
- **(C2) Unsaturated.** $\mathcal{R} \cup \mathcal{O} \neq \mathcal{C}$*.*
- **(C3) Rooted input.** *The input node is rooted.*
- **(C4) Paddable path.** *The constructed path* p *decomposes uniquely as* $p = p' \parallel 0^{rj}$ *with* p' *not ending in* 0^r*, and the prefix* p' *is a valid padded sponge input: equivalently,* $p' = x \parallel \mathsf{pad10^*}[r](|x|)$ *for an unpadded BDPV08 random oracle input* x*.*

When all four conditions hold, the forward primitive-interface query induces one simulator $\mathsf{RO}_{\mathsf{flat}}$ *call. Consequently, each primitive-interface query induces at most one simulator-side* $\mathsf{RO}_{\mathsf{flat}}$ *call.*

Proof. This is the branch structure of the [BDPVA08] simple-padded permutation simulator. The simulator invokes the random oracle only on the forward-query branch where the input node is rooted, unsaturated, paddable to an unpadded random oracle input x, and not already extended by an outgoing edge in the simulator graph. Forward queries violating any of these conditions and all inverse queries return outputs sampled or read from the simulator's internal graph without invoking the random oracle. Under the [BDPVA11, Theorem 3] bridge fixed in Section B.1, that simulator random oracle invocation is the simulator-side $\mathsf{RO}_{\mathsf{flat}}$ call used throughout this appendix. □

Lemma 18 (Flat Sponge Lift Bound)**.** *Let* G *be a single-stage TW128 game (Definition 5). Let* $\mathcal{A}$ *be a public permutation* G *adversary with construction interface cost at most* N *and at most* N_{prim} *direct primitive-interface queries. In the [BDPVA08] single-stage replacement, the combined construction-and-primitive transcript has length at most*

$$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}.$$

Consequently replacing the real public permutation and TW128 construction by the simulator-world primitive interface $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ *and the corresponding flat-RO construction interface costs at most* $\mathsf{Lift}_c(N_{\mathsf{tot}})$*.*

Proof. The [BDPVA08] replacement is applied before the derived-adversary construction, so its transcript contains two kinds of public primitive activity. First, each construction interface computation contributes the TW128 duplex calls it performs. By Lemmas 4 and 7, each such root or leaf output point is a typed restriction of the bridged flat sponge construction, and by the definition of N in Section 4.4 these calls are counted at the same per-block granularity as the underlying KECCAK-p[1600, 12] calls. This includes plaintext-computing computations, verification computations whether they accept or reject, the per-user sum of construction calls in mu-ind, the deterministic challenger decryption or encryption checks actually performed in the committing games, and the hash games' challenger encryptions—the collision game's s submitted-input encryptions and the preimage game's source encryption and post-output check.

Second, each direct adversary query to (π, π^{-1}) contributes one primitive-interface query and is counted in N_{prim}. These two counts are disjoint in the public permutation experiment: N counts construction-internal use of the primitive, while N_{prim} counts only the adversary's public primitive interface queries. The execution-uniform caps ensure that every adaptive transcript seen by the single-stage replacement has length at most $N + N_{\mathsf{prim}} = N_{\mathsf{tot}}$. If $N_{\mathsf{tot}} \geq 2^c$ then $\mathsf{Lift}_c(N_{\mathsf{tot}}) \geq 1$ and the claimed bound holds trivially, so assume $N_{\mathsf{tot}} < 2^c$, satisfying the hypothesis of [BDPVA08, Theorem 2]. That theorem gives distinguishing or success-probability cost at most the random-permutation differentiating bound $f_P(N_{\mathsf{tot}})$ of [BDPVA08, Lemma 5]. The [BDPVA08, Lemmas 4 and 5] product bounds give the random-transformation bound $f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}(1 - i/2^c)$, and the factor of $f_P(N_{\mathsf{tot}})$ corresponding to the f_T factor at index i is that factor divided by a quantity $1 - (i-1)/2^{r+c} \in (0, 1]$, hence at least as large, so $f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}})$. With $N_{\mathsf{tot}} < 2^c$ every factor lies in $[0, 1]$ and

$$f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}\Big(1 - \frac{i}{2^c}\Big) \leq \sum_{i=1}^{N_{\mathsf{tot}}} \frac{i}{2^c} = \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)}{2^{c+1}} = \mathsf{Lift}_c(N_{\mathsf{tot}}),$$

the second inequality by the Weierstrass product inequality $\prod_i(1 - x_i) \geq 1 - \sum_i x_i$ for $x_i \in [0, 1]$. $\square$

Lemma 19 (Derived-Adversary Execution Contract)**.** *Let* G *be a single-stage TW128 game (Definition 5). Let* $\mathcal{A}$ *be a simulator-world* G *adversary whose public primitive interface is answered by* $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$*, with at most* N_{prim} *direct primitive-interface queries. Define* $\mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ *to run* $\mathcal{A}$ *internally, answer each primitive-interface query by running the same simulator against direct finite-output queries to its own* $\mathsf{RO}_{\mathsf{flat}}$ *oracle, forward construction interface queries unchanged in games with construction oracles, and forward the adversary's final structured output unchanged in the output-checked* $\mathsf{H}_{\mathsf{TW128}}$ *hash and committing games. Then the simulator-world execution of* $\mathcal{A}$ *and the random oracle execution of* $\mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ *have identical joint distributions, and*

$$q_{\mathsf{RO}}(\mathcal{B}_{\mathrm{derived}}) \leq N_{\mathsf{prim}}.$$

In this coupling, simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ *calls and construction-internal* RF *lookups use the same lazy table. Therefore, if a simulator-induced query is the bridged input* $\mathsf{flat}(X)$ *of a well-formed TW128 transcript prefix* X *and a forwarded construction computation later reaches the same prefix* X*, both interfaces read the same* $\mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X))$ *value, projected to the requested output length. Moreover,* $\mathcal{B}_{\mathrm{derived}}$ *inherits the construction-side caps of* $\mathcal{A}$*:*

$$q_{\mathsf{v}}(\mathcal{B}_{\mathrm{derived}}) = q_{\mathsf{v}}(\mathcal{A}) \quad \textit{(AE, mu-ind, INT-RUP)},$$
$$n_{\mathsf{leaf}}(\mathcal{B}_{\mathrm{derived}}) \leq n_{\mathsf{leaf}}(\mathcal{A}) \quad \textit{(all games above)}.$$

In the mu-ind game these resource inheritances hold per user, and the New*-query transcript is unchanged.*

Proof. Couple the two executions by using the same lazy $\mathsf{RO}_{\mathsf{flat}}$ table for the simulator and for all construction-internal $\mathsf{RF}(\cdot)$ lookups. In the simulator-world execution, $\mathcal{A}$ sees construction answers from the selected game interface and public primitive answers from $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$. In the derived random oracle execution, $\mathcal{B}_{\text{derived}}$ runs the same $\mathcal{A}$, forwards construction interface queries or the final structured output required by the selected game unchanged, and answers primitive-interface queries by invoking the same simulator code with the same lazy $\mathsf{RO}_{\mathsf{flat}}$ table. The construction interface, hash challenger, or committing challenger therefore receives the same forwarded queries or final output, the simulator receives the same primitive queries in the same order, and the lazy table supplies the same values. The full views of $\mathcal{A}$ in the simulator-world execution and of $\mathcal{B}_{\text{derived}}$ in the random oracle execution are identically distributed.

The simulator's random oracle calls may be on arbitrary [BDPVA08] H-input strings, not only bridged $\mathsf{TW128}$ transcript inputs. These calls are the direct $\mathsf{RO}_{\mathsf{flat}}$ queries of $\mathcal{B}_{\text{derived}}$ and are counted in $q_{\mathsf{RO}}(\mathcal{B}_{\text{derived}})$. Construction-internal $\mathsf{RF}(\cdot)$ lookups made by $\mathsf{Enc}_K^{\mathsf{RO}}$, $\mathsf{Dec}_K^{\mathsf{RO}}$, $\mathsf{PDec}_K^{\mathsf{RO}}$, $\mathsf{Ver}_K^{\mathsf{RO}}$, or a committing challenger are forwarded to the same $\mathsf{RO}_{\mathsf{flat}}$ table but are not counted in q_{RO}, exactly as in Section 4.4.

By Lemma 17, each primitive-interface query induces at most one direct $\mathsf{RO}_{\mathsf{flat}}$ query by $\mathcal{B}_{\text{derived}}$, and inverse queries or non-RO-touching forward queries induce none. Since $\mathcal{A}$ makes at most N_{prim} direct primitive-interface queries, this gives $q_{\mathsf{RO}}(\mathcal{B}_{\text{derived}}) \leq N_{\mathsf{prim}}$.

Lemma 7's bridge places $\mathsf{TW128}$ construction restrictions and simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls in a single lazy table. If the simulator invokes $\mathsf{RO}_{\mathsf{flat}}$ on $\mathsf{flat}(X)$ for a well-formed $\mathsf{TW128}$ transcript prefix X, and a forwarded construction computation later reaches X, the construction lookup $\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X))$ reads the same table entry and returns the requested projection. Since $\mathsf{TW128}$ uses $r = r_{\max}$, the [BDPVA11, Theorem 3] output post-processing does not alter $\mathsf{TW128}$ output bits. Thus simulator-induced direct queries and construction-internal lookups at the same bridged input are transcript-consistent. In the AE-style games, a simulator call whose input is accepted by $\mathsf{KeyedTWOut}$ for the hidden key is still a direct $\mathsf{RO}_{\mathsf{flat}}$ query and is covered by the same $\mathsf{bad}_{\mathsf{key}}$ accounting used by the direct random oracle mu-ind proof. The derivation step itself does not prove a key-guessing statement; it only supplies the derived adversary and the query bound at which the random oracle bounds of Sections 5 and 6 are instantiated.

Finally, the execution-uniform caps of Section 4.4 hold along every oracle-answer trace of $\mathcal{A}$, including traces generated by the simulator on the ideal public primitive interface. The derived-adversary construction does not add construction interface encryption, plaintext-computing, or verification queries and forwards $\mathcal{A}$'s construction interface queries unchanged along each trace. $\mathcal{B}_{\text{derived}}$ therefore inherits the relevant construction resources: $q_{\mathsf{v}}(\mathcal{B}_{\text{derived}}) = q_{\mathsf{v}}(\mathcal{A})$ in the AE, mu-ind, and $\mathsf{INT\text{-}RUP}$ games, and $n_{\mathsf{leaf}}(\mathcal{B}_{\text{derived}}) \leq n_{\mathsf{leaf}}(\mathcal{A})$ for the distinct construction-generated final-leaf-transcript cap. In mu-ind, construction interface forwarding preserves the New calls and these caps user by user. □

B.2 Lift Application

The lift bound of Lemma 18 and the derived-adversary construction of Lemma 19 combine into a single template, stated once and then instantiated by the bounds of Sections 5 and 6 and by the committing-notion bounds of Corollary 6. Every public-permutation bound is obtained by adding $\mathsf{Lift}_c(N_{\mathsf{tot}})$ and substituting $q_{\mathsf{RO}} \leq N_{\mathsf{prim}}$ in the corresponding random oracle theorem.

Corollary 9 (Lift Application)**.** *Let* G *be a single-stage* $\mathsf{TW128}$ *game (Definition 5). For every public permutation* G*-adversary* $\mathcal{A}$ *against* $\mathsf{TW128}$ *with the resources of Section 4.4, the derived adversary* $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$ *of Lemma 19 is a random oracle* G*-adversary with* $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$ *and the construction-side caps of Lemma 19—in particular* $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$

and $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$*, where these apply—such that*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}).$$

If moreover $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}) \le \varepsilon_{\mathsf{G}}(q_{\mathsf{RO}}(\mathcal{B}), n_{\mathsf{leaf}}(\mathcal{B}))$ *for some* ε_{G} *nondecreasing in each argument, then*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \varepsilon_{\mathsf{G}}(N_{\mathsf{prim}}, N_{\mathsf{leaf}}).$$

Proof. By the flat sponge lift bound (Lemma 18), replacing the real public permutation and $\mathsf{TW128}$ construction by $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ and the flat-RO construction interface costs at most $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ counts both the construction calls of N—including any construction oracle exposed during $\mathcal{A}$'s run and, in the win-event games, the challenger's construction computations, pre- and post-output (in the preimage game, the source encryption computing T^* as well as the post-output check)—and the N_{prim} direct primitive queries. The derived-adversary contract (Lemma 19) expresses the resulting simulator-world execution as the identically distributed random oracle execution of $\mathcal{B} = \mathcal{B}_{\text{derived}}(\mathcal{A})$, with $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and the stated construction-side inheritances, giving the first bound. The second follows by the monotonicity of ε_{G}. When G is parameterized by a challenge fixed before $\mathcal{A}$ runs—the everywhere-preimage target tag, or the standard preimage source X^*, whose tag $T^* = \mathsf{H}_{\mathsf{TW128}}(X^*)$ each coupled world computes—the coupling holds for each fixed challenge and the challenge-independent bound holds for the maximum or average over challenges defining $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{TW128}}(\mathcal{A})$. □

C Vectorized Kernel Details

This appendix details the optimized cores summarized in Section 8.1: their shared batching structure, chunk 0 fusion, instruction selection, register layout, remainder handling, and constant-time addressing.

The optimized paths share the same logical decomposition but differ in their batch widths, remainder strategy, and state writeback points:

Path	Cores and writeback
`amd64` AVX-512	Wide: one eight-state ZMM group. Remainder: masked 2–7 state AVX-512 core. Single duplex: AVX-512 permutation. Chunk 0: all active states are written back through `state8`.
`amd64` AVX2	Wide: two four-state YMM groups. Remainder: 2–7 states with dummy inactive lanes. Single duplex: general purpose register permutation. Chunk 0: all active states are written back through `state8`.
`arm64` NEON+SHA3	Wide: four two-state NEON pairs. Remainder: two-state pair core. Single duplex: NEON/SHA-3 permutation. Chunk 0: pair $(0, 1)$ is written back in the wide core, and both states are written back in the pair core.

Shared batching structure. The implementation separates the sequential root duplex from the independent leaf duplexes. The root duplex normally uses a single-duplex ($\times 1$) core, while complete leaf chunks are grouped into eight-instance batches. Every complete leaf chunk is exactly $\beta = 49\rho$ bytes, so all eight instances in a batch follow the same sequence of 49 full ρ-blocks: the first 48 close with MSG_MORE, and the last closes with MSG_LAST. A single public block index therefore drives the whole batch. The batched state is lane-major: for each of the 25 Keccak lanes, the kernel stores the corresponding 64-bit word of each active instance together in vector registers. Equivalently, for Keccak lane i the vector register holds $(S_0[i], \ldots, S_7[i])$ across the active duplex states, so one vector operation updates the same lane in several states.

Chunk 0 fusion. Chunk 0 is part of the root transcript, but its message phase has the same kernel-visible block schedule as a complete leaf chunk. The optimized paths exploit this by placing the post-AD root state into lane 0 of the first batched call and placing the first leaf chunks in the remaining lanes. When the message has at least seven complete leaf chunks, this fused call uses the $\times 8$ core; below that, it uses the same remainder machinery that drains incomplete batches. The output state from lane 0 is written back to the root duplex, which then absorbs the leaf tags produced by the same call.

Remainder and tail handling. Messages are otherwise decomposed into full eight-chunk batches plus a remainder. On `amd64`, a register-resident remainder kernel handles two to seven complete chunks in one call, reading each active chunk in place; on `arm64`, a 2-wide kernel drains the remainder two chunks at a time. A single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail use the single-duplex core. The `amd64` assembly chunk kernels handle the 48 full MSG_MORE blocks and leave the final MSG_LAST block plus leaf tag extraction to shared Go code, keeping byte-granular tail logic out of assembly. The `arm64` batched kernels process the complete chunk and store back the states needed for chunk 0 fusion: pair $(0, 1)$ in the $\times 8$ kernel, and both active states in the 2-wide kernel.

`amd64`. The `amd64` implementation provides AVX-512 and AVX2 cores and selects between them once at startup from an AVX-512 feature probe requiring OS support for YMM/ZMM state, `AVX512F`, and `AVX512VL`. The single-duplex permutation has two variants: a general-purpose-register core adapted from the standard library's KECCAK-p permutation (using the lane-complement trick so that χ requires no separate negation), and an AVX-512 variant; the root duplex uses whichever matches the selected core.

The eight-way AVX-512 core packs the lane-major state into 25 ZMM registers, one per lane with all eight instances in the register's eight 64-bit slots, and performs the permutation with no cross-lane shuffles: rotations use `VPROLQ`, and `VPTERNLOGQ` fuses the θ D-term formation and the χ and-not into single three-input logic instructions. The fused chunk kernel reads the eight chunks with contiguous loads, transposes each rate block into the lane-major layout in registers (`VPUNPCKLQDQ`/`VPUNPCKHQDQ` and `VSHUFI64X2`), absorbs on the lane-major state, and transposes each output block back for a contiguous store. Since $\rho = 167 = 20 \cdot 8 + 7$, the first 20 rate lanes use this transposed path and only the trailing 7 bytes in lane 20 use a masked gather. These sequential accesses are prefetcher-friendly, whereas per-lane gathers and scatters at the chunk stride β (`VPGATHERQQ`/`VPSCATTERQQ`) would stall on cold memory and cap the long-message rate.

The register-resident kernel that drains a two-to-seven-chunk AVX-512 remainder instead uses that per-lane gather/scatter form under a mask, reading the active chunks in place: the transpose's shuffle work is constant regardless of the chunk count and would be spent partly on inactive lanes, whereas the gathers scale with the active element count.

The eight-way AVX2 core, lacking 512-bit registers, runs the eight instances as two groups of four (four 64-bit lanes per YMM register), rotating by shift-or and computing χ with `VPANDN`. The AVX2 path drains remainders with a variant of the grouped-by-four kernels that aims each inactive instance's loads and stores at a dummy block inside the kernel frame with a zero advance stride, so inactive instances touch no memory beyond the remainder and the hot loop stays branch-free.

`arm64`. The `arm64` path uses NEON plus the Armv8.2 SHA-3 extension (`FEAT_SHA3`), available on Apple Silicon and on Neoverse V1/V2 and N2. The permutation is expressed with the extension's fused instructions: `EOR3` (three-input XOR) for the θ column parities, `RAX1` (rotate-by-one and XOR) for the θ D-terms, `XAR` (XOR then rotate) to fuse the θ accumulation with the ρ rotation, and `BCAX` (bit-clear and XOR) for χ. The single-duplex core stores each Keccak lane in one 64-bit `D` lane across the 25 vector registers. The eight-way core packs two instances per 128-bit register and processes the batch of eight as four such pairs, using `ZIP1` and duplicating moves to interleave and extract the two instances of each lane in the fused chunk kernel. The same pair machinery backs a 2-wide kernel that drains a leaf-chunk remainder two chunks at a time—one instance per 64-bit half of a 128-bit register—at roughly twice the single-duplex per-chunk rate. Both batched kernels store the permutation state back into the batch after the closing block—the $\times 8$ kernel for its first pair, the 2-wide kernel for both instances—which is the writeback that permits the root duplex to occupy lane 0.

Constant-time addressing. The optimized kernels contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number. The AVX-512 steady-state transpose uses contiguous loads and stores; the remaining gathers and scatters, for the partial rate lane and register-resident remainder kernel, use fixed public chunk strides under masks determined by the public chunk count. The AVX2 remainder kernel redirects inactive instances to a dummy block in its stack frame with a zero advance stride, again selected only by the public chunk count. The `arm64` pair kernels use fixed lane interleaving and extraction patterns. None of these addresses depends on the key, nonce, plaintext, or decrypted plaintext.

D Test Vectors

The following are selected known-answer test vectors for TW128 encryption, as specified in § 3. The complete vector set, including full byte strings for the long cases and the regeneration harness, is maintained with the reference implementation at `https://github.com/codahale/treewrap`.

All byte strings are written in hexadecimal. For each vector we list the key K, nonce U, associated data A, message M, ciphertext C, and tag T; the sealed output of encryption is $C \parallel T$. The empty string is written ε. The parameters used below are $\rho = 167$ bytes, $\beta = 8183$ bytes, and 32-byte keys, nonces, and tags. The leaf index is encoded as a little-endian 64-bit integer and the phase suffixes are appended least significant bit first.

To keep the longer vectors compact, any value exceeding 32 bytes is given by its length in bytes together with its SHA-256 digest rather than in full. For generated associated data and message inputs, byte i is defined as $x_i = (17i + 31) \bmod 256$ for $i = 0, 1, \ldots$, so each such input is determined by its length alone and can be reconstructed and checked against the stated digest; a ciphertext given by its length and digest can likewise be checked by encrypting the corresponding message and hashing the result.

Vector 1. Empty associated data and empty message: the associated data and aggregation phases are both elided, and the root tag is emitted by the single closing message call with no leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	ε
C	ε
T	`da65bbda0b20b1e936f5326458166633d1e5a3f7aebc0b318d24b051a4efa697`

Vector 2. Multi-block associated data with an empty message: the associated data spans two rate blocks and no message bytes are processed.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2797 A	168 bytes, SHA-256 4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a
M	ε
C	ε
T	2093878418f3fca96ad3a8d1748a9be4d2a8b518e917d988fd703dcb7bf72173

Vector 3. Empty associated data and a single-block message: the root message phase only, no leaves.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
2798 U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
A	ε
M	1f30415263748596a7b8c9daebfc0d1e2f405162738495a6b7c8d9eafb0c1d2e
C	676a1441e20daea0f1ee4571ee4543842c946d59e0d3df6f1dd2f92dc4f876aa
T	1873332d359ff2fe5197488224af7bf6ad821875fdbd964100de424aa77342be

Vector 4. Empty associated data and a two-block message: the root keystream is chained across two rate blocks.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2799 A	ε
M	168 bytes, SHA-256 4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a
C	168 bytes, SHA-256 df3f0a98a91a69ed474edbc52e0dc1d8e54bfd6fc514966ce7b878d48be0bf9a
T	aab2413074a202cfadd5eb784663b1831eb330025972d163d03e9ed5a964b765

Vector 5. A message of exactly one chunk: the root is full, there is still no leaf, and the aggregation phase is elided so the closing root message call emits the tag.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2800 A	ε
M	8183 bytes, SHA-256 6532a370a4ab6b5f4094dd8a6016512ab8e8416abe910bc1a0b532979224151d
C	8183 bytes, SHA-256 8d58f8aa3413ea0ba1b8cc3ed4e9a334521d8dafe9694a0661fb8251049907e9
T	c2201637f1116c1303b8982622756770d23acb85384f4658fcb63c37e4b629dd

Vector 6. A message of one chunk and one further byte: one root chunk and one leaf, exercising leaf tag aggregation.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2801 A	ε
M	8184 bytes, SHA-256 c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7
C	8184 bytes, SHA-256 ef1b2f7607ade503ab497fc61e62cef2b44c6328fa06e499718eb7da5fdd04d3
T	c96bea48e07a8431fd19721ffbe8afe6a73047f96d953e3281f10126eeca06a1

Vector 7. A multi-leaf message of two chunks and one further byte: the full tree path with two leaves.

K	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
U	202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f
2802 A	ε
M	16367 bytes, SHA-256 17e31f3700d0602aab8a686024ecc05b8192e80a531483c67dd90d5a26557c5e
C	16367 bytes, SHA-256 6d83d8fbdc2a7c2ef590d29a36527a45019e50d1f8c8db037959081685a46908
T	cdc2e50ac5380419f06772a2f3af2146050bd0340b35f61b67f839351767804e

Vector 8. Multi-block associated data with a short message: the associated data blocks dominate the cost.

	K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
	U	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
2803	A	337 bytes, SHA-256
		`04ca7b24930bc6b5fe8b95ff1eb0933907657c1283cc3a7e69c5f7f02220f281`
	M	`1f30415263748596a7b8c9daebfc0d1e`
	C	`0dd615a0a05ed82a78c3999b9e80cbed`
	T	`22e4e6725f548ad0660041d66cf2f18dc45c69b447cdaae1f147d33a3b6aa1a9`

Vector 9. An all-ones key and nonce with a multi-chunk message.

	K	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	U	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
	A	168 bytes, SHA-256
2804		`4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
	M	8184 bytes, SHA-256
		`c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7`
	C	8184 bytes, SHA-256
		`49818c320ab282296639798d840cbd60751e1623f449dcbf9423371d44145dfb`
	T	`310c9fff20e9651f8ed47eed9bb182a086ed1af1b3f1799e35307a4139865fc7`